



PES
UNIVERSITY

COMPUTER NETWORKS

TEAM NETWORKS

Department of Computer Science and Engineering

COMPUTER NETWORKS

Application Layer

Department of Computer Science and Engineering

Unit – 2 Application Layer

2.3 The Domain Name System

2.4 P2P Applications

2.5 Socket Programming with TCP & UDP

2.6 Other Application Layer Protocols

people: many identifiers:

- SSN, name, passport #

Internet hosts, routers:

- IP address (32 bit) - used for addressing datagrams
- “name”, e.g., cs.umass.edu - used by humans

Q: how to map between IP address and name, and vice versa ?

Domain Name System:

- *distributed database*
implemented in hierarchy of many *name servers*
- *application-layer protocol:*
hosts, name servers
communicate to *resolve* names
(address/name translation)
 - note: core Internet function,
*implemented as
application-layer protocol*
 - complexity at network’s
“edge”

www.abc.example.com -> Canonical Host Name

www.example.com -> Alias Name

DNS services

- hostname to IP address translation
- host aliasing
 - canonical, alias names
- mail server aliasing
- load distribution
 - replicated Web servers: many IP addresses correspond to one name

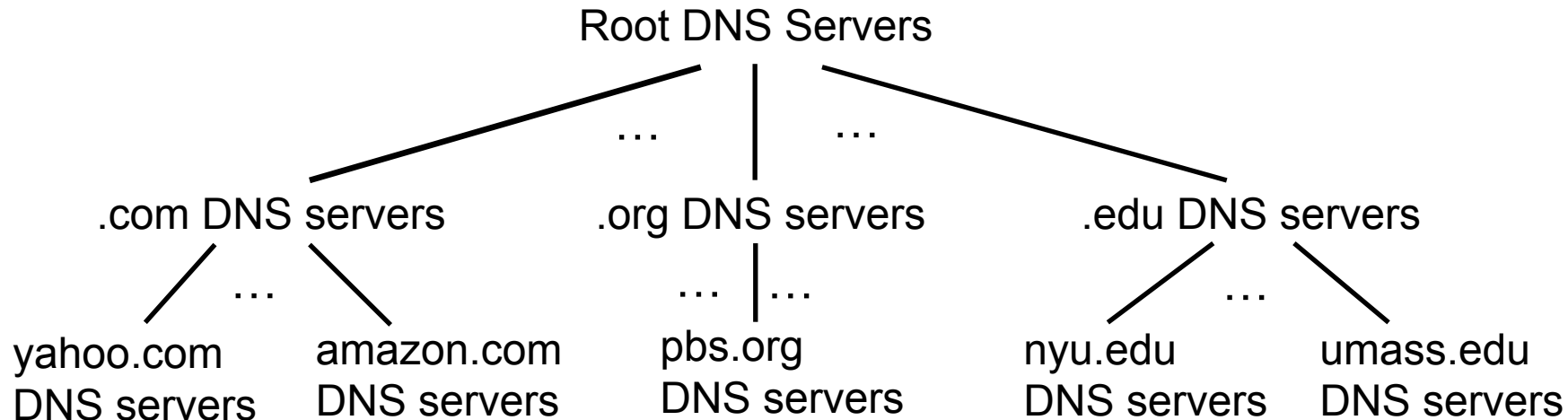
Q: Why not centralize DNS?

- single point of failure
- traffic volume
- distant centralized database
- maintenance

www.abc.example.com ->
Canonical Host Name
bob@example.com ->
Alias Name

A: doesn't scale!

- Comcast DNS servers alone: 600B DNS queries per day
- Akamai DNS servers alone: 2.2T DNS queries/day



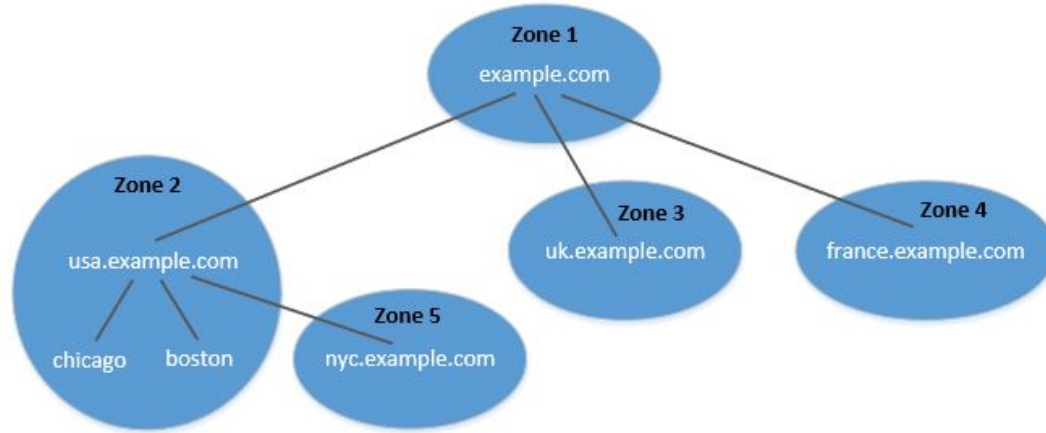
Root

Top Level Domain

Authoritative

Client wants IP address for `www.amazon.com`; 1st approximation:

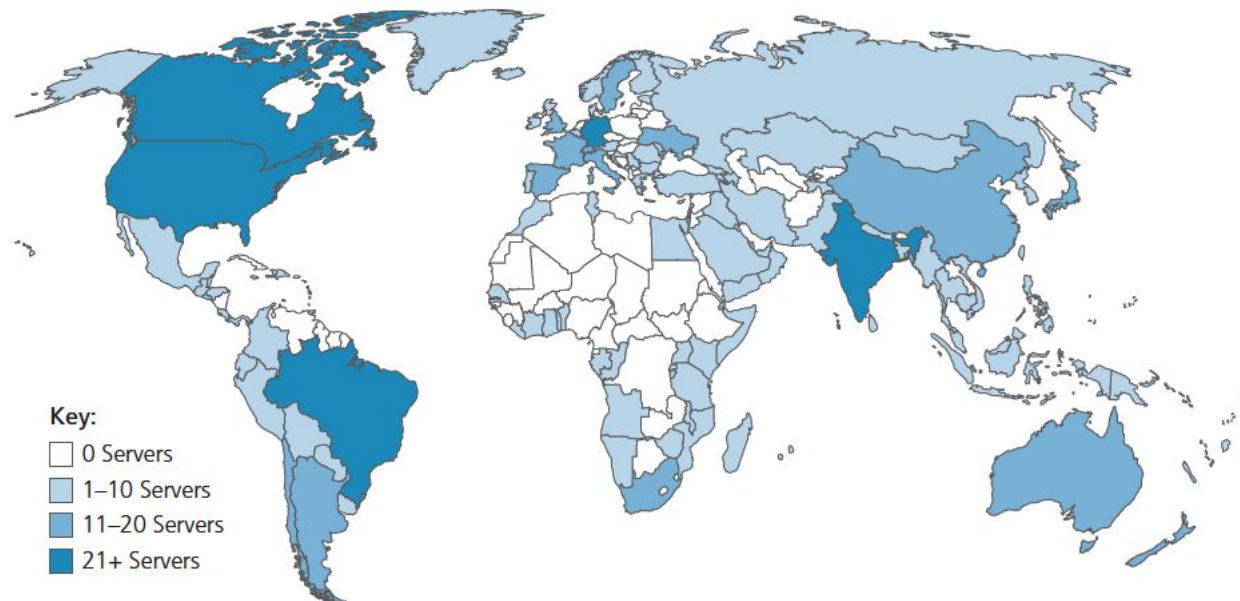
- client queries root server to find .com DNS server
- client queries .com DNS server to get amazon.com DNS server
- client queries amazon.com DNS server to get IP address for `www.amazon.com`



- DNS is organized according to zones.
 - A zone groups contiguous domains and subdomains on the domain tree.
 - Assign management authority to an entity.
-
- The tree structure depicts subdomains within example.com domain.
 - Multiple DNS zones one for each country. The zone keeps records of who the authority is for each of its subdomains.
 - The zone for example.com contains only the DNS records for the hostnames that do not belong to any subdomain like mail.example.com

- official, contact-of-last-resort by name servers that can not resolve name
- *incredibly important* Internet function
 - Internet couldn't function without it!
 - DNSSEC – provides security (authentication and message integrity)
- ICANN (Internet Corporation for Assigned Names and Numbers) manages root DNS domain

13 logical root name “servers”
worldwide each “server” replicated
many times (~200 servers in US)

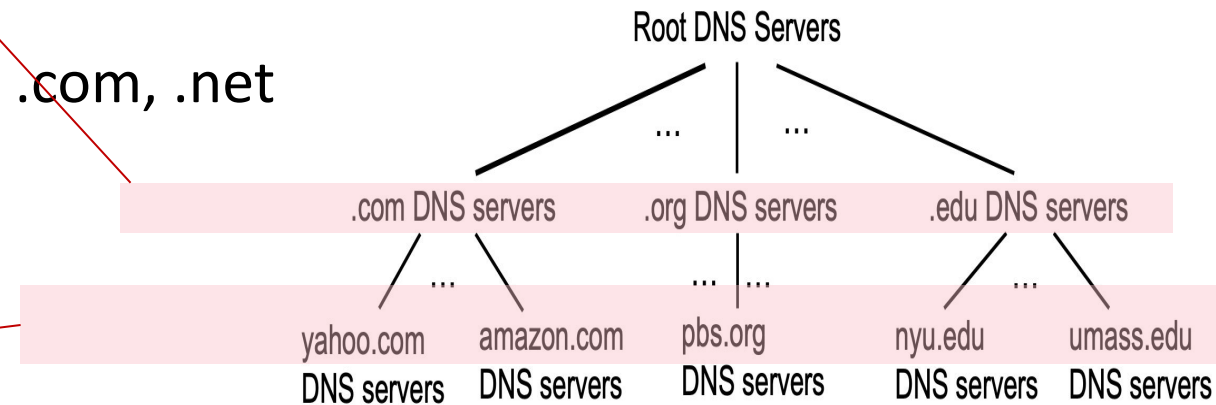


Top-Level Domain (TLD) servers:

- responsible for .com, .org, .net, .edu, .aero, .jobs, .museums, and all top-level country domains, e.g.: .cn, .uk, .fr, .ca, .jp
- Network Solutions: authoritative registry for .com, .net TLD
- Educause: .edu TLD

Authoritative DNS servers:

- organization's own DNS server(s), providing authoritative hostname to IP mappings for organization's named hosts
- can be maintained by organization or service provider



- does not strictly belong to hierarchy
- each ISP (residential ISP, company, university) has local DNS Server
 - also called “default name server”
 - MacOS: `% scutil --dns`
 - Windows: `>ipconfig /all`
- when host makes DNS query, query is sent to its local DNS server

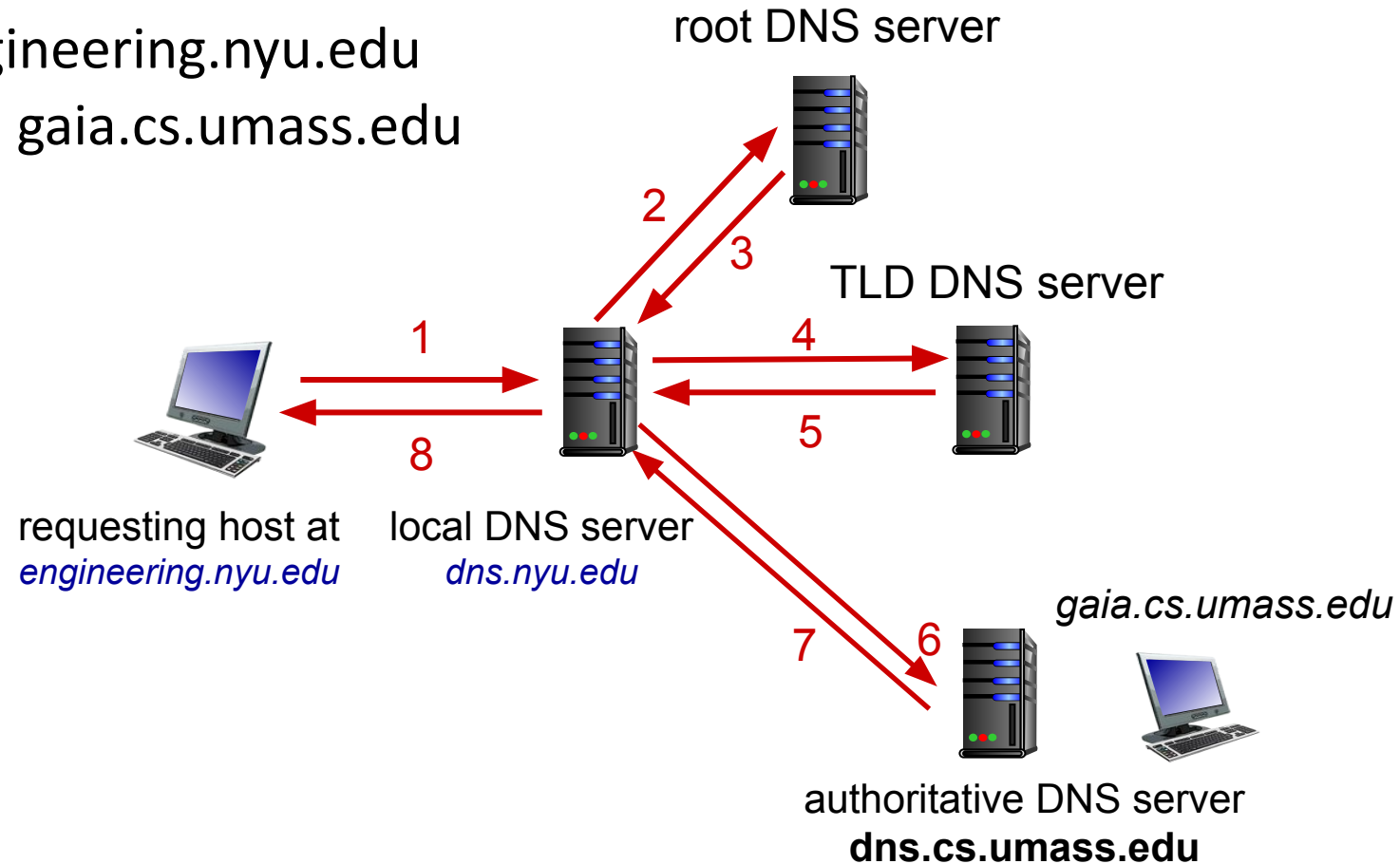
Local DNS server returns reply, answering:

 - From its local cache of recent name-to-address translation pairs (but may be out of date!)
 - acts as proxy, forwards query into hierarchy

Example: host at `engineering.nyu.edu` wants IP address for `gaia.cs.umass.edu`

Iterated query:

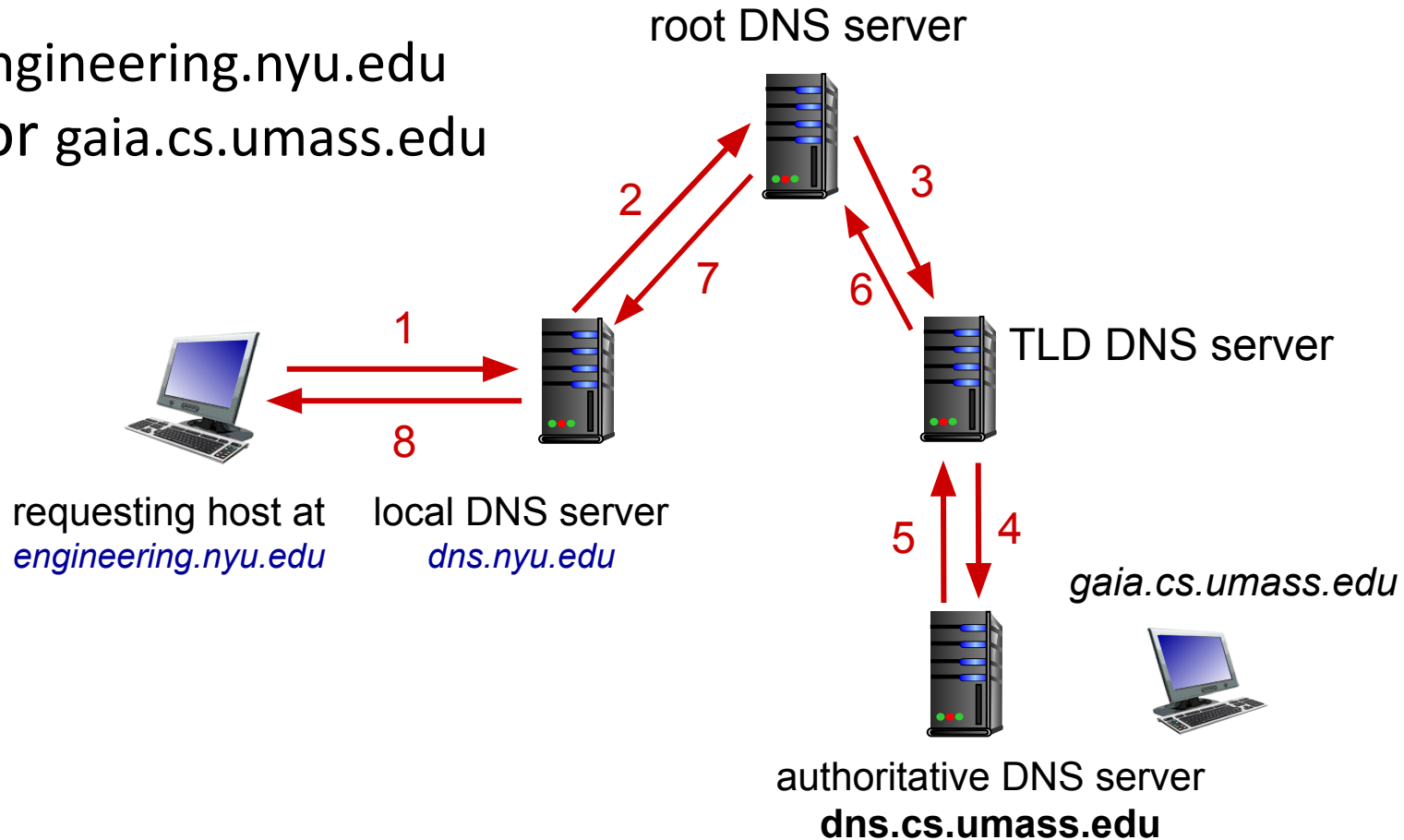
- contacted server replies with name of server to contact
- “I don’t know this name, but ask this server”



Example: host at engineering.nyu.edu wants IP address for gaia.cs.umass.edu

Recursive query:

- puts burden of name resolution on contacted name server
- heavy load at upper levels of hierarchy?



- Suppose that a host **apricot.nyu.edu** queries **dns.nyu.edu** for the IP address for the hostname **cnn.com**. After an hour later, another NYU host, say, **kiwi.nyu.edu**, also queries **dns.nyu.edu**.
- once (any) name server learns mapping, it *cached* mapping and immediately returns a cached mapping in response to a query
 - Caching improves response time
 - cache entries timeout (disappear) after some time (TTL)
 - TLD servers typically cached in local name servers
 - thus root name servers not often visited
- cached entries may be *out-of-date* (best-effort name-to-address translation!)
 - if name host changes IP address, may not be known Internet-wide until all TTLs expire!
- update/notify mechanisms proposed IETF standard
 - RFC 2136

DNS: distributed database storing resource records (RR)

RR format: (name, value, type, ttl)

type=A

- name is hostname
- value is IP address

relayl.bar.foo.com, 145.37.93.126, A

type=NS

- name is domain (e.g., foo.com)
- value is hostname of authoritative name server for this domain

foo.com, dns.foo.com, NS

type=CNAME

- name is alias name for some “canonical” (the real) name
- www.ibm.com is really servereast.backup2.ibm.com
- value is canonical name

ibm.com, servereast.backup2.ibm.com, CNAME

type=MX

- value is canonical name of a mailserver associated with alias hostname name

example.com, mail.example.com, MX

COMPUTER NETWORKS

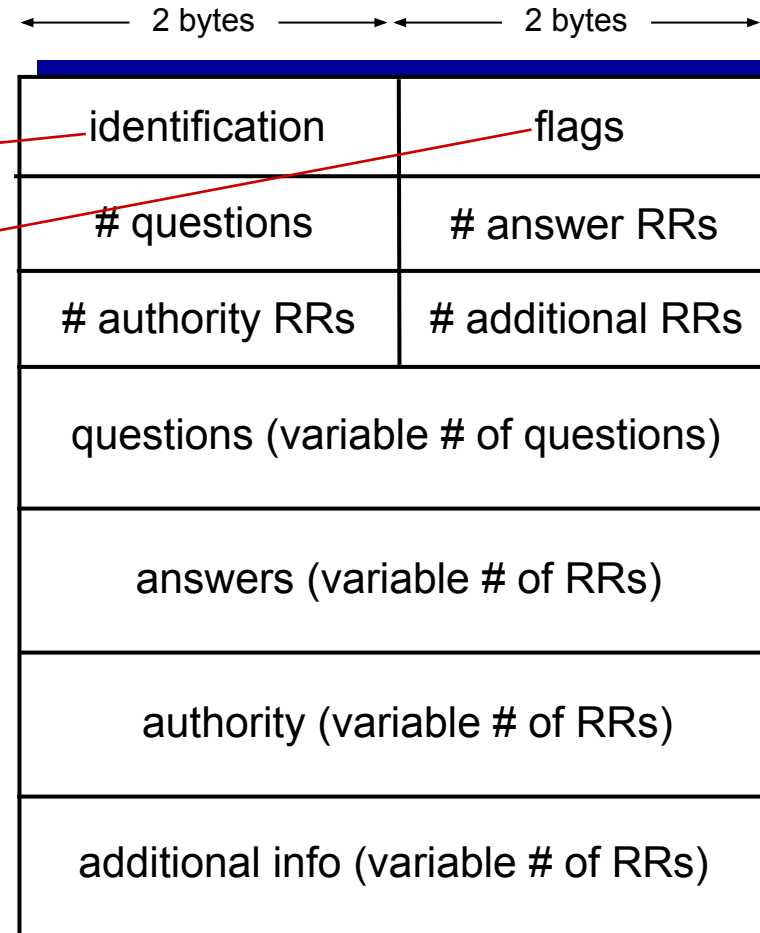
DNS Protocol Messages



DNS *query* and *reply* messages, both have same *format*:

message header:

- **identification**: 16 bit # for query, reply to query uses same #
- **flags**:
 - query or reply (1-bit)
 - recursion desired
 - recursion available
 - reply is authoritative



12 bytes

Name, type fields for a query

RRs in response to query

Records for authoritative servers

Additional "helpful" info that may be used

COMPUTER NETWORKS

DNS Protocol Messages



DNS *query* and *reply* messages, both have same *format*:

← 2 bytes → ← 2 bytes →

identification	flags
# questions	# answer RRs
# authority RRs	# additional RRs
questions (variable # of questions)	
answers (variable # of RRs)	
authority (variable # of RRs)	
additional info (variable # of RRs)	

name, type fields for a query

RRs in response to query

records for authoritative servers

additional “helpful” info that
may be used

Type (for example, A, NS, CNAME, and MX), the Value, and the TTL.

Directly send the query to this server.

```
seed@ubuntu:~$ dig @a.root-servers.net www.example.net
```

(Only a portion of the reply is shown here)

```
;; QUESTION SECTION:
```

```
;www.example.net.          IN      A
```

```
;; AUTHORITY SECTION:
```

```
net.      172800  IN      NS      m.gtld-servers.net.  
net.      172800  IN      NS      l.gtld-servers.net.  
net.      172800  IN      NS      k.gtld-servers.net.
```

```
;; ADDITIONAL SECTION:
```

```
m.gtld-servers.net.  172800  IN      A      192.55.83.30  
l.gtld-servers.net.  172800  IN      A      192.41.162.30  
k.gtld-servers.net.  172800  IN      A      192.52.178.30
```

No answer (the root does not know the answer)

Go ask them!

COMPUTER NETWORKS

Steps 2-3: Ask .net & example.net servers



```
seed@ubuntu:~$ dig @m.gtld-servers.net www.example.net

;; QUESTION SECTION:
;www.example.net.                IN      A

;; AUTHORITY SECTION:
example.net.                     172800  IN      NS      a.iana-servers.net.
example.net.                     172800  IN      NS      b.iana-servers.net.

;; ADDITIONAL SECTION:
a.iana-servers.net.             172800  IN      A          199.43.132.53
b.iana-servers.net.             172800  IN      A          199.43.133.53
```

Go ask them!

```
seed@ubuntu:$ dig @a.iana-servers.net www.example.net

;; QUESTION SECTION:
;www.example.net.                IN      A

;; ANSWER SECTION:
www.example.net.                86400   IN      A          93.184.216.34
```

- Ask an example.net nameservers.

Finally got the answer

Example: new startup “Network Utopia”

- register name **networkutopia.com** at *DNS registrar* (e.g., Network Solutions)
 - provide names, IP addresses of authoritative name server (primary and secondary)
 - registrar inserts NS, A RRs into .com TLD server:
(networkutopia.com, dns1.networkutopia.com, NS)
(dns1.networkutopia.com, 212.212.212.1, A)
- create authoritative server locally with IP address 212.212.212.1
 - type A record for www.networkutopia.com
 - type MX record for networkutopia.com

COMPUTER NETWORKS

DNS Request - Wireshark Packet Capture



Microsoft: \Device\NPF_{483C83F4-DCBA-4863-B523-3C4E1B03D06F} [Wireshark 1.8.5 (SVN Rev 47350 from /trunk-1.8)]

File Edit View Go Capture Analyze Statistics Telephony Tools Internals Help

Filter: `ip.addr == 10.36.41.43` Expression... Clear Apply Save

No.	Time	Source	Destination	Protocol	Length	Info
13	13:51:23.477657000	173.194.43.37	10.36.41.43	TCP	54	https > 62364 [FIN, ACK] Seq=103 Ack=2 win=63784 Len=0
14	13:51:23.477694000	10.36.41.43	173.194.43.37	TCP	54	62364 > https [ACK] Seq=3 Ack=104 win=16478 Len=0
15	13:51:23.491240000	173.194.43.37	10.36.41.43	TCP	54	https > 62364 [ACK] Seq=104 Ack=3 win=63784 Len=0
16	13:51:27.041610000	10.36.41.43	10.40.4.44	DNS	72	standard query 0x9f7d A www.ietf.org
17	13:51:27.160178000	10.40.4.44	10.36.41.43	DNS	473	standard query response 0x9f7d A 64.170.98.30
18	13:51:27.166692000	10.36.41.43	10.40.4.44	DNS	88	standard query 0x6028 A tunnel.cfw.trustedsource.org
19	13:51:27.167744000	10.40.4.44	10.36.41.43	DNS	104	standard query response 0x6028 A 8.21.161.7
20	13:51:27.180583000	10.36.41.43	8.21.161.7	TCP	62	62382 > https [SYN] Seq=0 win=8192 Len=0 MSS=1460 SACK_PERM=1
21	13:51:27.258985000	8.21.161.7	10.36.41.43	TCP	62	https > 62382 [SYN, ACK] Seq=0 Ack=1 win=5840 Len=0 MSS=1460 SACK
22	13:51:27.259111000	10.36.41.43	8.21.161.7	TCP	54	62382 > https [ACK] Seq=1 Ack=1 win=17520 Len=0
23	13:51:27.259472000	10.36.41.43	8.21.161.7	TLSv1	149	Client Hello
24	13:51:27.336962000	8.21.161.7	10.36.41.43	TCP	54	https > 62382 [ACK] Seq=1 Ack=96 win=5840 Len=0
25	13:51:27.337735000	8.21.161.7	10.36.41.43	TLSv1	1446	Server Hello, Certificate, Certificate Request, Server Hello Done
26	13:51:27.340425000	10.36.41.43	8.21.161.7	TLSv1	1005	Certificate, Client Key Exchange, Certificate verify, Change Ciph
27	13:51:27.422036000	8.21.161.7	10.36.41.43	TLSv1	113	Change cipher spec, Encrypted Handshake Message
28	13:51:27.425726000	10.36.41.43	8.21.161.7	TLSv1	395	Application Data
29	13:51:27.502692000	8.21.161.7	10.36.41.43	TLSv1	192	Application Data, Application Data

Domain Name System (query)

[\[Response In: 17\]](#)

Transaction ID: 0x9f7d

Flags: 0x0100 Standard query

Questions: 1

Answer RRs: 0

Authority RRs: 0

Additional RRs: 0

Queries

www.ietf.org: type A, class IN

0000 00 1e 17 4c 01 3f cc a1 78 0a de 0b 08 00 45 00 ...La?... x..k..E.
0010 00 3a 47 7d 00 00 80 11 b1 93 0a 24 29 2b 0a 28 ...G}.... ..\$)+.C

No.	Time	Source	Destination	Protocol	Length	Info
13	13:51:23.477657000	173.194.43.37	10.36.41.43	TCP	54	https > 62364 [FIN, ACK] Seq=103 Ack=2 win=63784 Len=0
14	13:51:23.477694000	10.36.41.43	173.194.43.37	TCP	54	62364 > https [ACK] Seq=3 Ack=104 win=16478 Len=0
15	13:51:23.491240000	173.194.43.37	10.36.41.43	TCP	54	https > 62364 [ACK] Seq=104 Ack=3 win=63784 Len=0
16	13:51:27.041610000	10.36.41.43	10.40.4.44	DNS	72	Standard query 0x9f7d A www.ietf.org
17	13:51:27.160178000	10.40.4.44	10.36.41.43	DNS	473	Standard query response 0x9f7d A 64.170.98.30
18	13:51:27.166692000	10.36.41.43	10.40.4.44	DNS	88	Standard query 0x6028 A tunnel.cfw.trustedsource.org
19	13:51:27.167744000	10.40.4.44	10.36.41.43	DNS	104	Standard query response 0x6028 A 8.21.161.7
20	13:51:27.180583000	10.36.41.43	8.21.161.7	TCP	62	62382 > https [SYN] Seq=0 win=8192 Len=0 MSS=1460 SACK_PER
21	13:51:27.258985000	8.21.161.7	10.36.41.43	TCP	62	https > 62382 [SYN, ACK] Seq=0 Ack=1 win=5840 Len=0 MSS=14

Flags: 0x8180 Standard query response, No error

Questions: 1

Answer RRs: 1

Authority RRs: 6

Additional RRs: 11

Queries

www.ietf.org: type A, class IN

Name: www.ietf.org

Type: A (Host address)

Answers

www.ietf.org: type A, class IN, addr 64.170.98.30

Authoritative nameservers

ietf.org: type NS, class IN, ns ns1.yyz1.afiliast.net

ietf.org: type NS, class IN, ns ns0.ietf.org

ietf.org: type NS, class IN, ns ns1.sea1.afiliast.net

ietf.org: type NS, class IN, ns ns1.ams1.afiliast.net

ietf.org: type NS, class IN, ns ns1.mia1.afiliast.net

```
000  cc af 78 0a de 6b 00 1e f7 4c 61 3f 08 00 45 00  ..x..k.. .La?..E.
010  01 cb 63 b4 40 00 7e 11 55 cb 0a 28 04 2c 0a 24  ..c.@.~. U..(..$.
020  29 2b 00 35 c3 d5 01 b7 1a 58 9f 7d 81 80 00 01  )+.5.... .X.}....
030  00 01 00 06 00 0b 03 77 77 77 04 69 65 74 66 03  .....w ww.ietf.
040  6f 72 67 00 00 01 00 01 c0 0c 00 01 00 01 00 00  org.....
050  07 08 00 04 40 aa 62 1e c0 10 00 02 00 01 00 00  ....@.b. ....
060  07 08 00 04 40 aa 62 1e c0 10 00 02 00 01 00 00  ....@.b. ....
070  69 6c 69 61 73 2d 6e 73 74 04 69 6e 66 6f 00 c0  ilias-ns t'info
```

- DNS (Domain Name System) – Explained – <https://youtu.be/JkEYOt08-rU>
- How a DNS Server (Domain Name System) works – <https://youtu.be/rdVPfIECed8>
- Wireshark Lab: DNS v7.0 – http://www-net.cs.umass.edu/wireshark-labs/Wireshark_DNS_v7.0.pdf



 Thank You
For Your Attention

Unit – 2 Application Layer

2.3 The Domain Name System

2.4 P2P Applications

2.5 Socket Programming with TCP & UDP

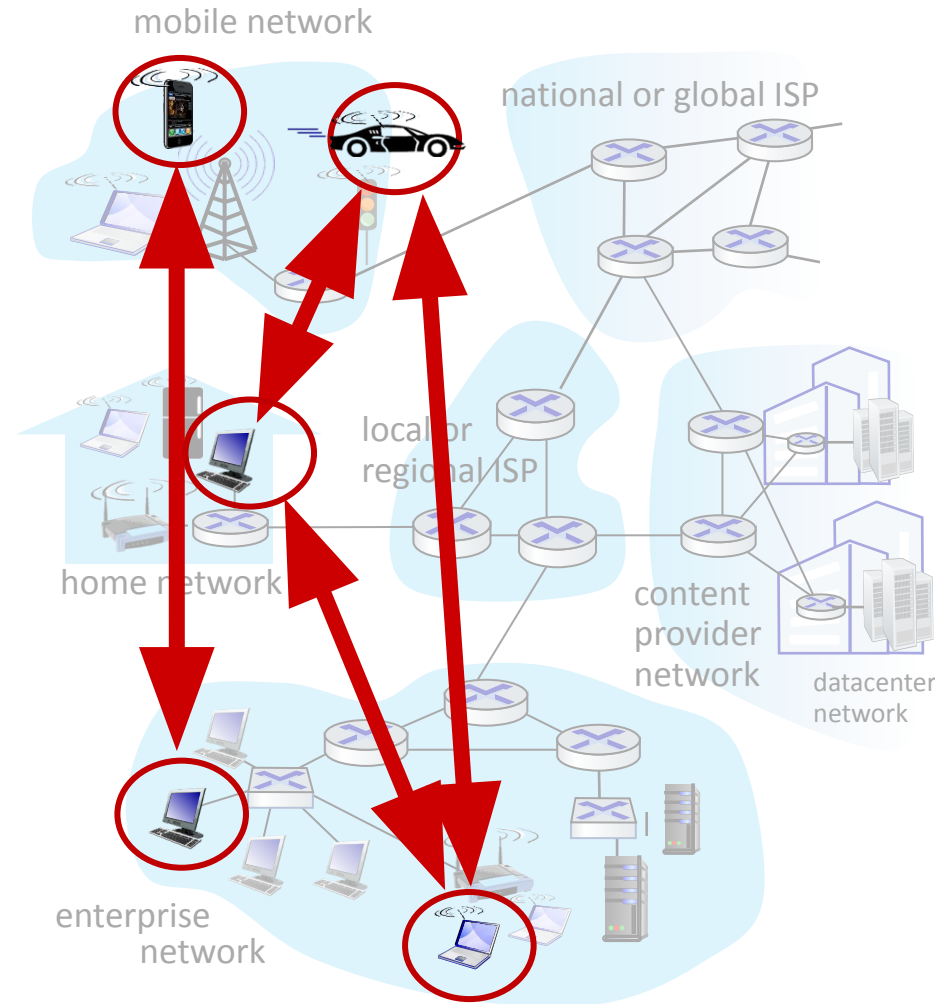
2.6 Other Application Layer Protocols



COMPUTER NETWORKS

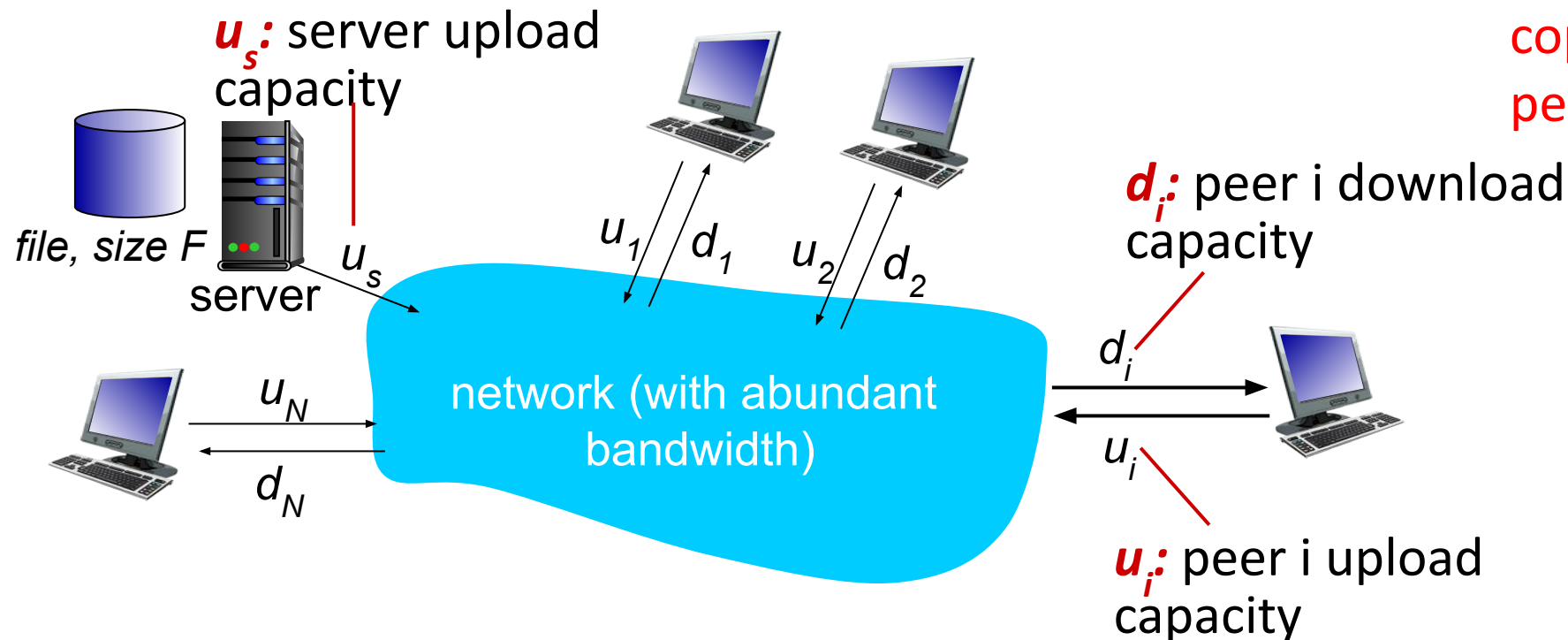
Peer-to-peer (P2P) architecture

- *no* always-on server
- arbitrary end systems directly communicate
- peers request service from other peers, provide service in return to other peers
 - *self scalability* – new peers bring new service capacity, and new service demands
- peers are intermittently connected and change IP addresses
 - complex management
- examples: P2P file sharing (BitTorrent), media streaming (Spotify), VoIP (Skype)



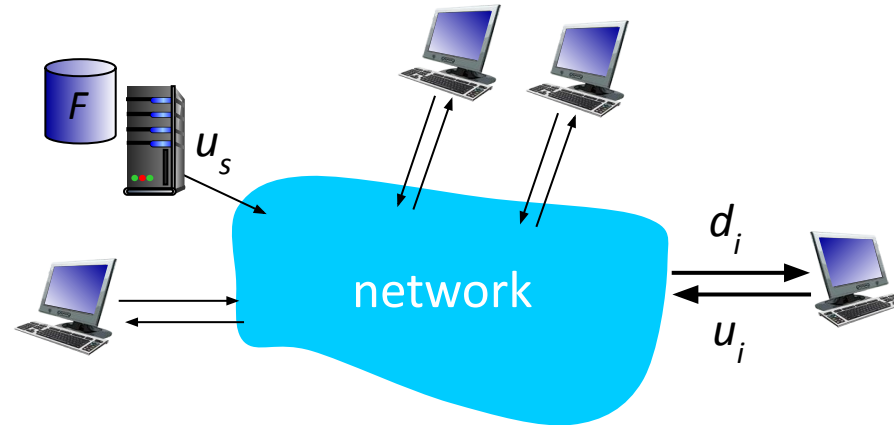
Q: how much time to distribute file (size F) from one server to N peers?

- peer upload/download capacity is limited resource



The **distribution time** is the time it takes to get a copy of the file to all N peers.

- **server transmission:** must sequentially send (upload) N file copies:
 - time to send one copy: F/u_s
 - time to send N copies: NF/u_s
- **client:** each client must download file copy
 - d_{min} = min client download rate =
i.e., $d_{min} = \min \{d_1, d_2, \dots, d_N\}$
 - min client download time: F/d_{min}



time to distribute F
to N clients using
client-server approach $D_{c-s} > \max\{NF/u_s, F/d_{min}\}$

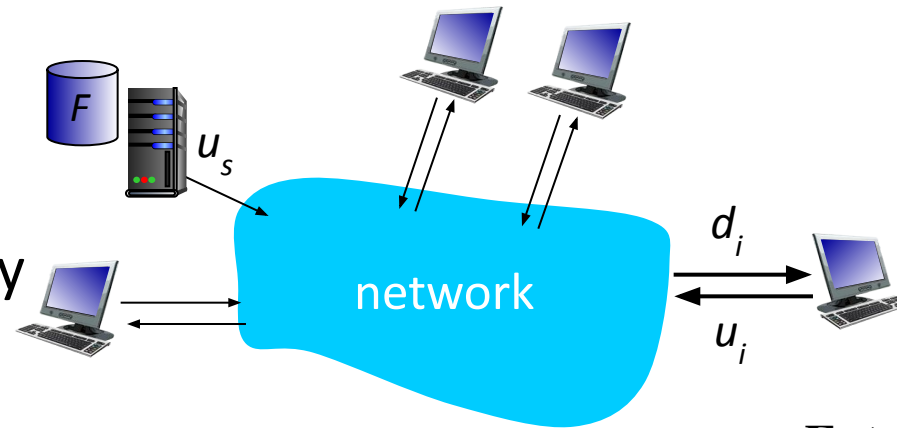
increases linearly in N

- **server transmission:** must upload at least one copy:

- time to send one copy: F/u_s

- **client:** each client must download file copy

- min client download time: F/d_{min}



- **clients:** as aggregate must download NF bits

- max upload rate (limiting max download rate) is

$$u_s + \sum u_i$$

Total upload capacity of the system as a whole

Eqtn - provides a lower bound for the minimum distribution time for the P2P architecture.

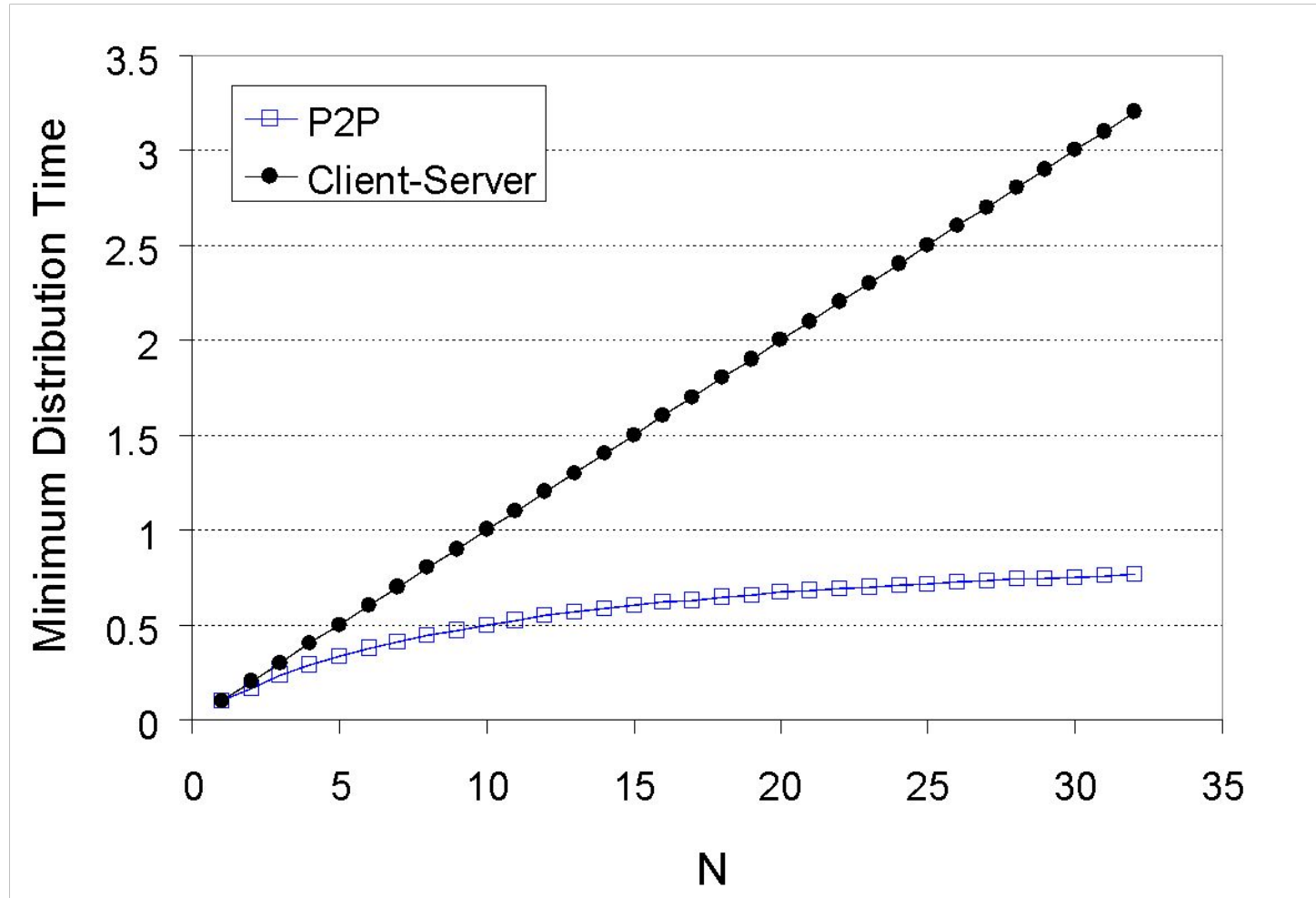
time to distribute F
to N clients using
P2P approach

$$D_{P2P} > \max\{F/u_s, F/d_{min}, NF/(u_s + \sum u_i)\}$$

increases linearly in N ...

... but so does this, as each peer brings service capacity

Client (all peers) upload rate = u , $F/u = 1$ hour, $u_s = 10u$, $d_{min} \geq u_s$

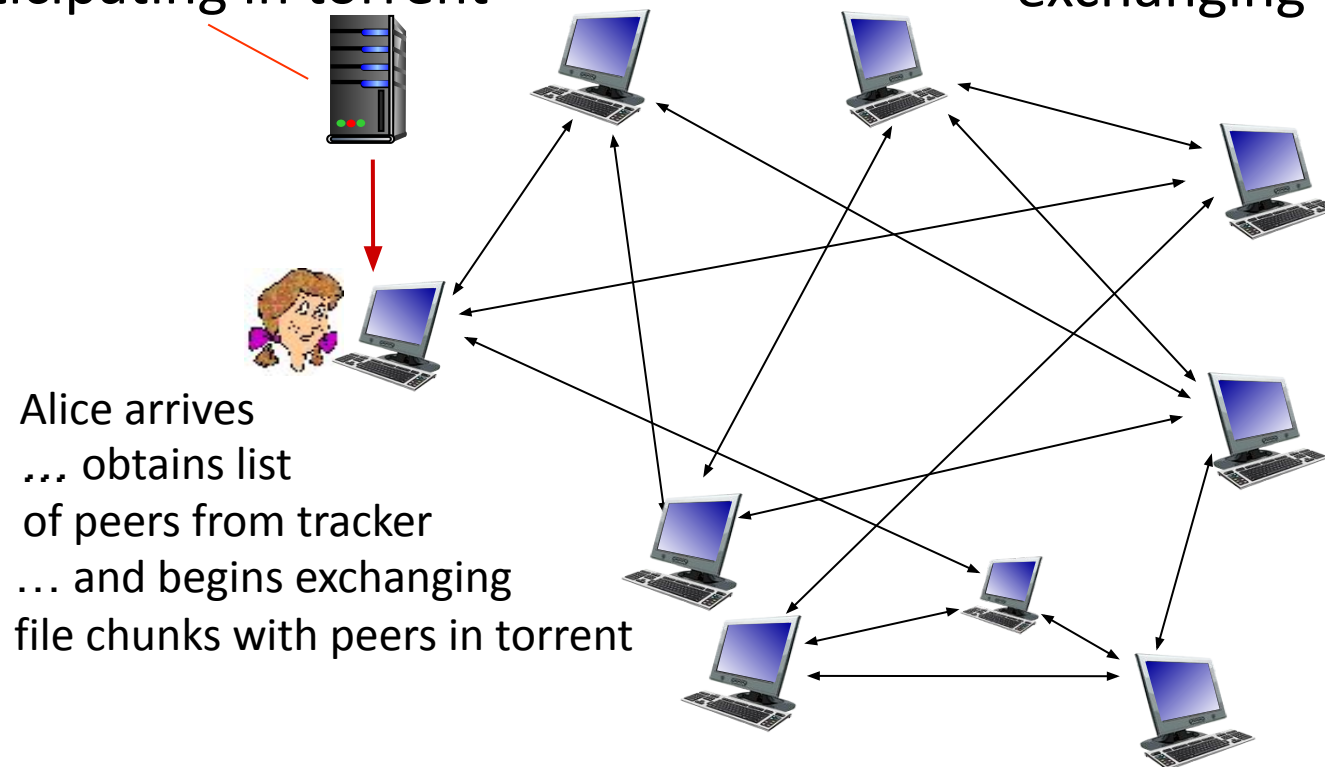


- A peer can transmit the entire file in one hour.
- The server transmission rate is 10 times the peer upload rate.
- Peer download rates are set large enough so as not to have an effect.

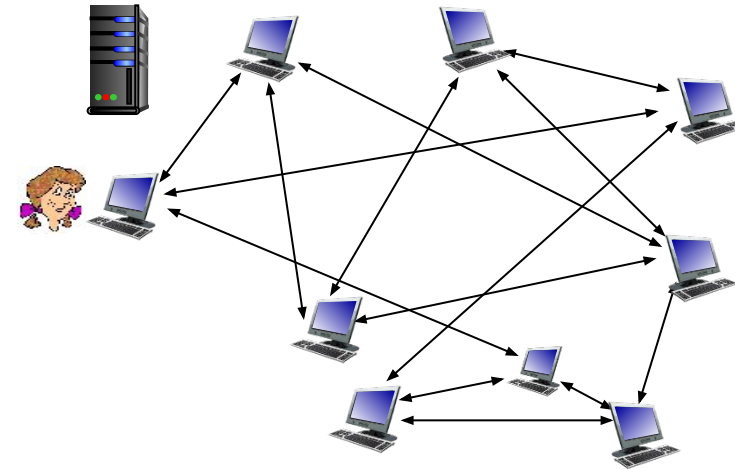
- file divided into 256Kb chunks
- peers in torrent send/receive file chunks

tracker: tracks peers participating in torrent

torrent: group of peers exchanging chunks of a file



- peer joining torrent:
 - has no chunks, but will accumulate them over time from other peers
 - registers with tracker to get list of peers, connects to subset of peers (“neighbors”)
- while downloading, peer uploads chunks to other peers
- peer may change peers with whom it exchanges chunks
- *churn*: peers may come and go
- once peer has entire file, it may (selfishly) leave or (altruistically) remain in torrent



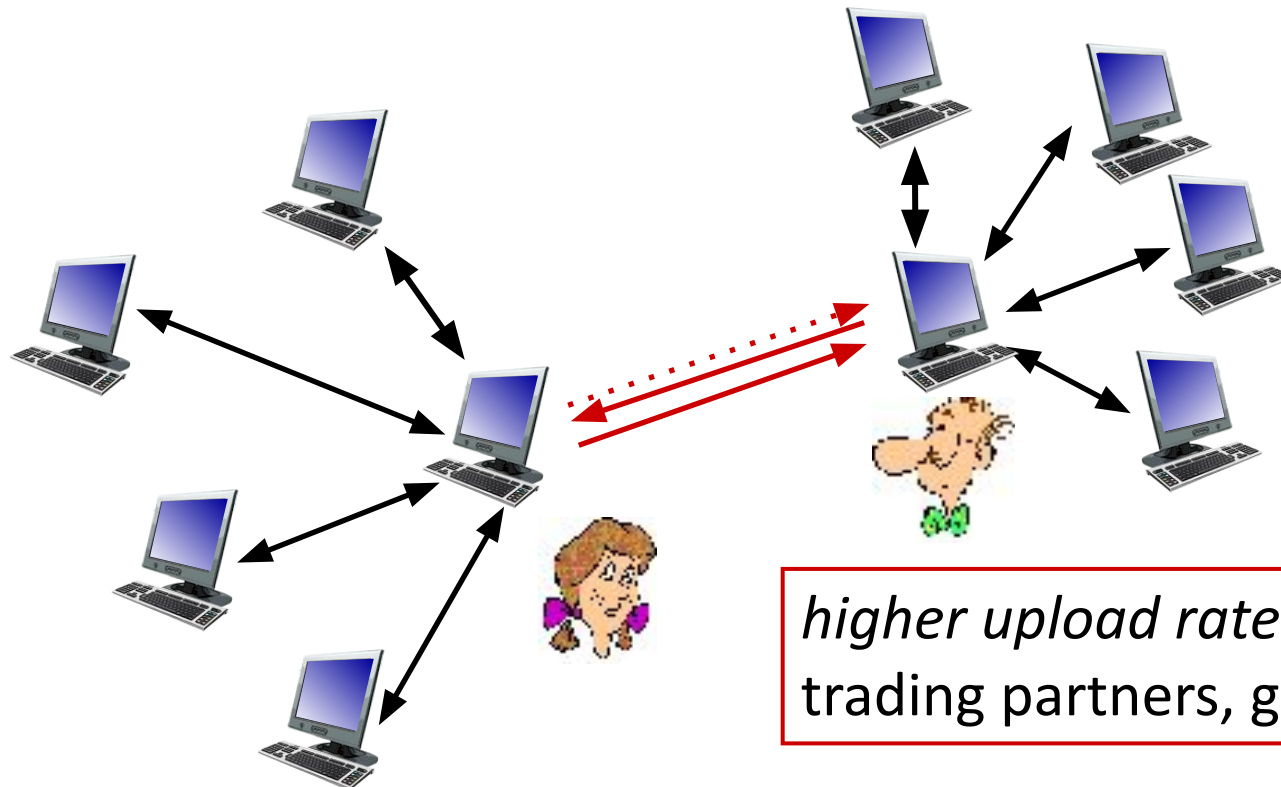
Requesting chunks:

- at any given time, different peers have different subsets of file chunks
- periodically, Alice asks each peer for list of chunks that they have
- Alice requests missing chunks from peers, **rarest first**

Sending chunks: tit-for-tat

- Alice sends chunks to those four peers currently sending her chunks *at highest rate*
 - other peers are choked by Alice (do not receive chunks from her)
 - re-evaluate top 4 every 10 secs
- every 30 secs: randomly select another peer, starts sending chunks
 - “**optimistically unchoke**” this peer
 - newly chosen peer may join top 4

- (1) Alice “optimistically unchokes” Bob
- (2) Alice becomes one of Bob’s top-four providers; Bob reciprocates
- (3) Bob becomes one of Alice’s top-four providers



**Pieces (mini-chunks),
pipelining, random first
selection, endgame mode,
and anti-snubbing**

*higher upload rate: find better
trading partners, get file faster !*

- BitTorrent (BTT) White Paper –
[https://www.bittorrent.com/btt/btt-docs/BitTorrent \(BTT\) White Paper v0.8.7 Feb 2019.pdf](https://www.bittorrent.com/btt/btt-docs/BitTorrent_(BTT)_White_Paper_v0.8.7_Feb_2019.pdf)
- Peer-to-peer networking with BitTorrent –
<http://web.cs.ucla.edu/classes/cs217/05BitTorrent.pdf>
- Torrents Explained: How BitTorrent Works –
<https://youtu.be/urzQeD7ftbl>



Thank You
For Your Attention

Unit – 2 Application Layer

2.3 The Domain Name System

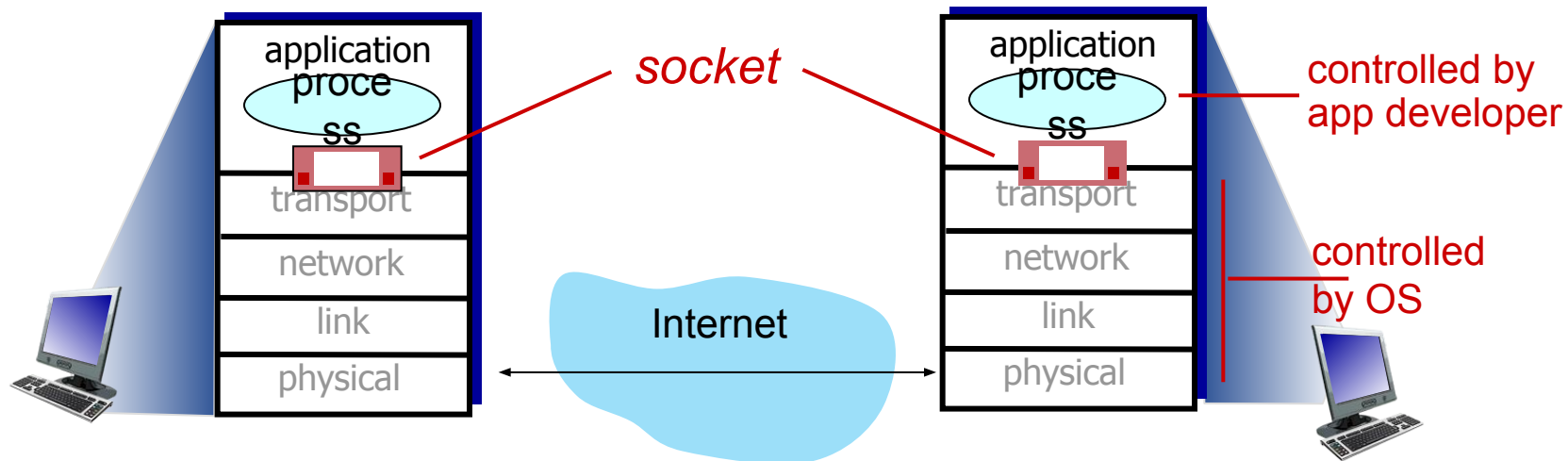
2.4 P2P Applications

2.5 Socket Programming with TCP & UDP

2.6 Other Application Layer Protocols

goal: learn how to build client/server applications that communicate using sockets

socket: door between application process and end-end-transport protocol



Two socket types for two transport services:

- *UDP*: unreliable datagram
- *TCP*: reliable, byte stream-oriented

Application Example:

1. client reads a line of characters (data) from its keyboard and sends data to server
2. server receives the data and converts characters to uppercase
3. server sends modified data to client
4. client receives modified data and displays line on its screen

UDP: no “connection” between client & server

- no handshaking before sending data
- sender explicitly attaches IP destination address and port # to each packet
- receiver extracts sender IP address and port# from received packet

UDP: transmitted data may be lost or received out-of-order

Application viewpoint:

- UDP provides *unreliable* transfer of groups of bytes (“datagrams”) between client and server



server (running on serverIP)

create socket, port= x:
`serverSocket =
socket(AF_INET,SOCK_DGRAM)`

read datagram from
`serverSocket`

write reply to
`serverSocket`
specifying
client address,
port number

client



create socket:
`clientSocket =
socket(AF_INET,SOCK_DGRAM)`

Create datagram with server IP and
port=x; send datagram via
`clientSocket`

read datagram from
`clientSocket`

close
`clientSocket`



Python UDPClient

include Python's socket library → `from socket import *`

`serverName = 'hostname'`

`serverPort = 12000`

create UDP socket for server →

`clientSocket = socket(AF_INET,
SOCK_DGRAM)`

get user keyboard input →

attach server name, port to message; send into
socket →

`message = raw_input('Input lowercase sentence:')
clientSocket.sendto(message.encode(),`

read reply characters from socket into string →

`(serverName, serverPort))`

print out received string and close socket →

`modifiedMessage, serverAddress =
clientSocket.recvfrom(2048)
print modifiedMessage.decode()
clientSocket.close()`



Python UDPServer

```
from socket import *
```

```
serverPort = 12000
```

create UDP socket →

```
serverSocket = socket(AF_INET, SOCK_DGRAM)
```

bind socket to local port number 12000 →

```
serverSocket.bind("", serverPort))
```

loop forever →

```
print ("The server is ready to receive")
```

Read from UDP socket into message, getting client's address (client IP and port) →

while True:

```
message, clientAddress = serverSocket.recvfrom(2048)
```

```
modifiedMessage = message.decode().upper()
```

```
serverSocket.sendto(modifiedMessage.encode(),
```

send upper case string back to this client →

clientAddress)

Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

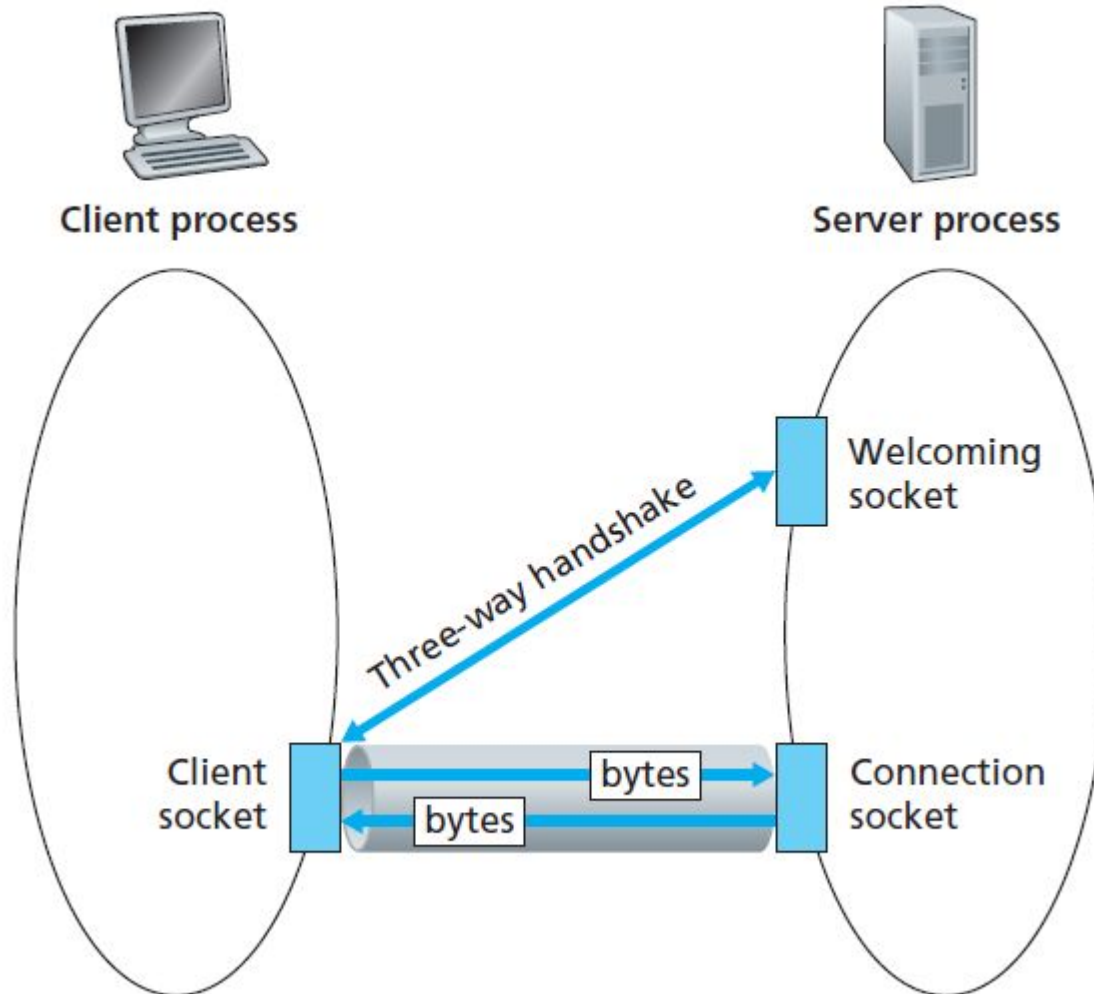
Client contacts server by:

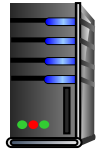
- Creating TCP socket, specifying IP address, port number of server process
- *when client creates socket:* client TCP establishes connection to server TCP

- when contacted by client, *server TCP creates new socket* for server process to communicate with that particular client
 - allows server to talk with multiple clients
 - source port numbers used to distinguish clients (more in Chap 3)

Application viewpoint

TCP provides reliable, in-order byte-stream transfer ("pipe") between client and server





server (running on `hostid`)

client



create socket,
port=`x`, for incoming
request:
`serverSocket = socket()`

wait for incoming
connection request
`connectionSocket =`
`serverSocket.accept()`

read request from
`connectionSocket`

write reply to
`connectionSocket`

close
`connectionSocket`

**TCP
connection setup**

create socket,
connect to `hostid`, port=`x`
`clientSocket = socket()`

send request using
`clientSocket`

read reply from
`clientSocket`

close
`clientSocket`

Python TCPClient

```
from socket import *
```

```
serverName = 'servername'
```

```
serverPort = 12000
```

create TCP socket for
server, remote port
12000

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

```
clientSocket.connect((serverName,serverPort))
```

```
sentence = raw_input('Input lowercase sentence:')
```

No need to attach
server name, port

```
clientSocket.send(sentence.encode())
```

```
modifiedSentence = clientSocket.recv(1024)
```

```
print ('From Server:', modifiedSentence.decode())
```

```
clientSocket.close()
```

Python TCPServer

	<pre>from socket import *</pre>
	<pre>serverPort = 12000</pre>
create TCP welcoming socket →	<pre>serverSocket = socket(AF_INET,SOCK_STREAM)</pre>
	<pre>serverSocket.bind(('',serverPort))</pre>
server begins listening for incoming TCP requests →	<pre>serverSocket.listen(1)</pre>
	<pre>print 'The server is ready to receive'</pre>
loop forever →	<pre>while True:</pre>
server waits on accept() for incoming requests, new socket created on return →	<pre>connectionSocket, addr = serverSocket.accept()</pre>
read bytes from socket (but not address as in UDP) →	<pre>sentence = connectionSocket.recv(1024).decode()</pre>
	<pre>capitalizedSentence = sentence.upper()</pre>
	<pre>connectionSocket.send(capitalizedSentence.encode())</pre>
close connection to this client (but <i>not</i> welcoming socket) →	<pre>connectionSocket.close()</pre>

Unit – 2 Application Layer

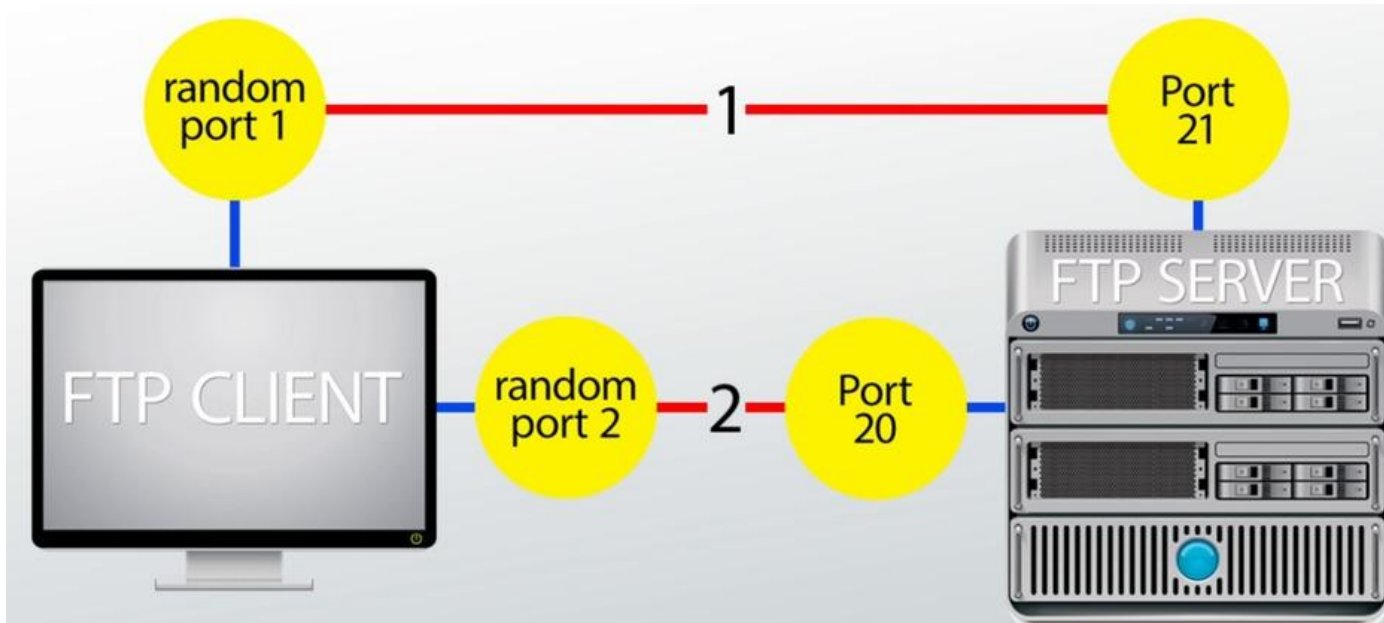
2.3 The Domain Name System

2.4 P2P Applications

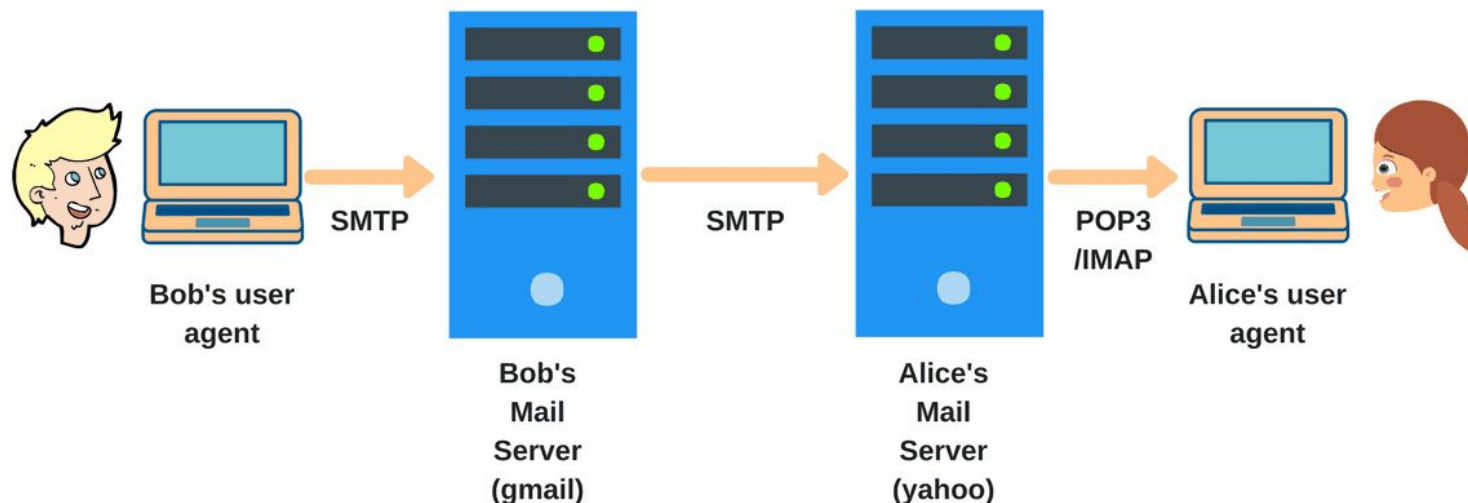
2.5 Socket Programming with TCP & UDP

2.6 Other Application Layer Protocols

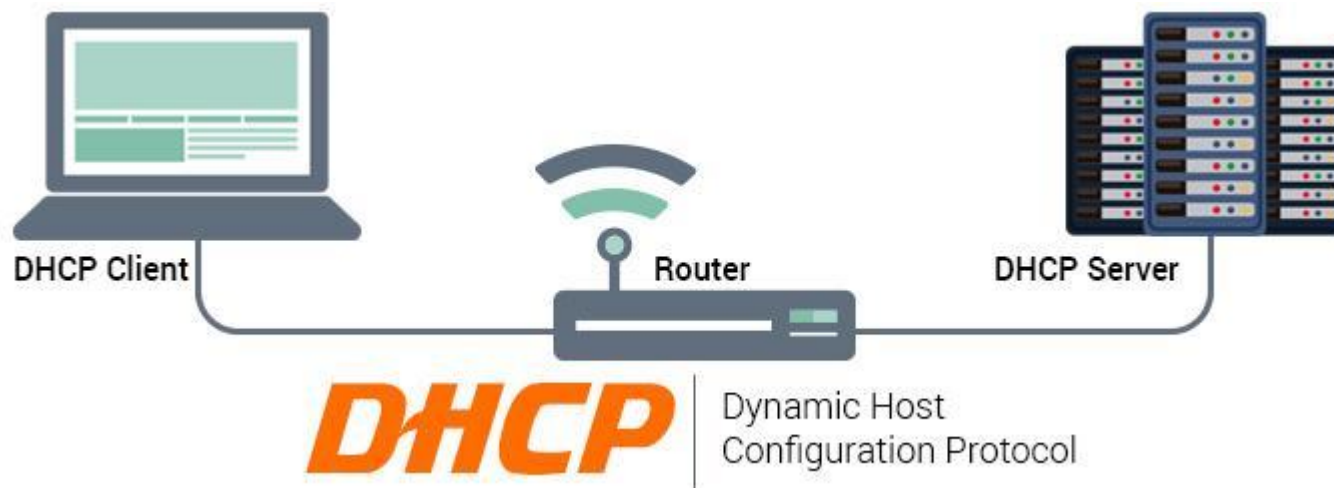
- File Transfer Protocol (FTP) - used to exchange large files on the internet TCP
- Invoked from the command prompt or some GUI.
- Allows to update (delete, rename, move, and copy) files at a server.
- Data connection (Port No. 20) & Control connection (Port No. 21)



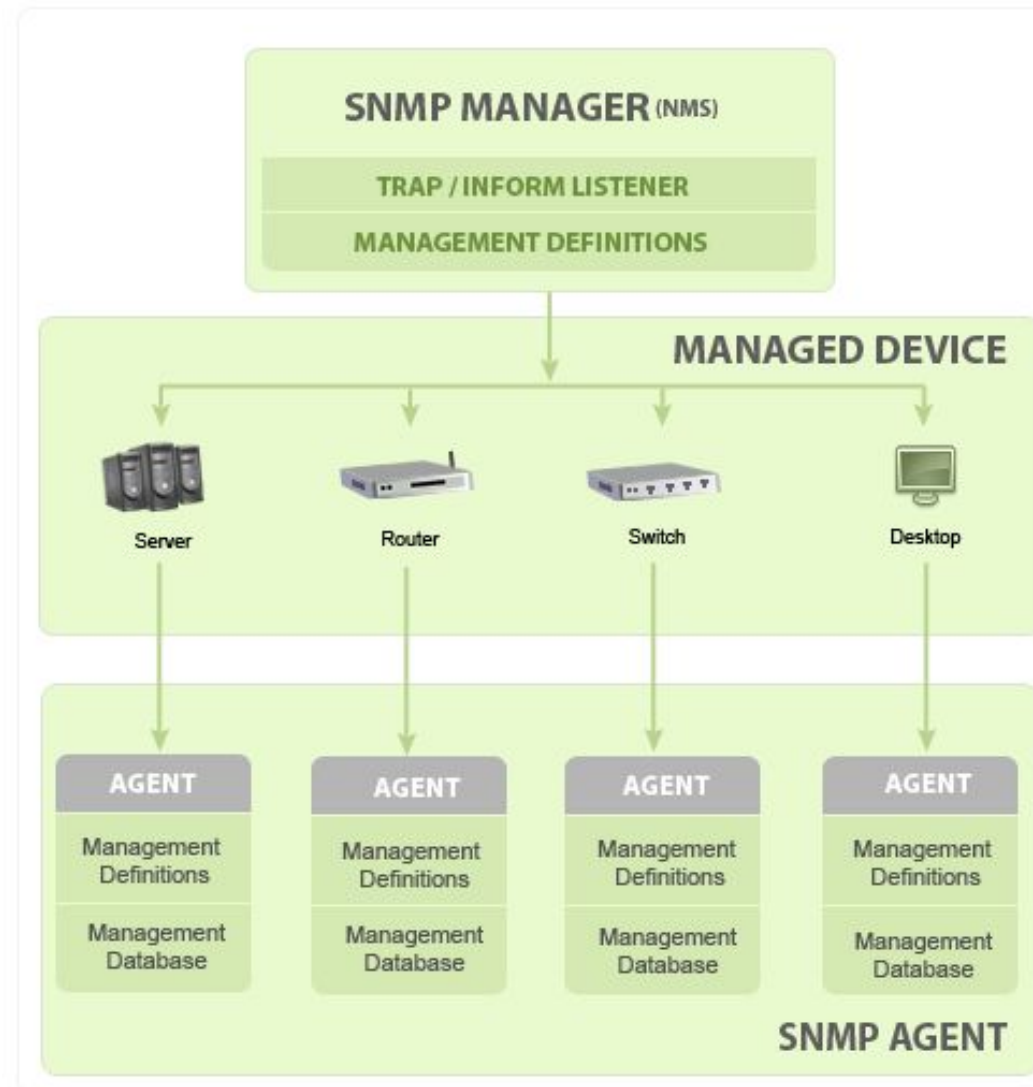
- **Simple Mail Transfer Protocol** - an internet standard for e-mail Transmission.
- Connections are secured with SSL (Secure Socket Layer).
- Messages are stored and then forwarded to the destination (relay).
- SMTP uses a port number 25 of TCP.



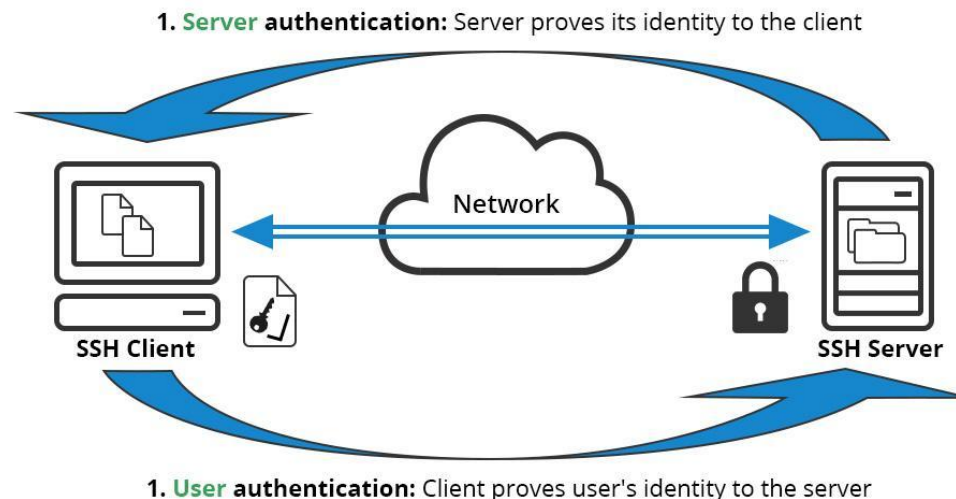
- **Dynamic Host Configuration Protocol** - assign IP addresses to computers in a network dynamically.
- IP addresses may change even when computer is in network (DHCP leases).
- DHCP port number for server is 67 and for the client is 68.
- A client-server model & based on **discovery, offer, request, and ACK**.
- Includes subnet mask, DNS server address, default gateway



- **Simple Network Management Protocol** - exchange management information between network devices.
- **Basic components & functionalities**
 - SNMP Manager
 - Managed Devices
 - SNMP Agents
 - MIB (Management Information Base)



- Allows a user to communicate with a remote device.
- Used mostly by network admin to remotely access and manage devices.
- Telnet client and server installed – uses TCP port no. 23
- SSH – uses public key **encryption** & TCP port 22 by default.



Port #	Application Layer Protocol	Type	Description
20	FTP	TCP	File Transfer Protocol - data
21	FTP	TCP	File Transfer Protocol - control
22	SSH	TCP/UDP	Secure Shell for secure login
23	Telnet	TCP	Unencrypted login
25	SMTP	TCP	Simple Mail Transfer Protocol
53	DNS	TCP/UDP	Domain Name Server
67/68	DHCP	UDP	Dynamic Host
80	HTTP	TCP	HyperText Transfer Protocol
123	NTP	UDP	Network Time Protocol
161,162	SNMP	TCP/UDP	Simple Network Management Protocol
389	LDAP	TCP/UDP	Lightweight Directory Authentication Protocol
443	HTTPS	TCP/UDP	HTTP with Secure Socket Layer

our study of network application layer is now complete!

- application architectures
 - client-server
 - P2P
- application service requirements:
 - reliability, bandwidth, delay
- Internet transport service model
 - connection-oriented, reliable: TCP
 - unreliable, datagrams: UDP
- specific protocols:
 - HTTP
 - DNS
 - P2P: BitTorrent
- socket programming:
TCP, UDP sockets

Most importantly: learned about *protocols*!

- typical request/reply message exchange:
 - client requests info or service
 - server responds with data, status code
- message formats:
 - *headers*: fields giving info about data
 - *data*: info(payload) being communicated

important themes:

- centralized vs. decentralized
- stateless vs. stateful
- scalability
- reliable vs. unreliable message transfer
- “complexity at network edge”



THANK YOU

TEAM NETWORKS

Department of Computer Science and Engineering





PES
UNIVERSITY

COMPUTER NETWORKS

TEAM NETWORKS

Department of Computer Science and Engineering

COMPUTER NETWORKS

Transport Layer

TEAM NETWORKS

Department of Computer Science and Engineering

Transport Layer - Roadmap

3.1 Transport-layer Services

3.2 Multiplexing and Demultiplexing

3.3 Connectionless Transport: UDP

3.4 Principles of reliable data transfer

Transport Layer - Roadmap

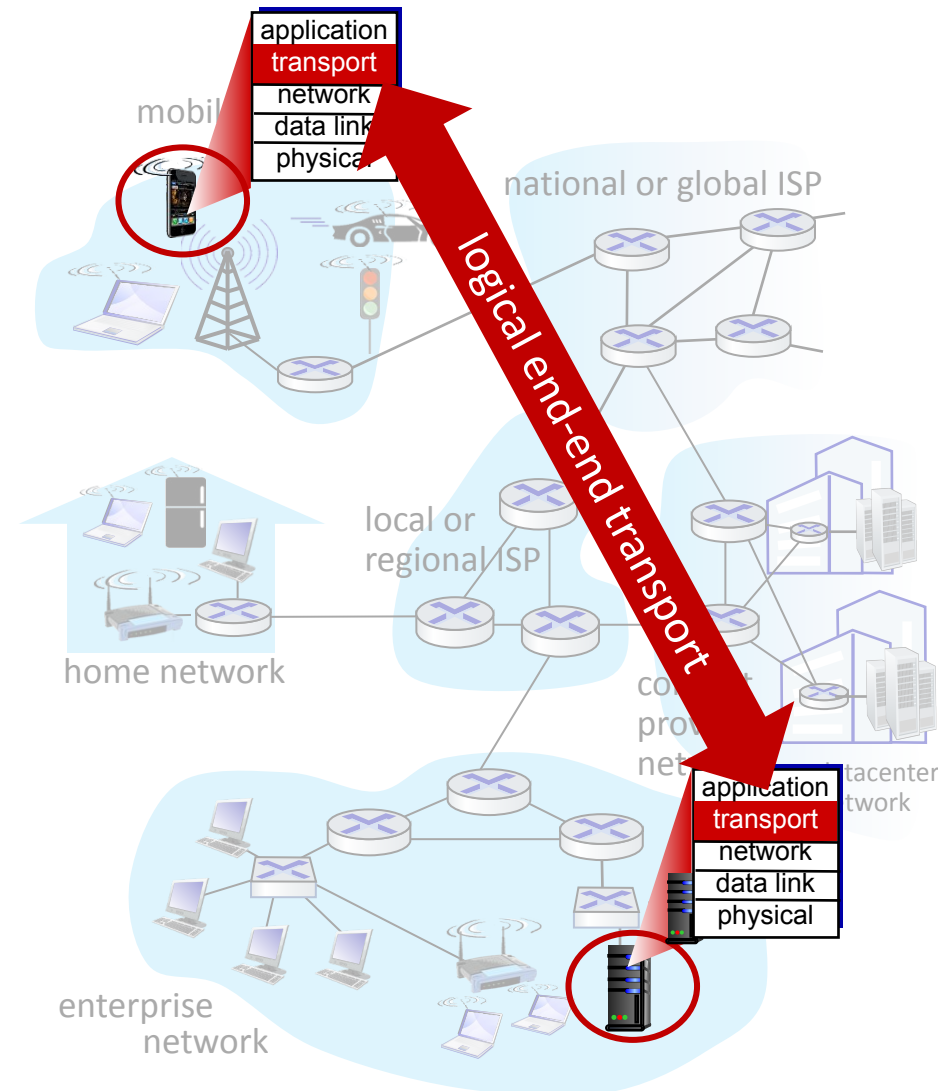
3.1 Transport-layer Services

3.2 Multiplexing and Demultiplexing

3.3 Connectionless Transport: UDP

3.4 Principles of reliable data transfer

- provide *logical communication* between application processes running on different hosts
- transport protocols actions in end systems:
 - sender: breaks application messages into *segments*, passes to network layer
 - receiver: reassembles segments into messages, passes to application layer
- two transport protocols available to Internet applications
 - TCP, UDP





household analogy:

*12 kids in Ann's house
sending letters to 12 kids
in Bill's house:*

- **hosts** = houses
- **processes** = kids
- **app messages** = letters in envelopes

Transport vs. Network Layer Services and Protocols

- **network layer:** logical communication between *hosts*
- **transport layer:** logical communication between *processes*
 - relies on, enhances, network layer services

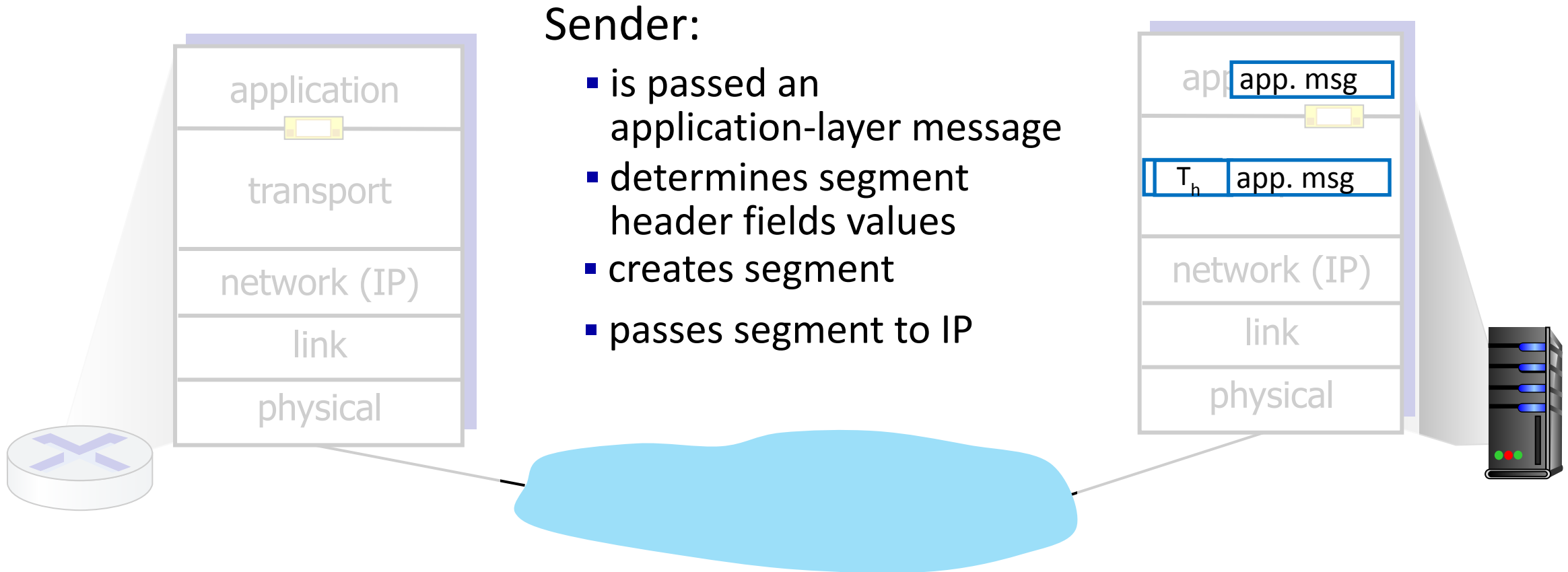
household analogy:

*12 kids in Ann's house
sending letters to 12 kids
in Bill's house:*

- **hosts** = houses
- **processes** = kids
- **app messages** = letters in envelopes

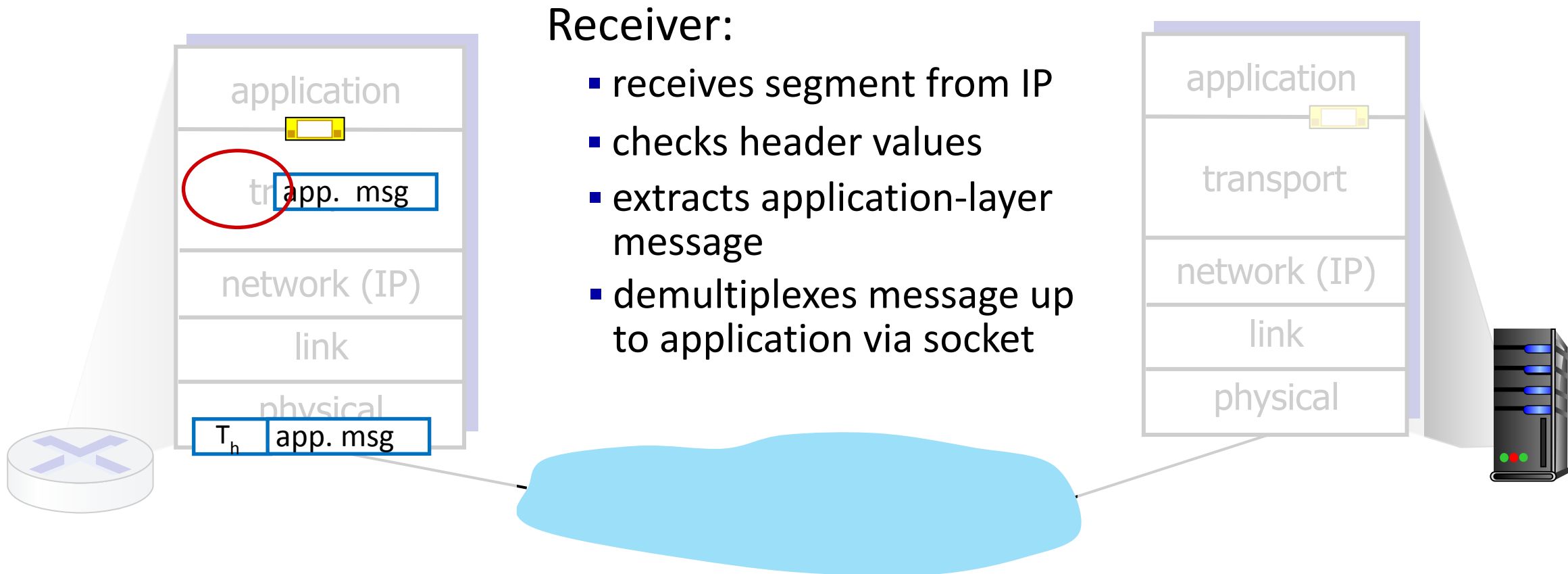
COMPUTER NETWORKS

Transport Layer Actions



COMPUTER NETWORKS

Transport Layer Actions

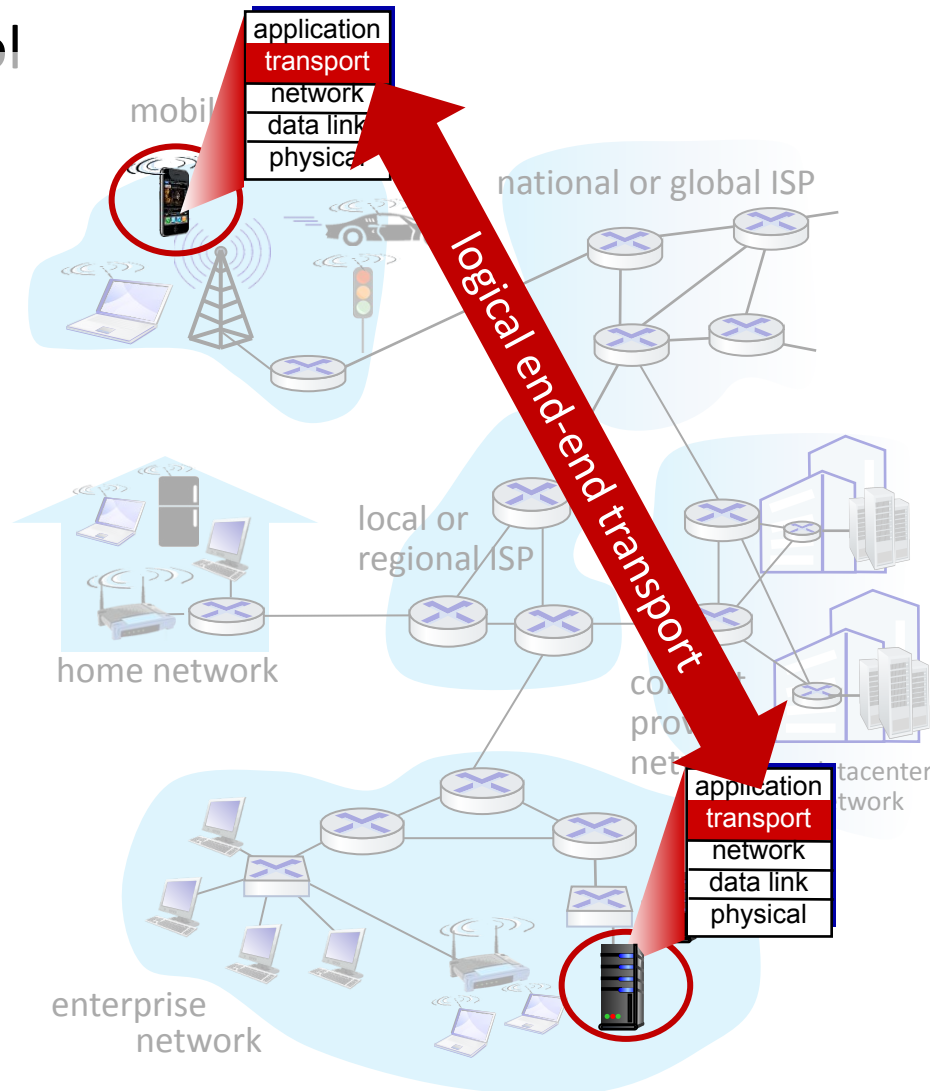


- **TCP:** Transmission Control Protocol

- reliable, connection oriented
- in-order delivery
- congestion control
- flow control
- connection setup

- **UDP:** User Datagram Protocol

- unreliable, connectionless
- unordered delivery
- no-frills extension of “best-effort” IP
- services not available:
 - delay guarantees
 - bandwidth guarantees



- Transport Layer – Explained –
<https://youtu.be/FxFJ1XlWtdI>
- Transport Layer Services – IIT Kharagpur
–<https://youtu.be/8-3CSAksYU>
- Transport Layer – Process to Process Delivery –
<https://youtu.be/9e4vTcaEYCg>



 Thank You
For Your Attention

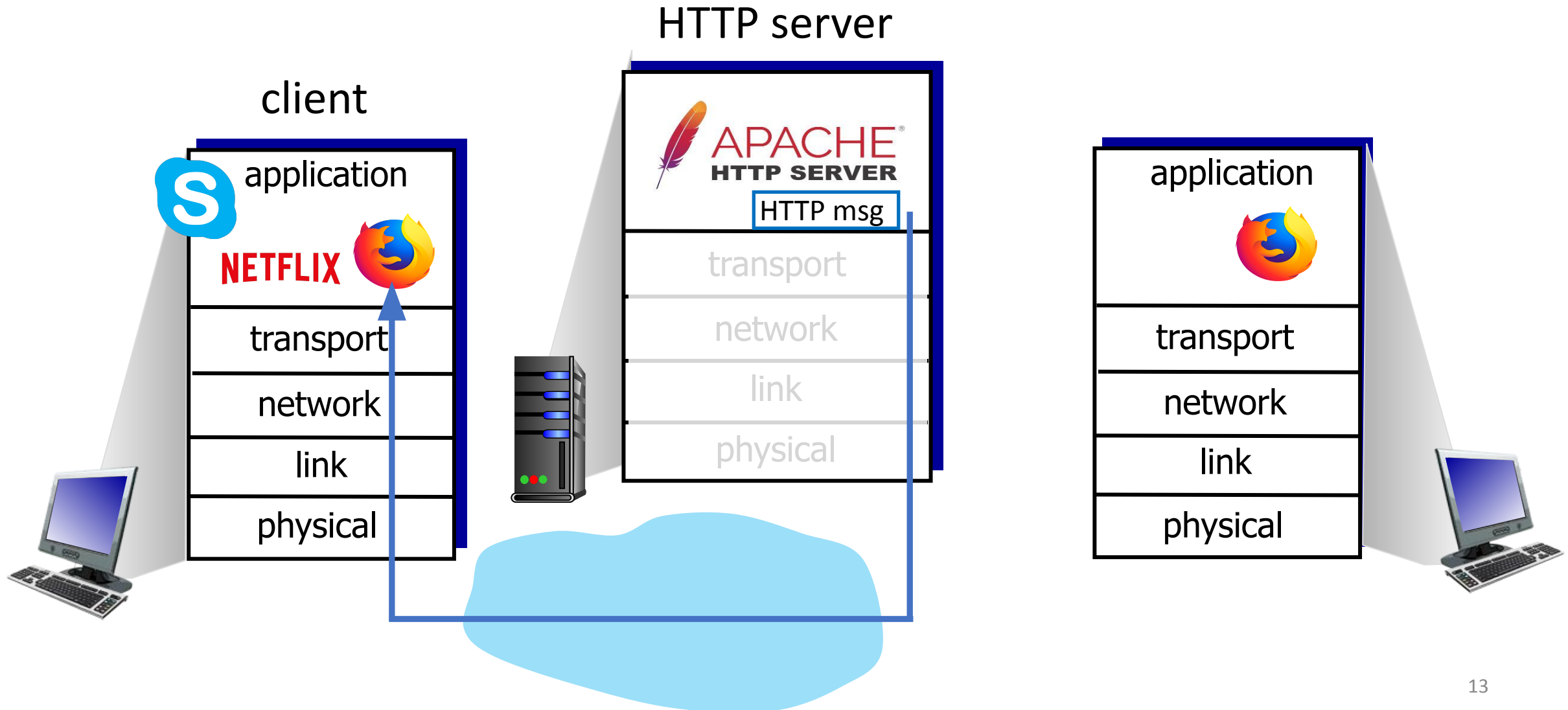
Transport Layer - Roadmap

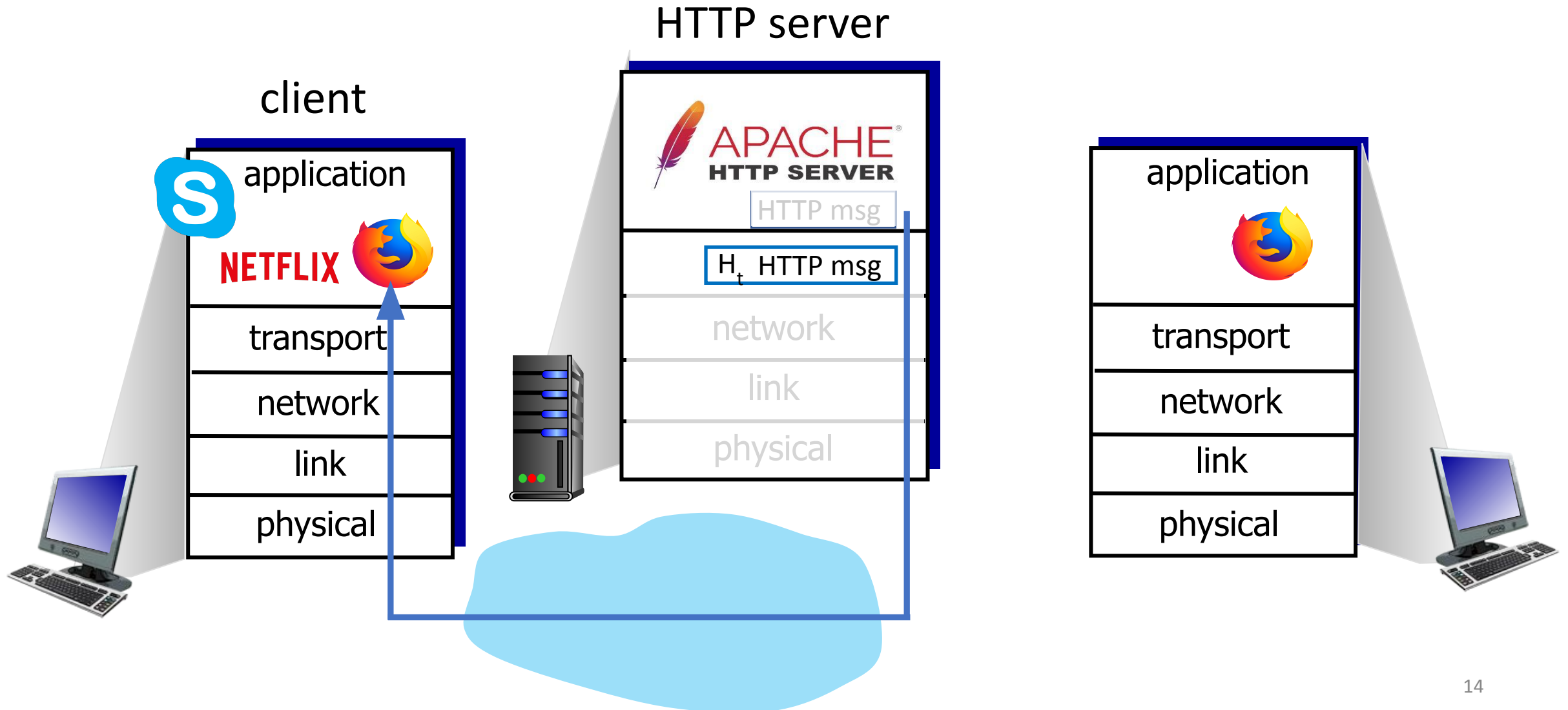
3.1 Transport-layer Services

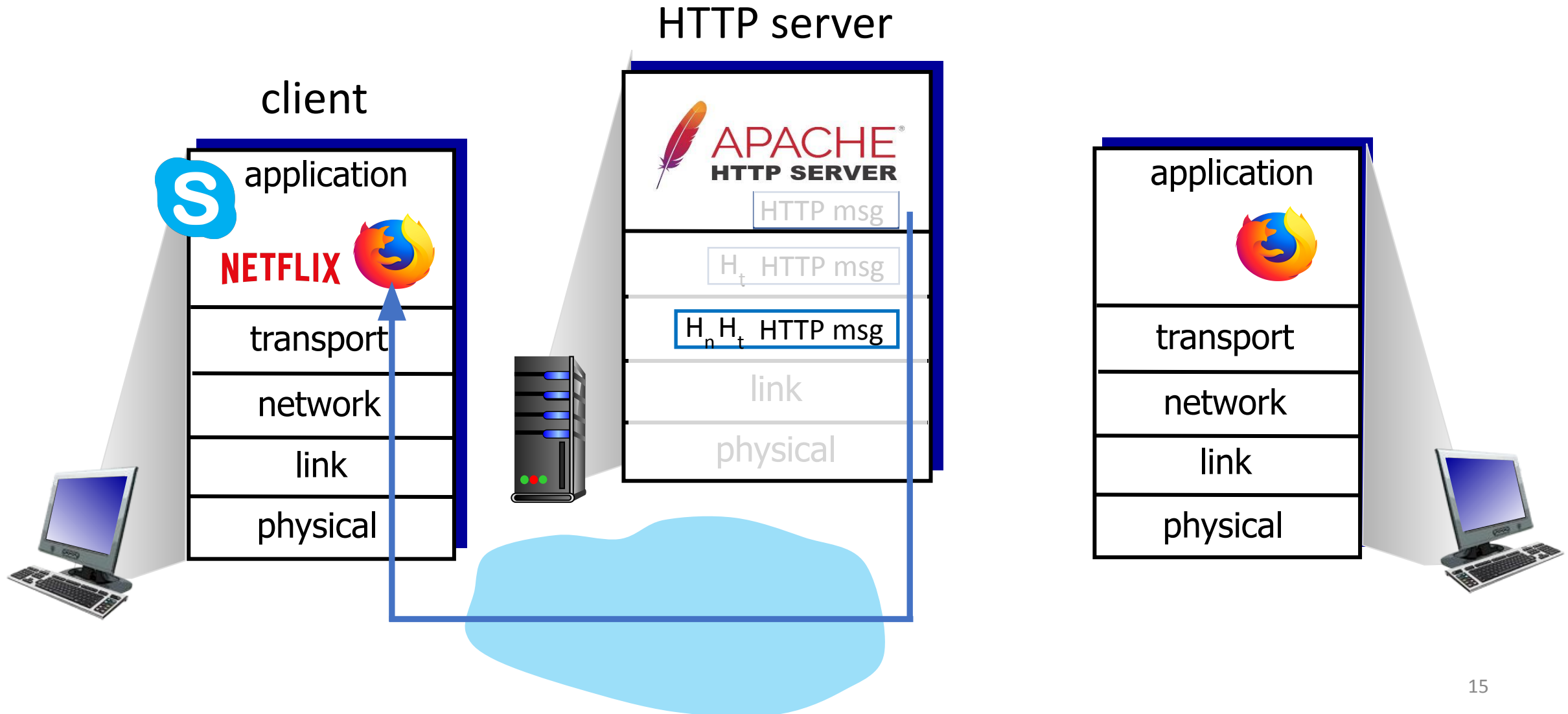
3.2 Multiplexing and Demultiplexing

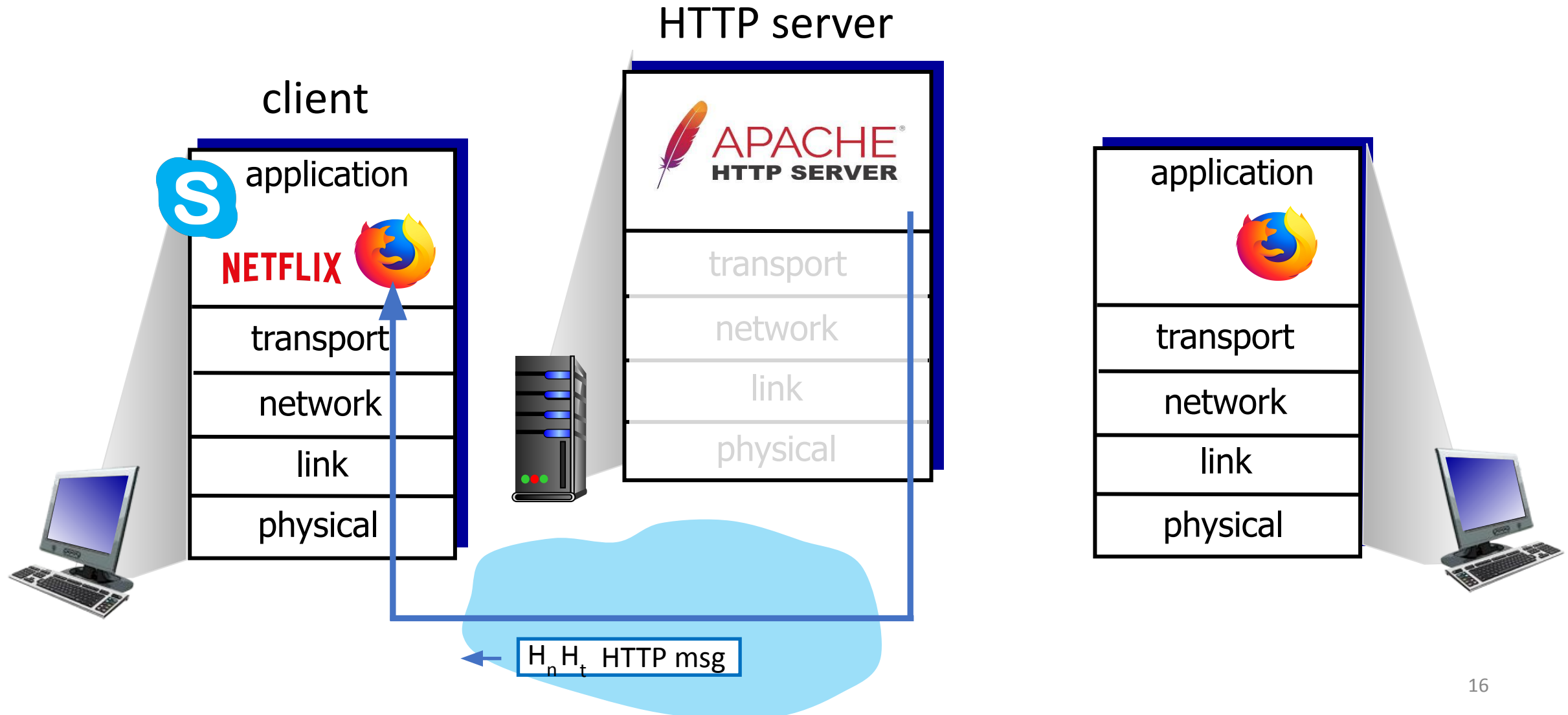
3.3 Connectionless Transport: UDP

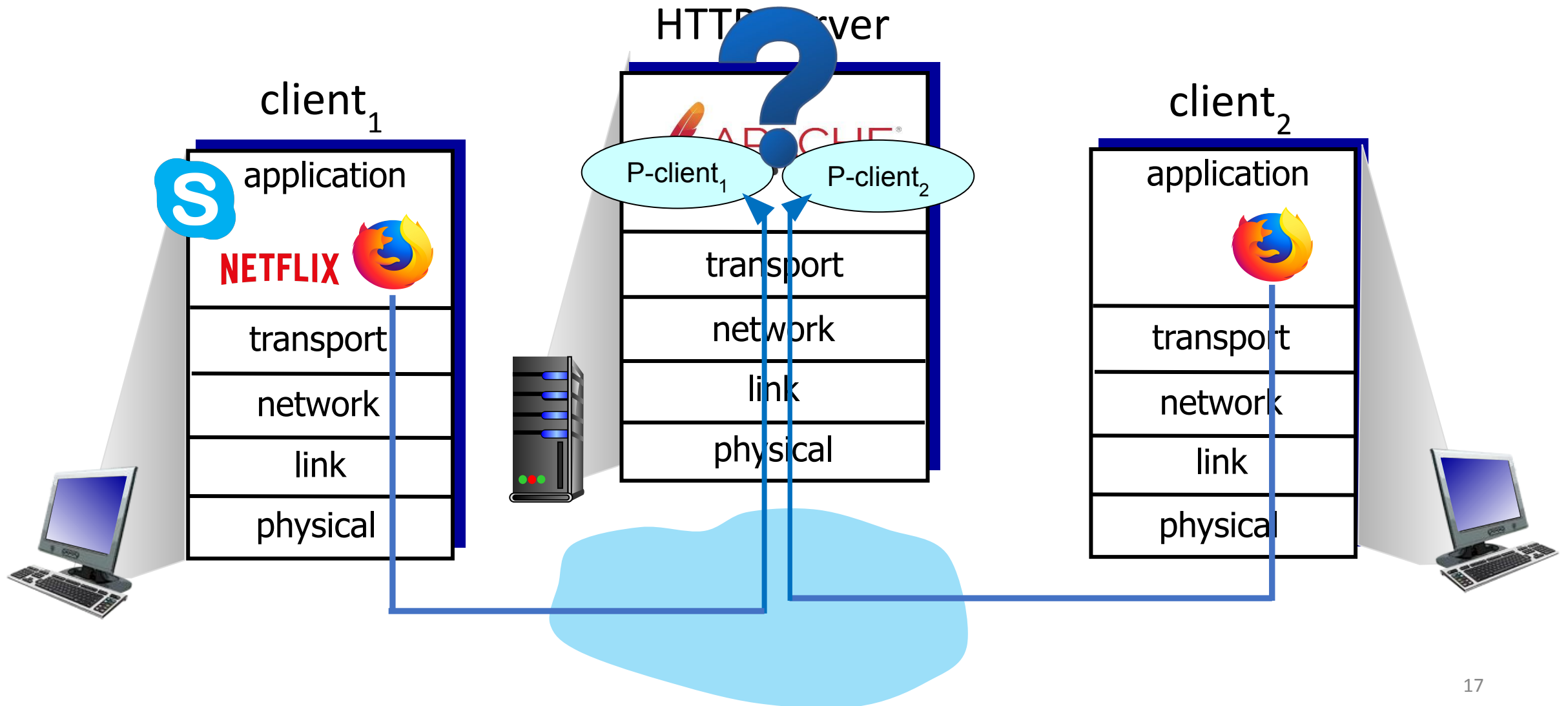
**Extending host-to-host delivery to
process-to-process delivery**









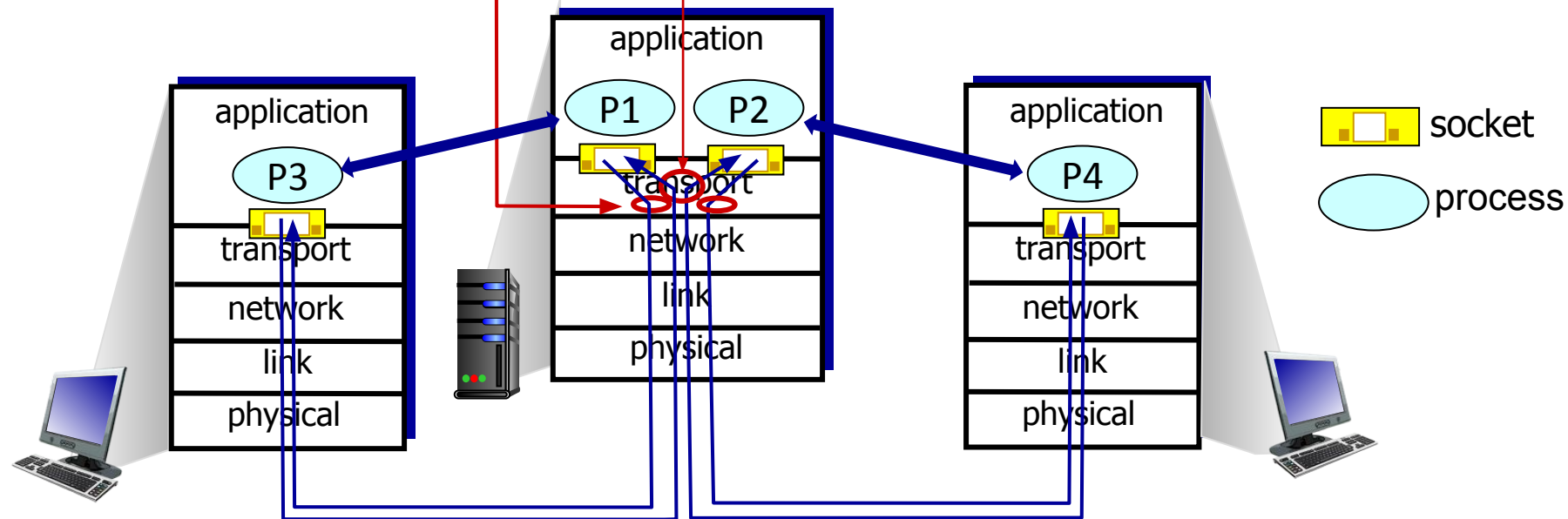


multiplexing at sender:

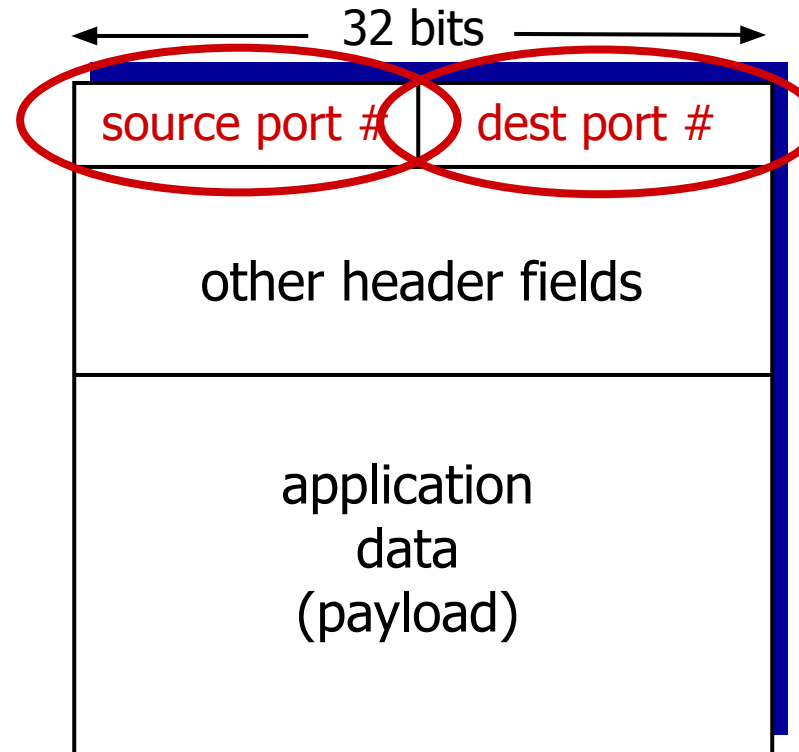
handle data from multiple sockets, add transport header (later used for demultiplexing)

demultiplexing at receiver:

use header info to deliver received segments to correct socket



- host receives IP datagrams
 - each datagram has source IP address, destination IP address
 - each datagram carries one transport-layer segment
 - each segment has source, destination port number
- host uses *IP addresses & port numbers* to direct segment to appropriate socket



TCP/UDP segment format

- Each port number - ranges from 0 to 65535.
- Port numbers ranging from 0 to 1023 are called **well-known port numbers** (restricted/reserved)

Recall:

- when creating socket, must specify *host-local* port #:

```
DatagramSocket mySocket1  
= new DatagramSocket(12534);
```

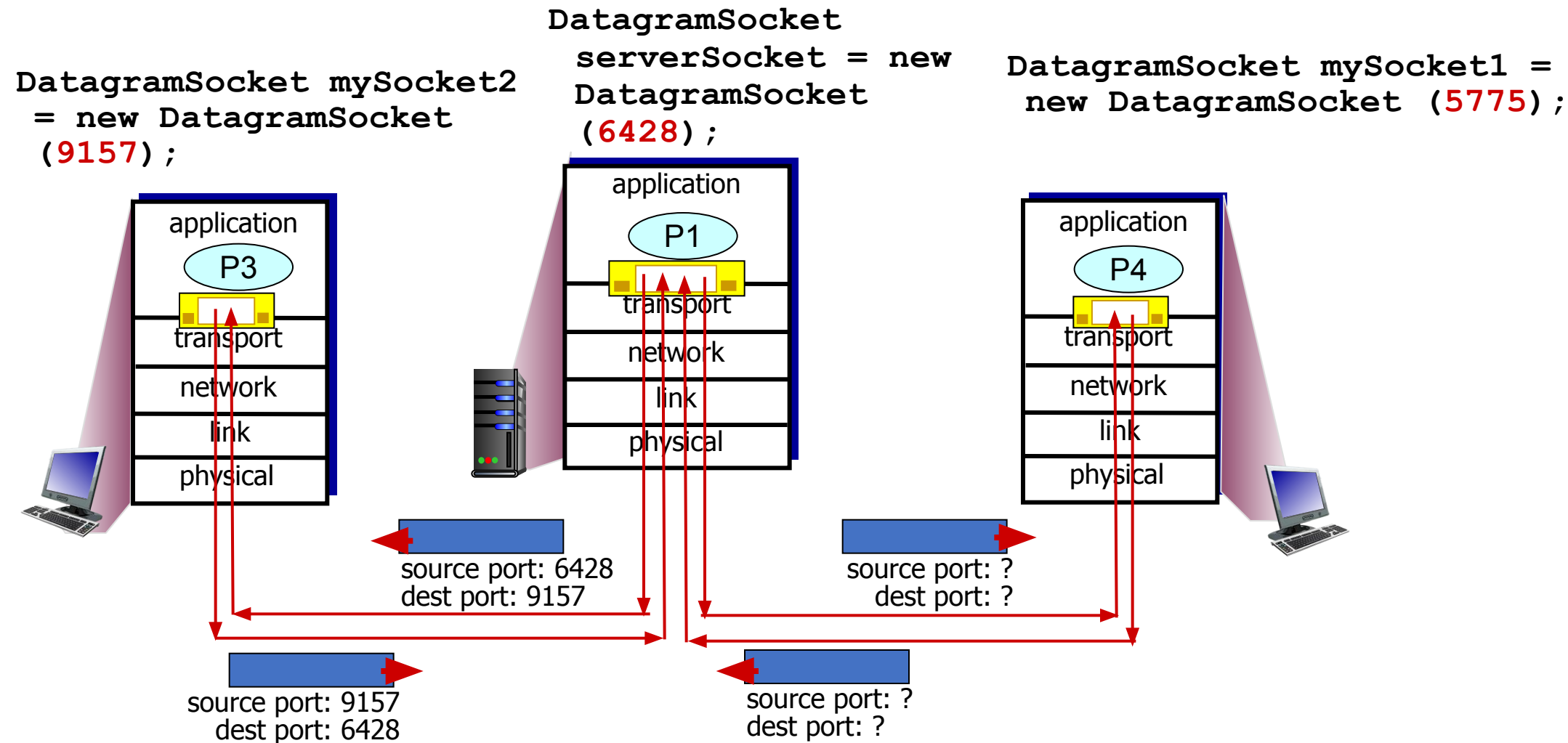
- when creating datagram to send into UDP socket, must specify
 - destination IP address
 - destination port #

when receiving host receives *UDP* segment:

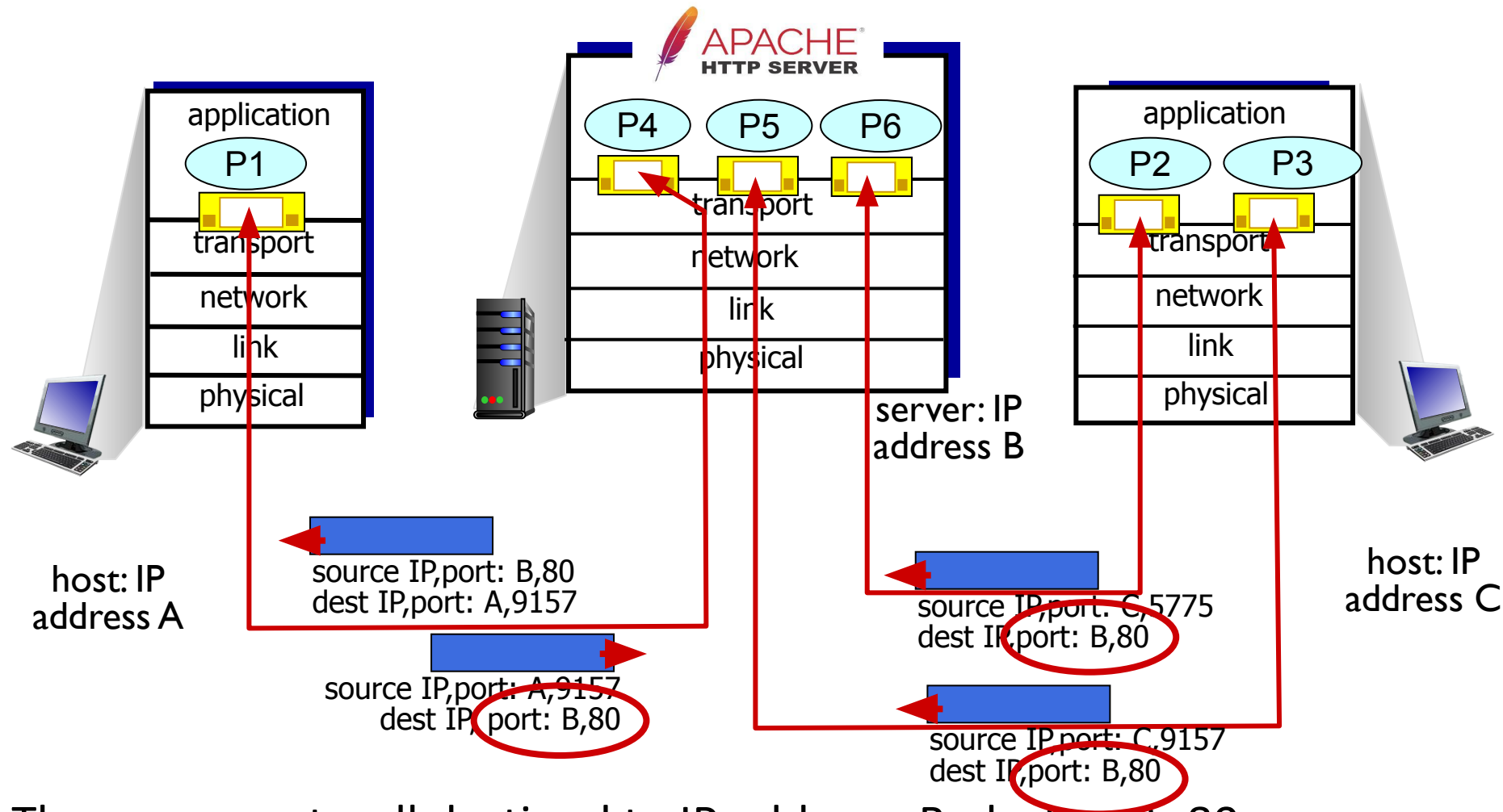
- checks destination port # in segment
- directs UDP segment to socket with that port #



IP/UDP datagrams with *same dest. port #*, but different source IP addresses and/or source port numbers will be directed to *same socket* at receiving host



- TCP socket identified by 4-tuple:
 - source IP address
 - source port number
 - dest IP address
 - dest port number
- demux: receiver uses **all four values (4-tuple)** to direct segment to appropriate socket
- server host may support many simultaneous TCP sockets:
 - each socket identified by its own 4-tuple
- web servers have different sockets for each connecting client
 - non-persistent HTTP will have different socket for each request



Three segments, all destined to IP address: B, dest port: 80
are demultiplexed to *different* sockets

- Multiplexing, demultiplexing: based on segment, datagram header field values
- **UDP:** demultiplexing using destination port number (only)
- **TCP:** demultiplexing using 4-tuple: source and destination IP addresses, and port numbers
- Multiplexing/demultiplexing happen at all layers

How many TCP connections can a server handle?

<https://medium.com/@in10se/how-servers-handle-more-than-65-536-connections-simultaneously-3a77189a160e>



- Transport Layer Multiplexing and Demultiplexing

<https://youtu.be/hgWCMry9EYo>

- Transport Layer – Process to Process Delivery –

<https://youtu.be/9e4vTcaEYCg>



 Thank You
For Your Attention

Transport Layer - Roadmap

3.1 Transport-layer Services

3.2 Multiplexing and Demultiplexing

3.3 Connectionless Transport: UDP

3.4 Principles of reliable data transfer

- “no frills,” “bare bones” Internet transport protocol
- “best effort” service, UDP segments may be:
 - lost
 - delivered out-of-order to app
- *connectionless*:
 - no handshaking between UDP sender, receiver
 - each UDP segment handled independently of others

Why is there a UDP?

- **no connection** establishment (which can add RTT delay)
- **no connection state** at sender, receiver (buffer, seq, ack, c-c parameters)
- **small header size** (8 vs 20 bytes)
- **no congestion control**
 - UDP can blast away as fast as desired!
 - can function in the face of congestion

- UDP use:
 - streaming multimedia apps (loss tolerant, rate sensitive)
 - DNS
 - SNMP
 - HTTP/3
- if reliable transfer needed over UDP (e.g., HTTP/3):
 - add needed reliability at application layer
 - add congestion control at application layer

COMPUTER NETWORKS

Popular Internet Applications using TCP/UDP



Application	Application-Layer Protocol	Underlying Transport Protocol
Electronic mail	SMTP	TCP
Remote terminal access	Telnet	TCP
Web	HTTP	TCP
File transfer	FTP	TCP
Remote file server	NFS	Typically UDP
Streaming multimedia	typically proprietary	UDP or TCP
Internet telephony	typically proprietary	UDP or TCP
Network management	SNMP	Typically UDP
Name translation	DNS	Typically UDP

INTERNET STANDARD

RFC 768

J. Postel

ISI

28 August 1980

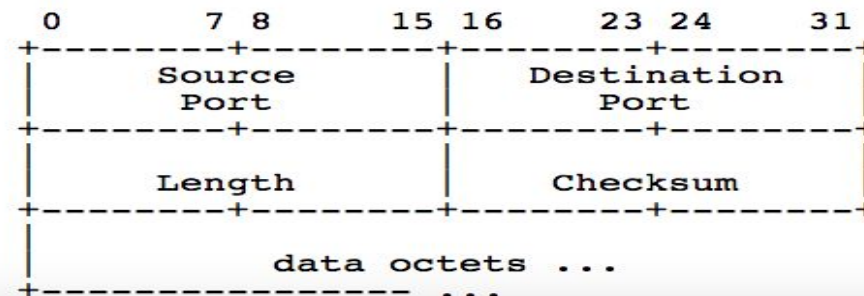
User Datagram Protocol

Introduction

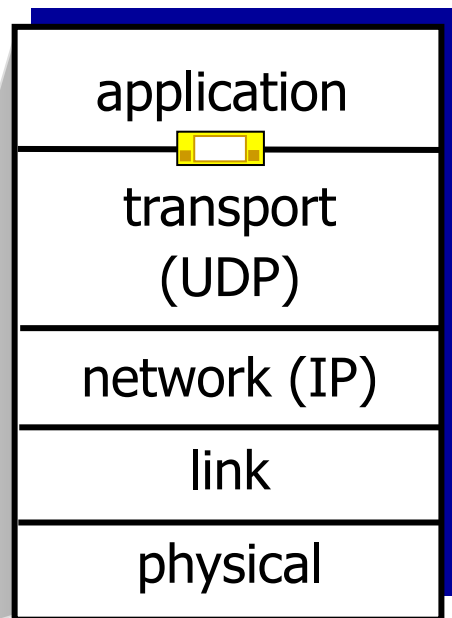
This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

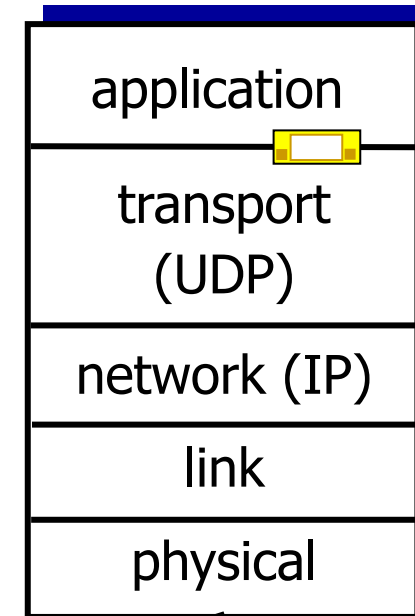
Format



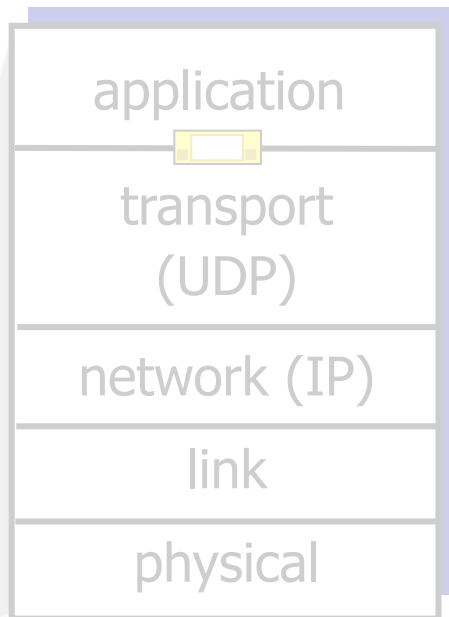
SNMP client



SNMP server



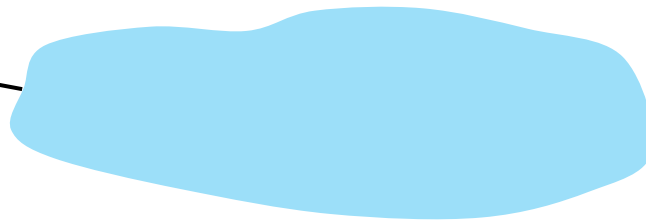
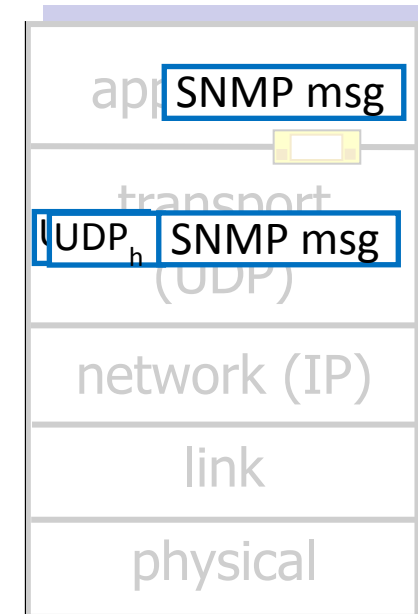
SNMP client



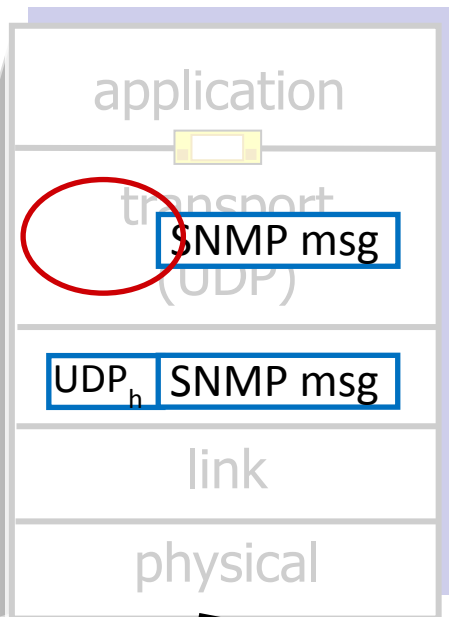
UDP sender actions:

- is passed an application-layer message
- determines UDP segment header fields values
- creates UDP segment
- passes segment to IP

SNMP server



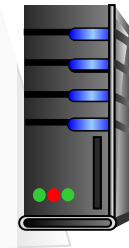
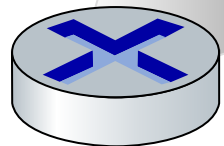
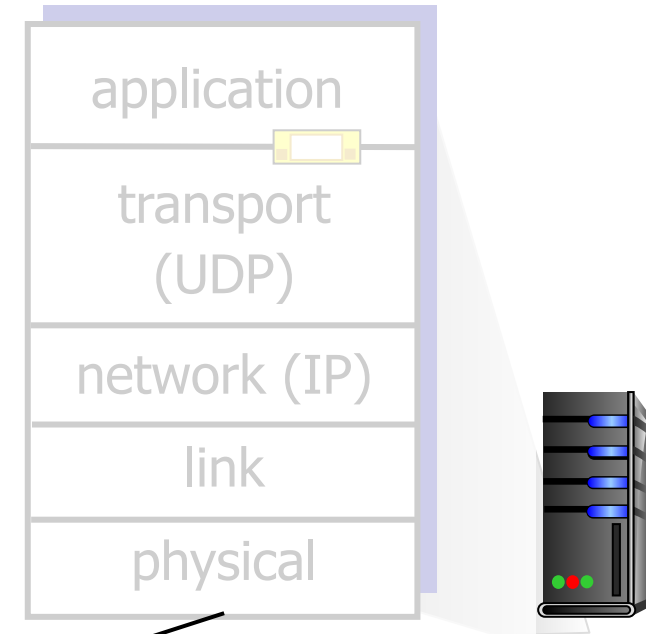
SNMP client

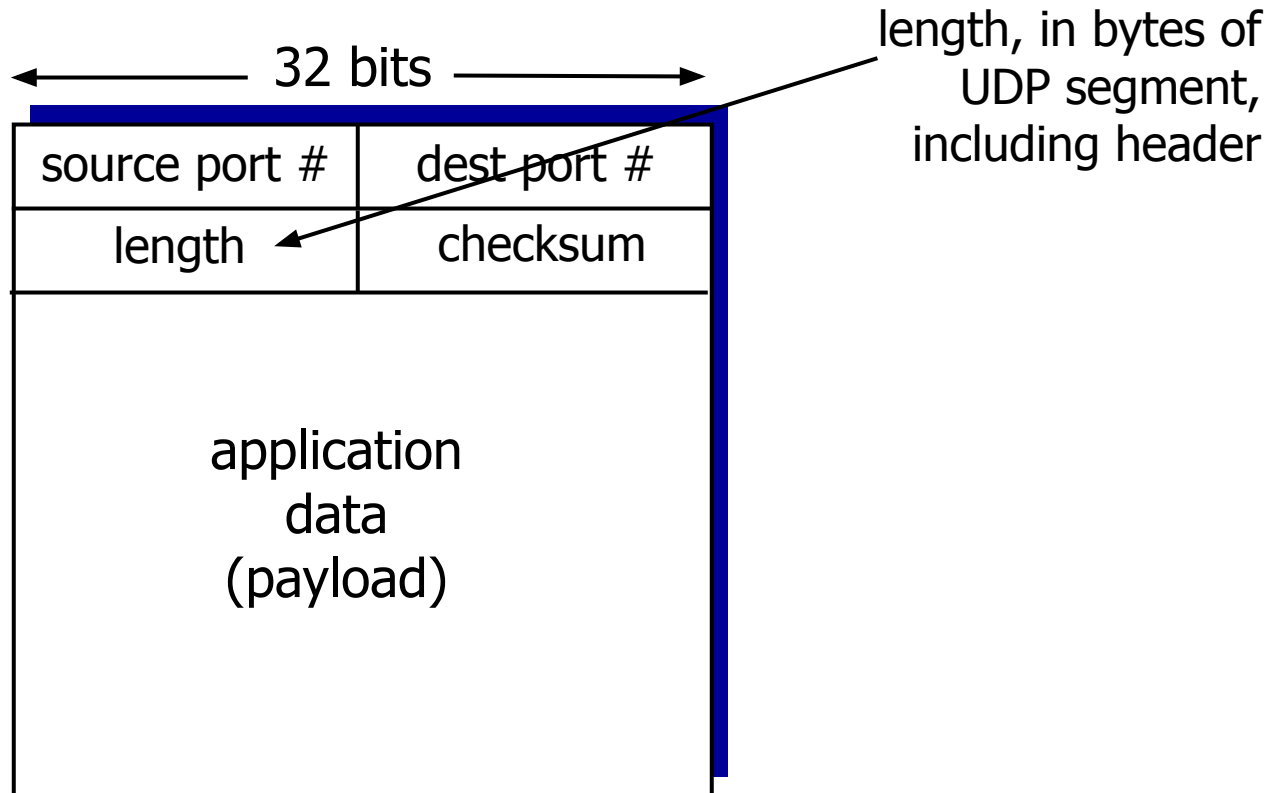


UDP receiver actions:

- receives segment from IP
- checks UDP checksum header value
- extracts application-layer message
- demultiplexes message up to application via socket

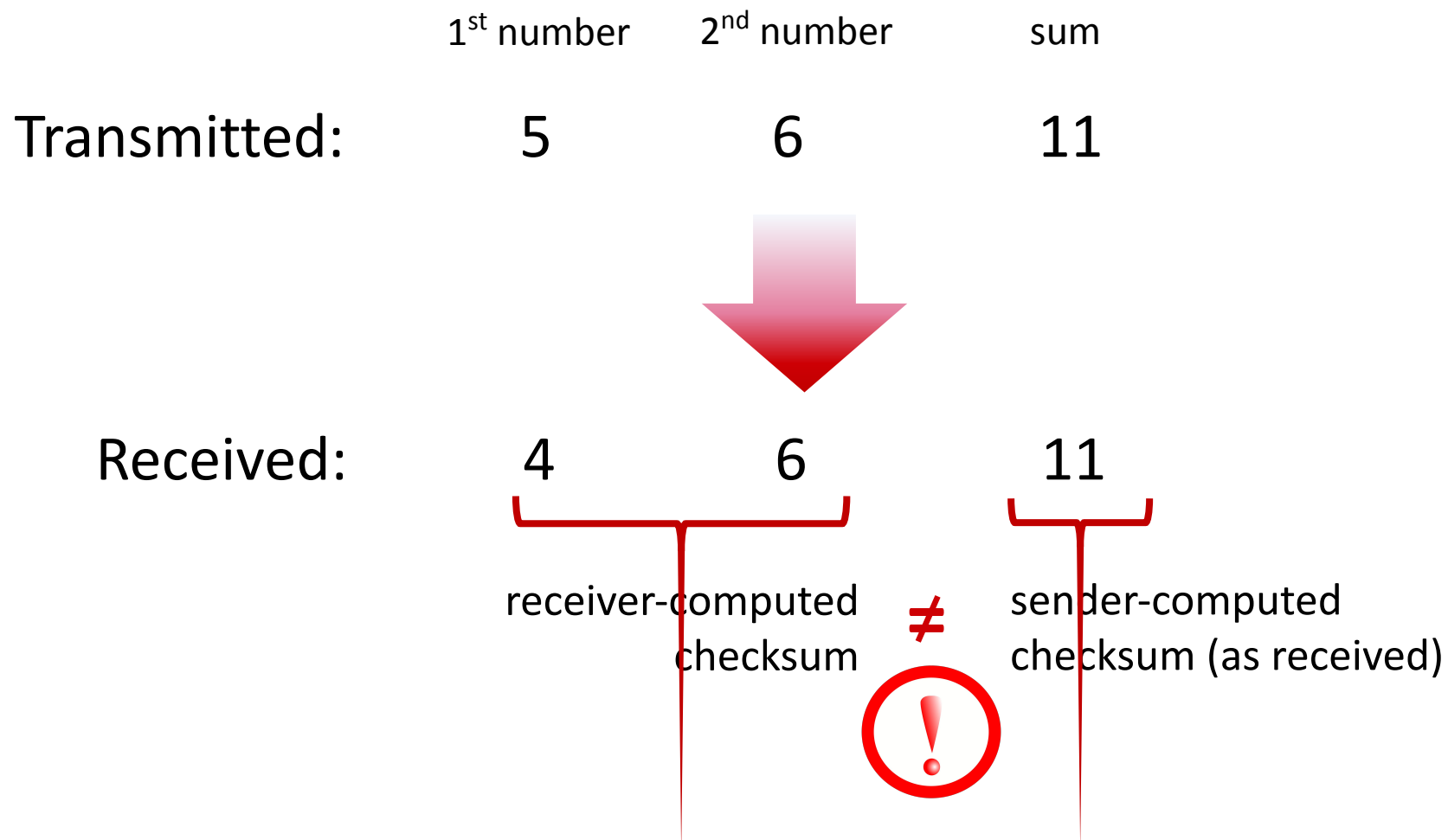
SNMP server





UDP segment format

Goal: detect “errors” (e.g., flipped bits) in transmitted segment



Goal: detect errors (*i.e.*, flipped bits) in transmitted segment

sender:

- treat contents of UDP segment (including UDP header fields and IP addresses) as sequence of 16-bit integers
- **checksum:** addition (one's complement sum) of segment content
- checksum value put into UDP checksum field

receiver:

- compute checksum of received segment
- check if computed checksum equals checksum field value:
 - Not equal - error detected
 - Equal - no error detected. *But maybe errors nonetheless?* More later

example: add two 16-bit integers

		1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
		1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1
sum		1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum		0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

example: add two 16-bit integers

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1

Even though numbers have changed (bit flips), *no* change in checksum!

What will be the checksum value if following 3 words are sent as data in UDP Segment?

```
0010 0101 1010 1100
1010 1010 1010 1010
0010 1011 1010 1000
```


What will be the checksum value if following 3 words are sent as data in UDP Segment?

0010 0101 1010 1100
1010 1010 1010 1010
0010 1011 1010 1000

Answer: 0000 0100 0000 0001

What will be the checksum value if following 3 words are sent as data in UDP Segment?

- 0000 1011 1111 0011
- 1010 1010 1010 1010
- 0100 0011 1100 1010

What will be the checksum value if following 3 words are sent as data in UDP Segment?

- 0000 1011 1111 0011
- 1010 1010 1010 1010
- 0100 0011 1100 1010

Ans: 0000 0101 1001 1000

If the data words sent are :

- 1011 0101 1001 1000
 - 0100 0100 0100 0100
 - 0011 0011 1011 0011
-
- And the checksum value sent is
 - 1101 0010 0110 1111

Whether the data delivered is correct or not?

If the data words sent are :

- 1011 0101 1001 1000
- 0100 0100 0100 0100
- 0011 0011 1011 0011

- And the checksum value sent is
- 1101 0010 0110 1111

Whether the data delivered is correct or not?

Answer: Yes, it is correct

COMPUTER NETWORKS

Internet Checksum: example

Sender's End

Frame 1:	11001100
Frame 2:	+ 10101010
Partial Sum:	<u>1 01110110</u>
	+ 1
	01110111
Frame 3:	+ 11110000
Partial Sum:	<u>1 01100111</u>
	+ 1
	01101000
Frame 4:	+ 11000011
Partial Sum:	<u>1 00101011</u>
	+ 1
Sum:	<u>00101100</u>
Checksum:	11010011

Receiver's End

Frame 1:	11001100
Frame 2:	+ 10101010
Partial Sum:	<u>1 01110110</u>
	+ 1
	01110111
Frame 3:	+ 11110000
Partial Sum:	<u>1 01100111</u>
	+ 1
	01101000
Frame 4:	+ 11000011
Partial Sum:	<u>1 00101011</u>
	+ 1
Sum:	<u>00101100</u>
Checksum:	<u>11010011</u>
Sum:	<u>11111111</u>
Complement:	00000000

Hence accept frames.

- UDP – RFC 768 <https://tools.ietf.org/html/rfc768>
- Networking - DNS and UDP
<https://youtu.be/vuyQ1PW6AwY>



 Thank You
For Your Attention

Transport Layer - Roadmap

3.1 Transport-layer Services

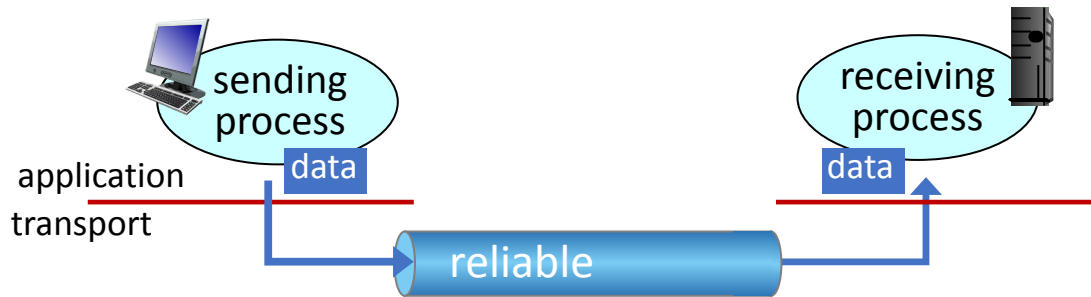
3.2 Multiplexing and Demultiplexing

3.3 Connectionless Transport: UDP

3.4 Principles of reliable data transfer

COMPUTER NETWORKS

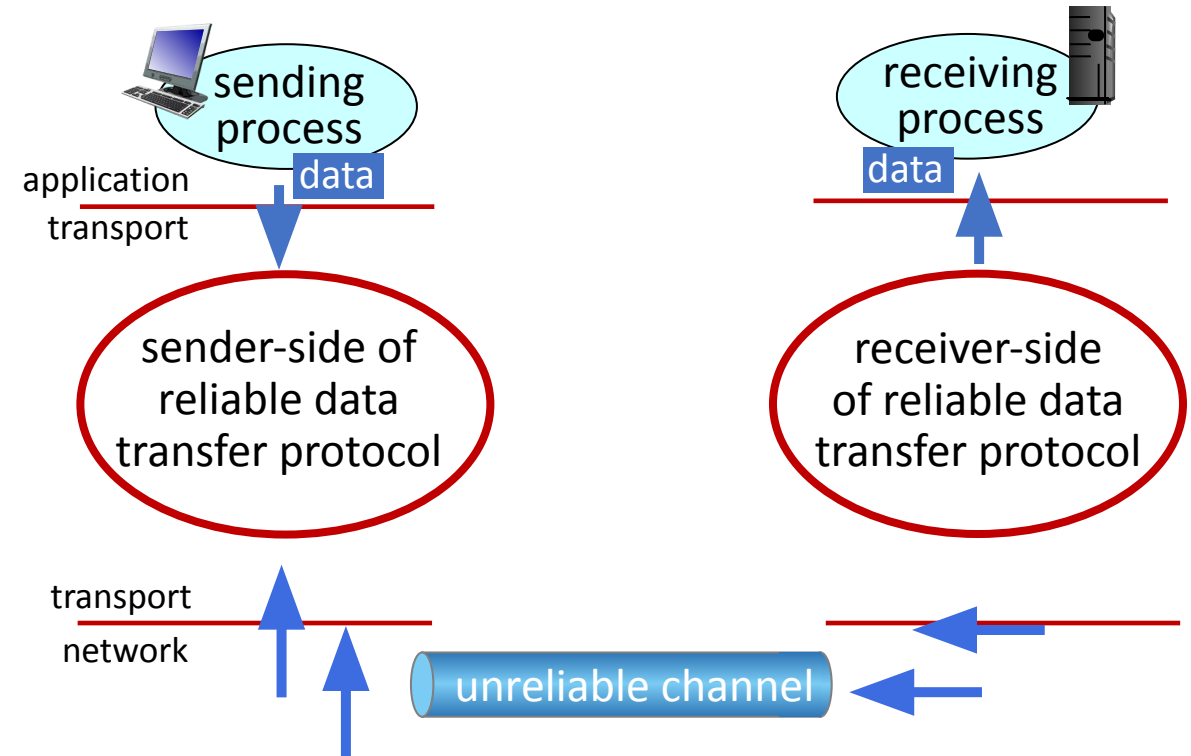
Principles of reliable data transfer



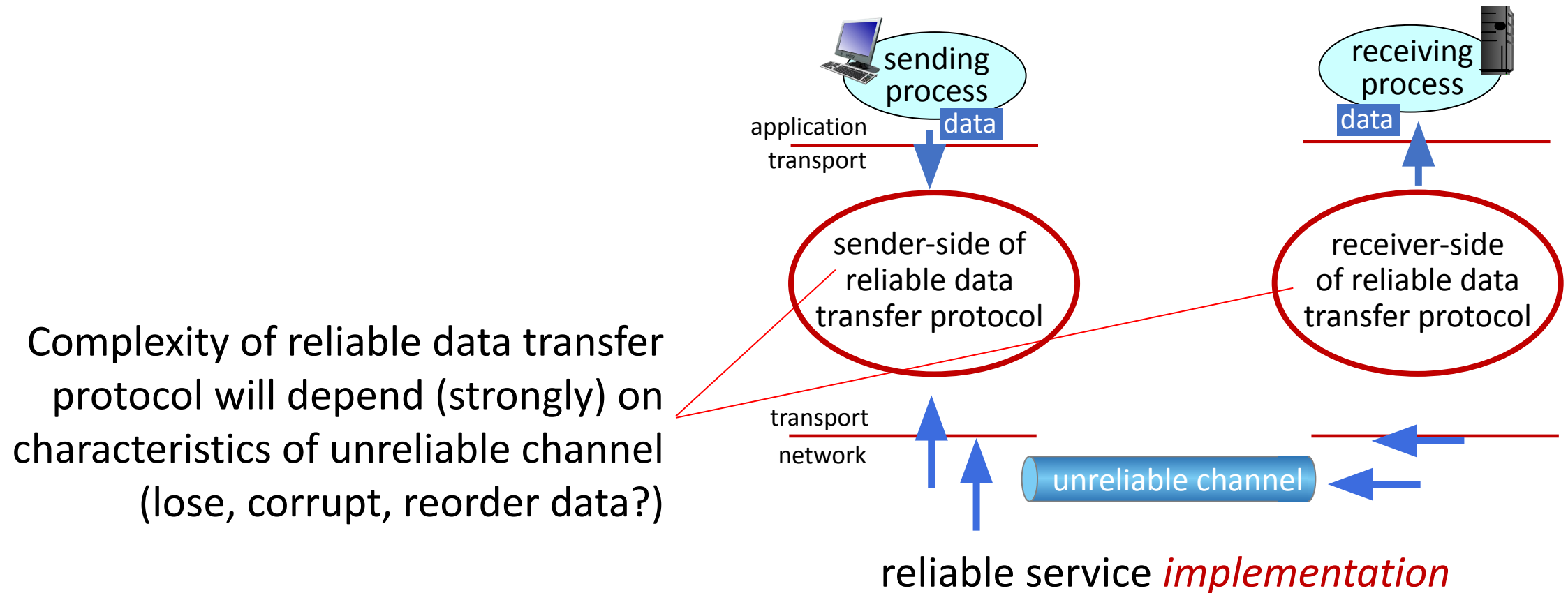
reliable service *abstraction*



reliable service *abstraction*

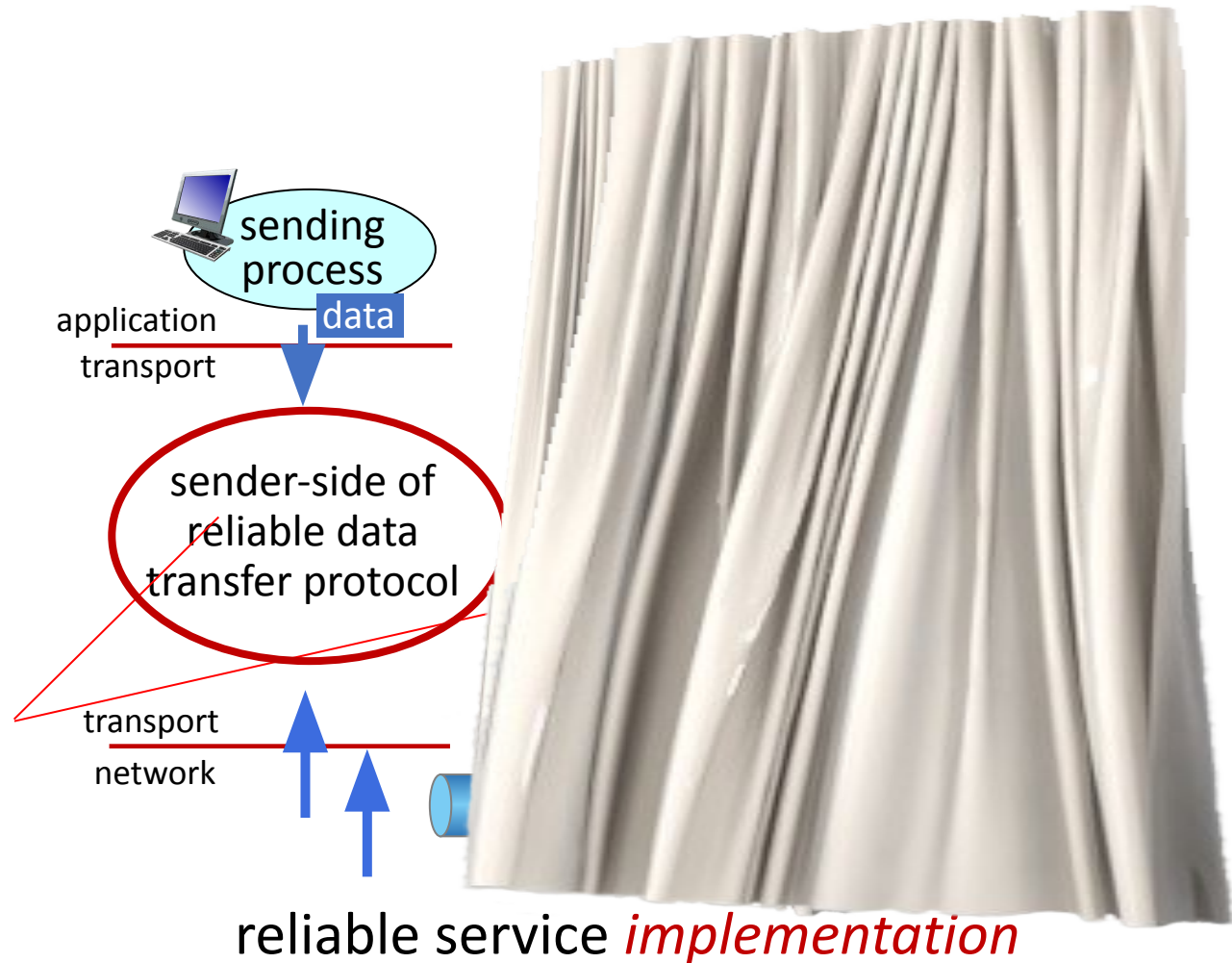


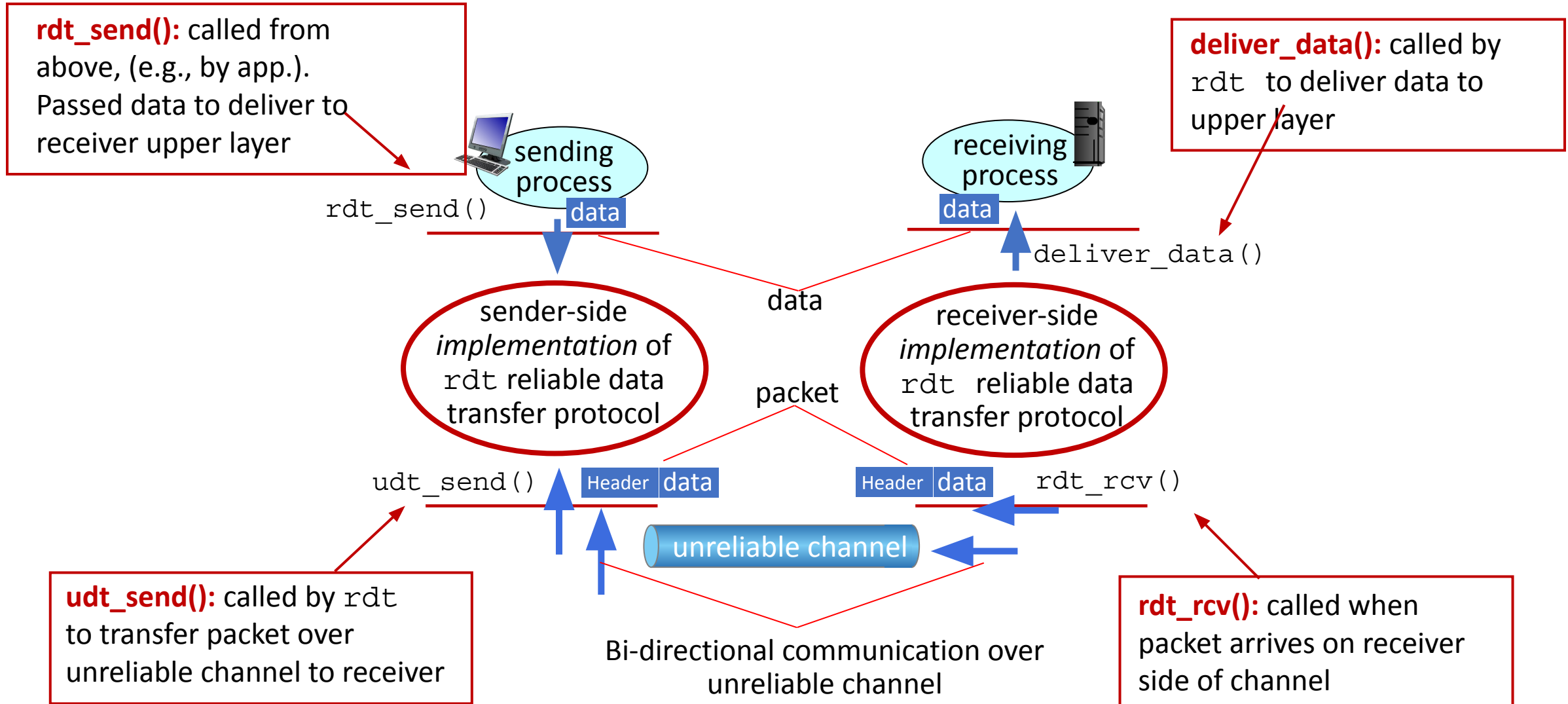
reliable service *implementation*



Sender, receiver do *not* know the “state” of each other, e.g., was a message received?

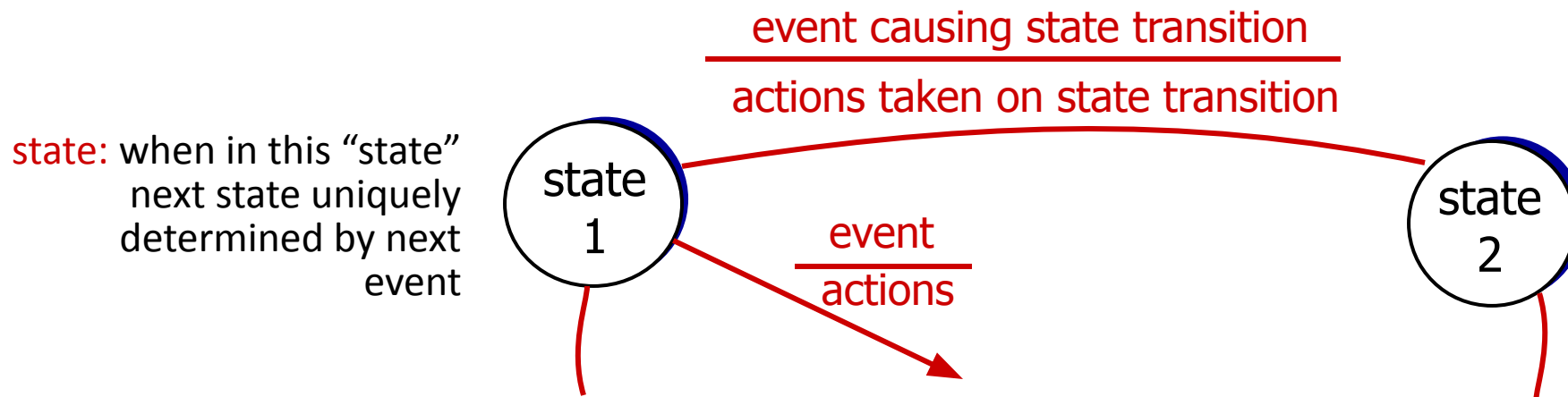
- unless communicated via a message



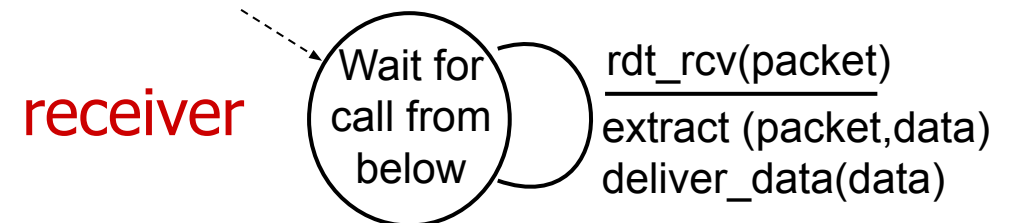
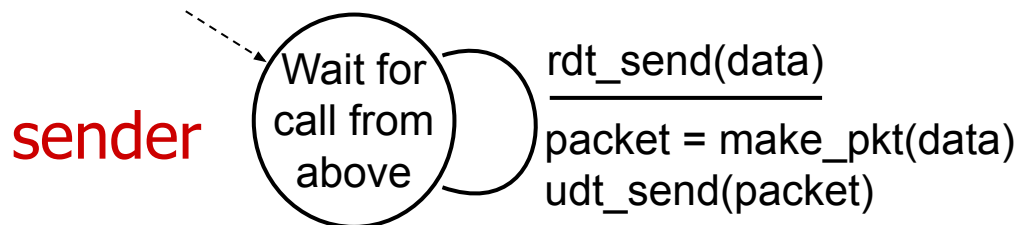


We will:

- incrementally develop sender, receiver sides of reliable data transfer protocol (rdt)
- consider only unidirectional data transfer
 - but control info will flow in both directions!
- use finite state machines (FSM) to specify sender, receiver



- underlying channel perfectly reliable
 - no bit errors
 - no loss of packets
- *separate* FSMs for sender, receiver:
 - sender sends data into underlying channel
 - receiver reads data from underlying channel



- underlying channel may flip bits in packet
 - checksum (e.g., Internet checksum) to detect bit errors
- *the* question: how to recover from errors?

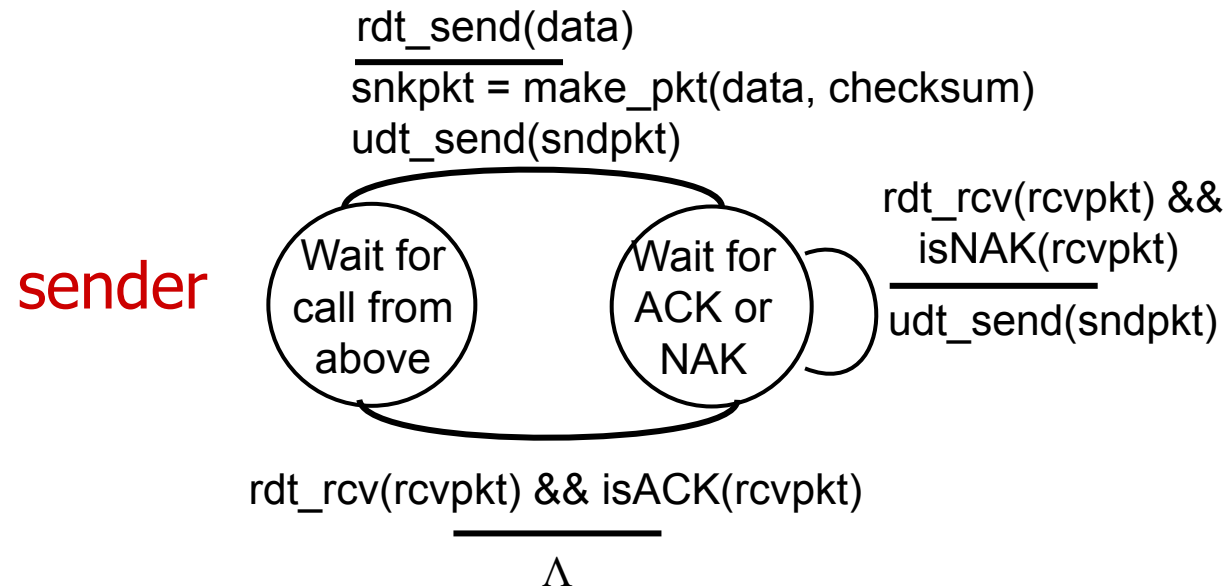
How do humans recover from “errors” during conversation?

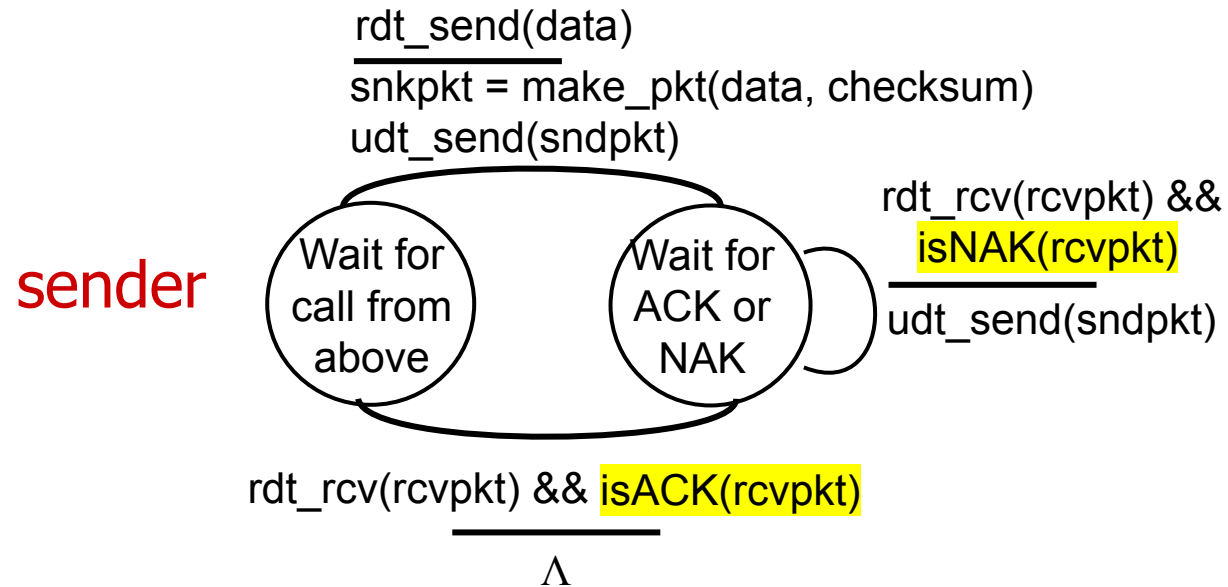
- underlying channel may flip bits in packet
 - checksum to detect bit errors
- *the question: how to recover from errors?*
 - *acknowledgements (ACKs)*: receiver explicitly tells sender that pkt received OK
 - *negative acknowledgements (NAKs)*: receiver explicitly tells sender that pkt had errors
 - sender *retransmits* pkt on receipt of NAK

**Automatic Repeat
reQuest**

stop and wait

sender sends one packet, then waits for receiver response



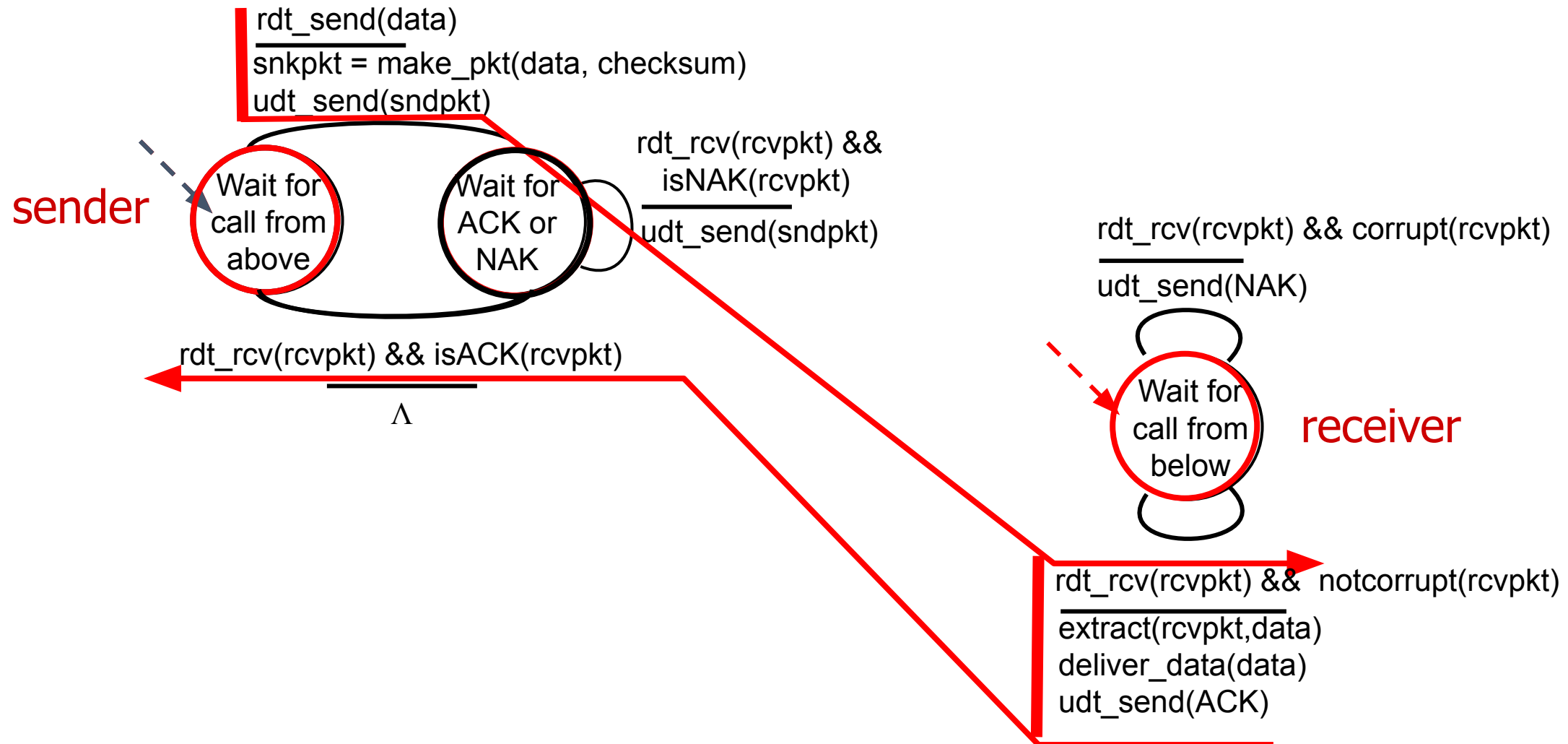


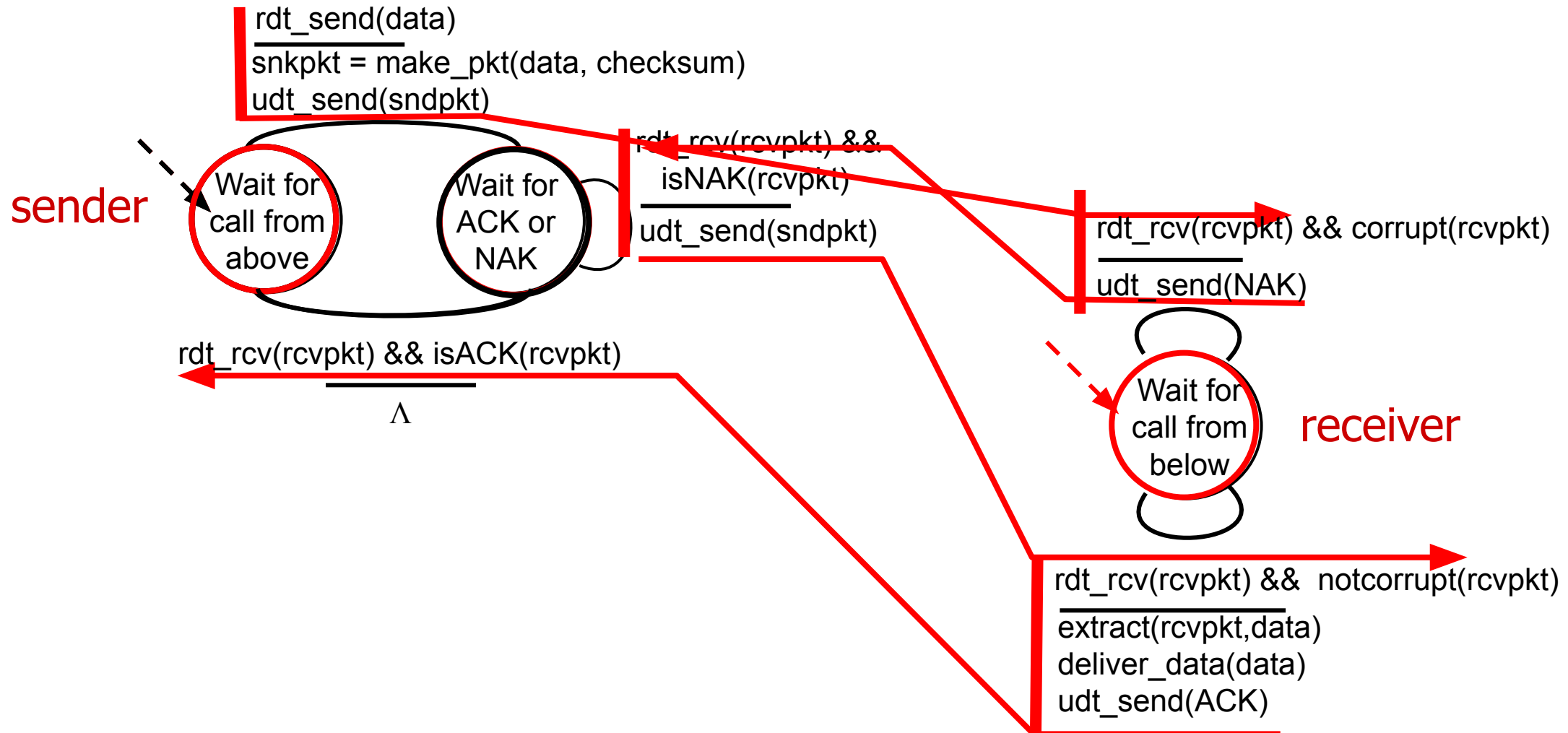
stop-and-wait

Note: “state” of receiver (did the receiver get my message correctly?) isn’t known to sender unless somehow communicated from receiver to sender

- that’s why we need a protocol!







what happens if ACK/NAK corrupted?

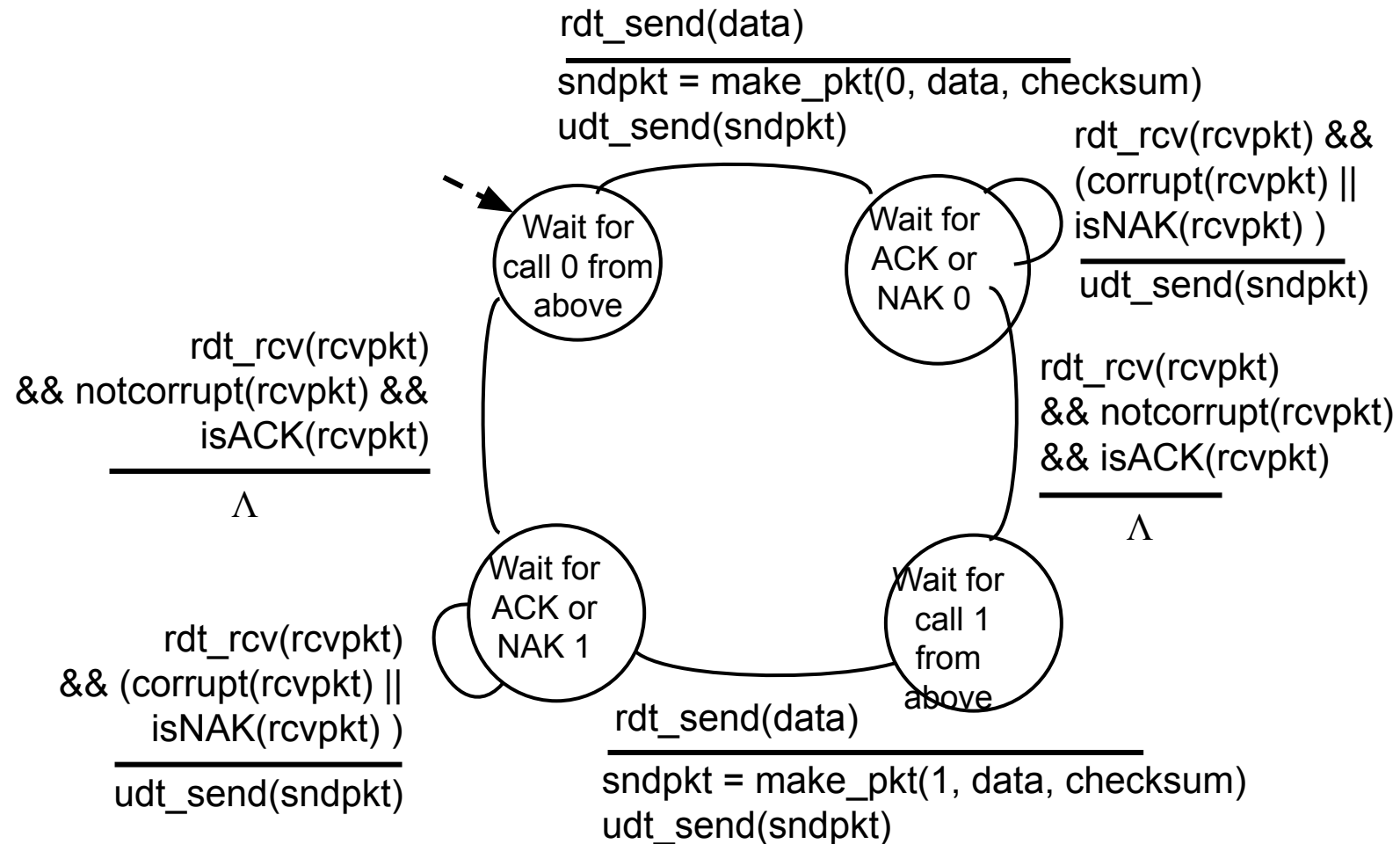
- sender doesn't know what happened at receiver!
- can't just retransmit: possible duplicate

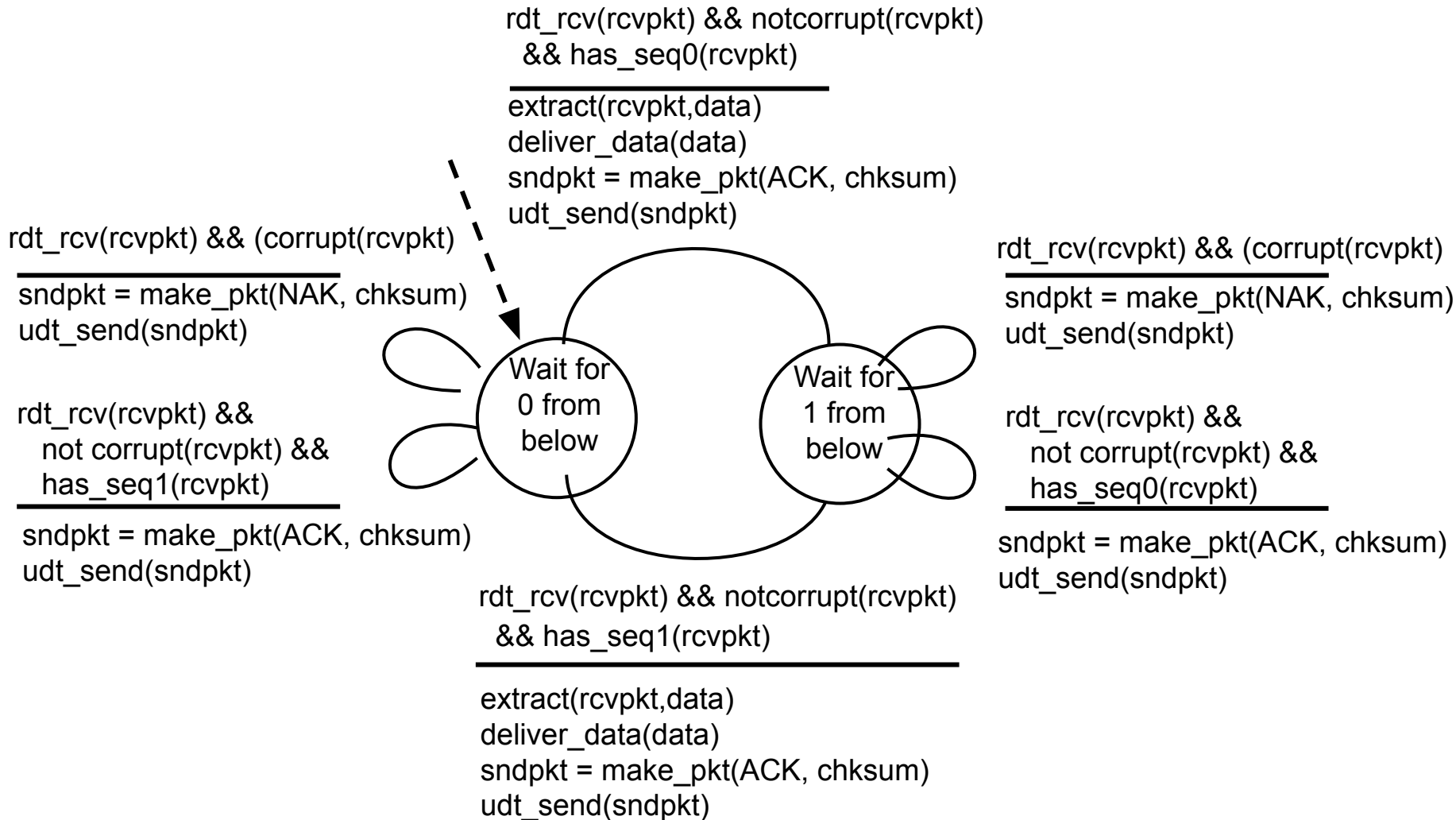
handling duplicates:

- sender retransmits current pkt if ACK/NAK corrupted
- sender adds *sequence number* to each pkt
- receiver discards (doesn't deliver up) duplicate pkt

stop and wait

sender sends one packet, then waits for receiver response





sender:

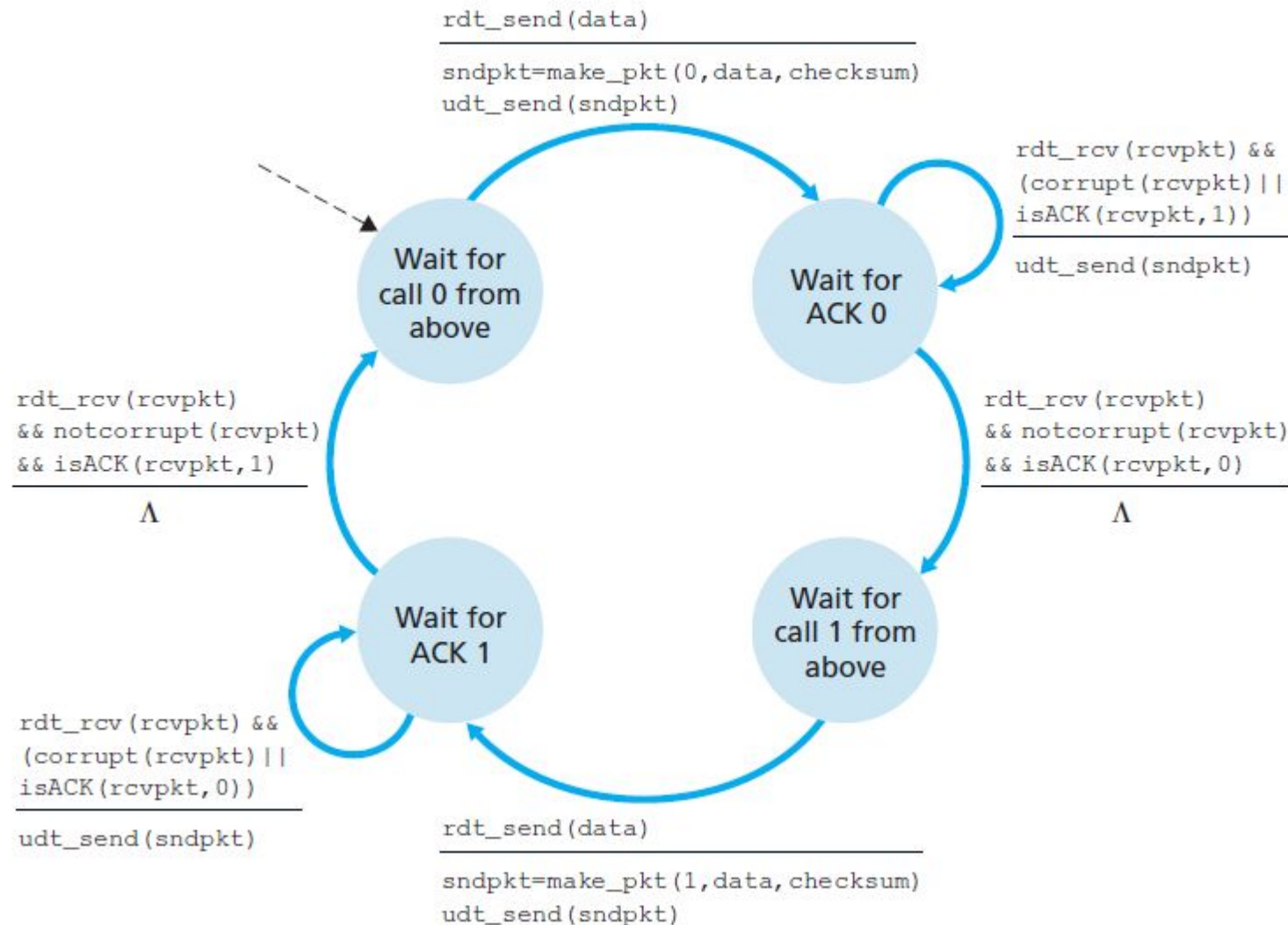
- seq # added to pkt
- two seq. #s (0,1) will suffice.
Why?
- must check if received ACK/NAK corrupted
- twice as many states
 - state must “remember” whether “expected” pkt should have seq # of 0 or 1

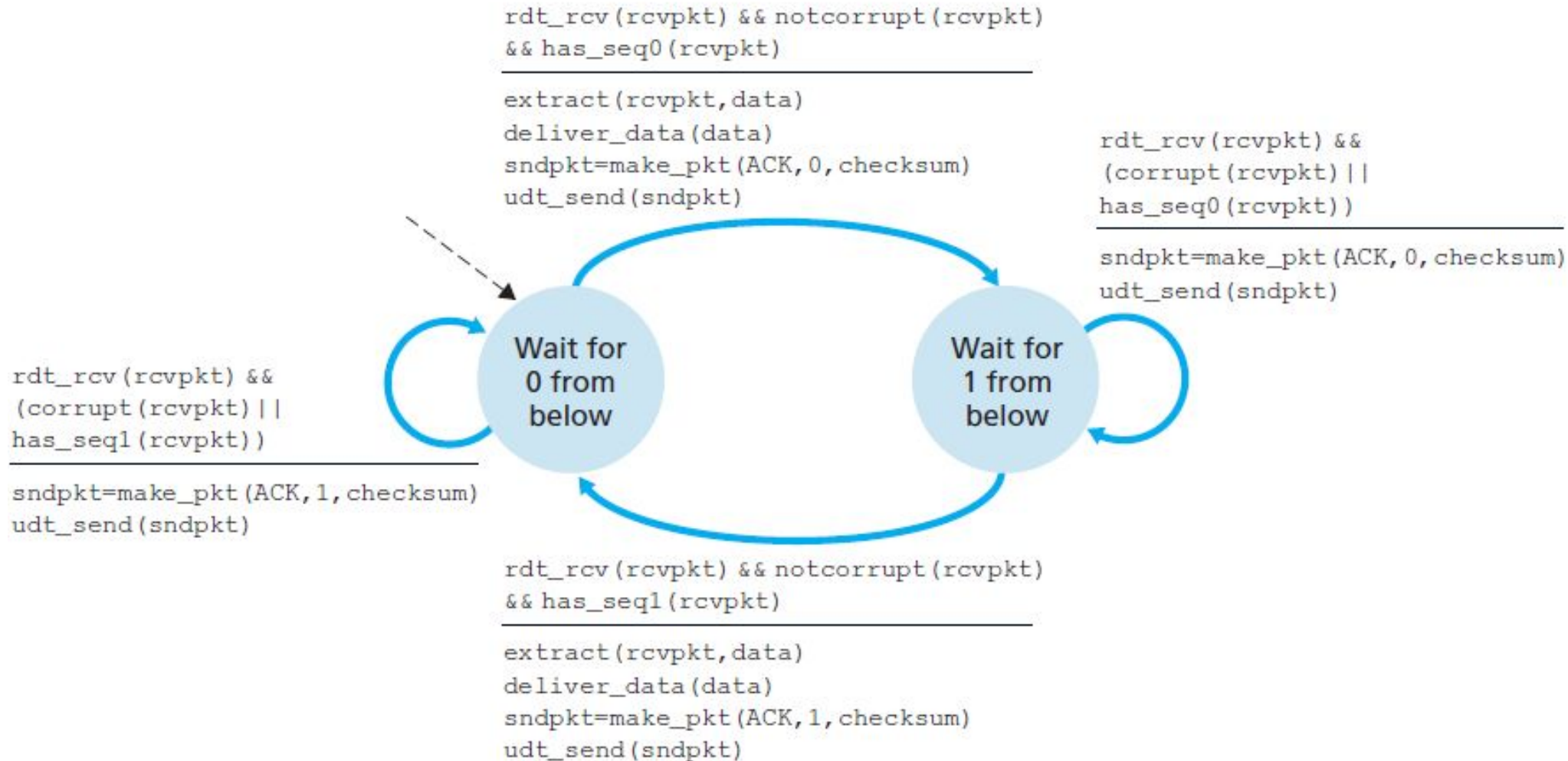
receiver:

- must check if received packet is duplicate
 - state indicates whether 0 or 1 is expected pkt seq #
- note: receiver can *not* know if its last ACK/NAK received OK at sender

- same functionality as rdt2.1, using ACKs only
- instead of NAK, receiver sends ACK for last pkt received OK
 - receiver must *explicitly* include seq # of pkt being ACKed
- duplicate ACK at sender results in same action as NAK:
retransmit current pkt

As we will see, TCP uses this approach to be NAK-free





New channel assumption: underlying channel can also lose packets (data, ACKs)

- checksum, sequence #s, ACKs, retransmissions will be of help ... but not quite enough

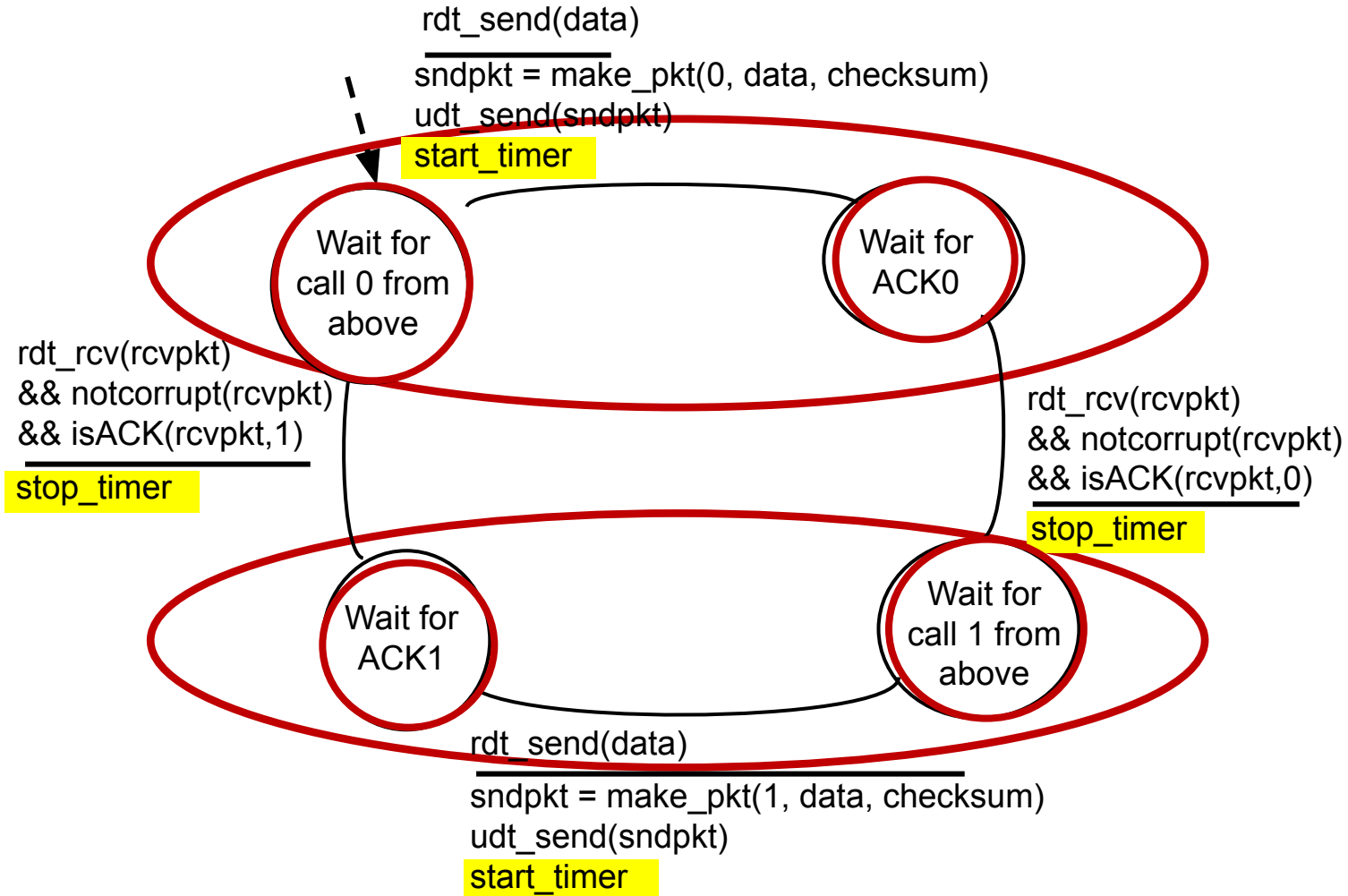
Q: How do *humans* handle lost sender-to-receiver words in conversation?

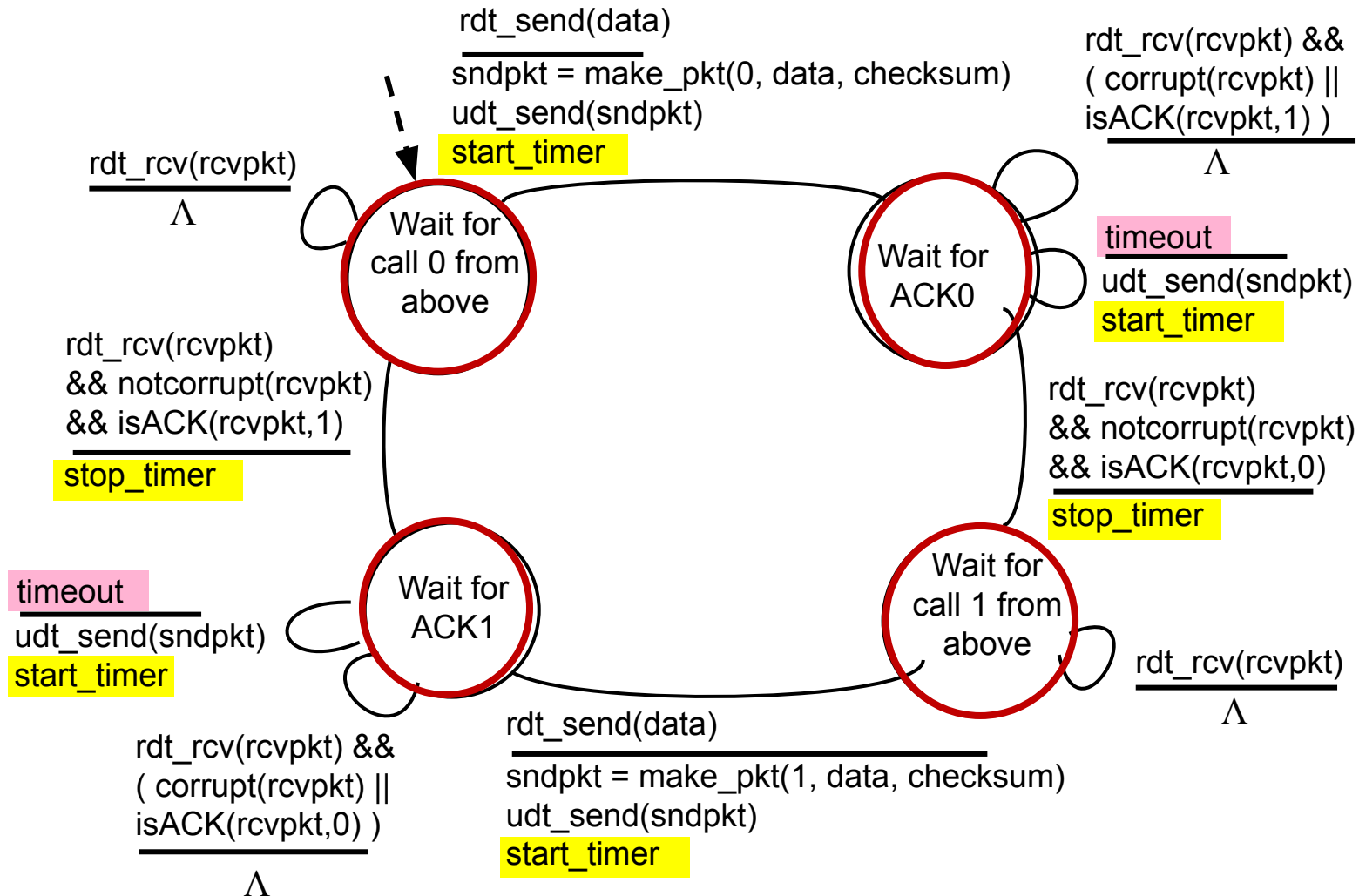
Approach: sender waits “reasonable” amount of time for ACK

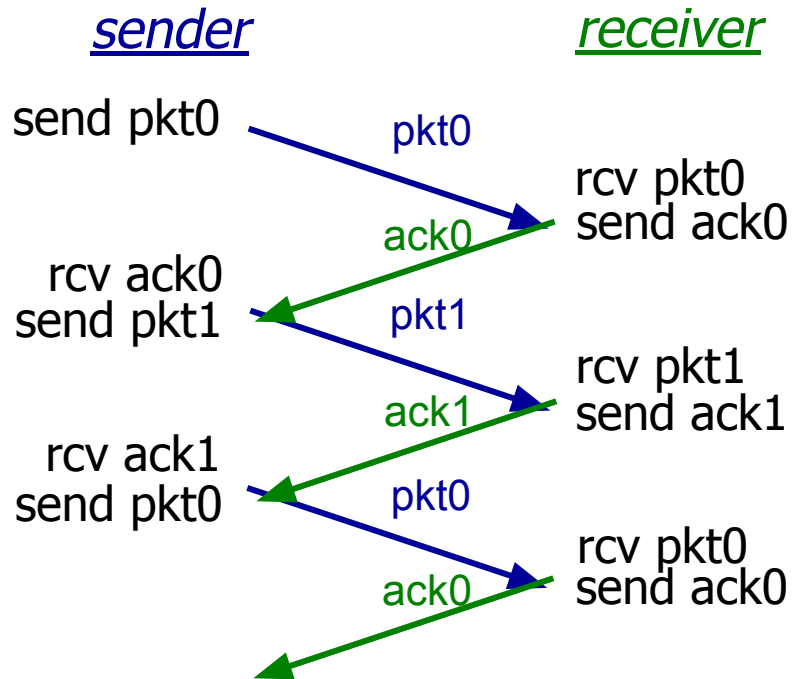
- retransmits if no ACK received in this time
- if pkt (or ACK) just delayed (not lost):
 - retransmission will be duplicate, but seq #s already handles this!
 - receiver must specify seq # of packet being ACKed
- use countdown timer to interrupt after “reasonable” amount of time



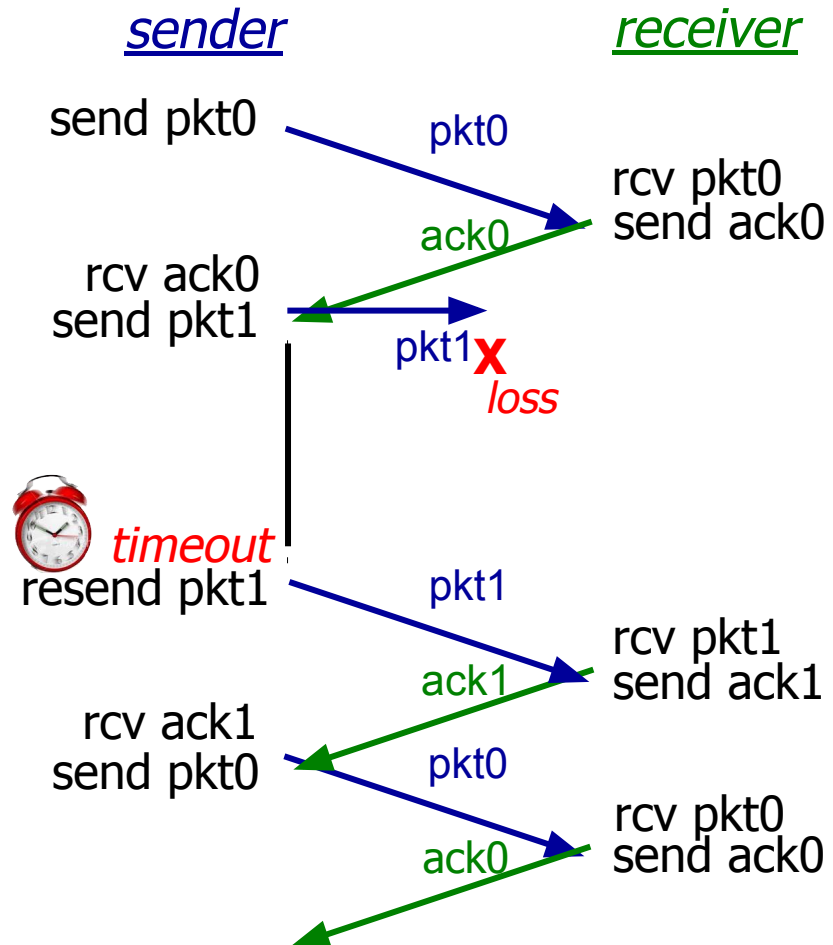
timeout



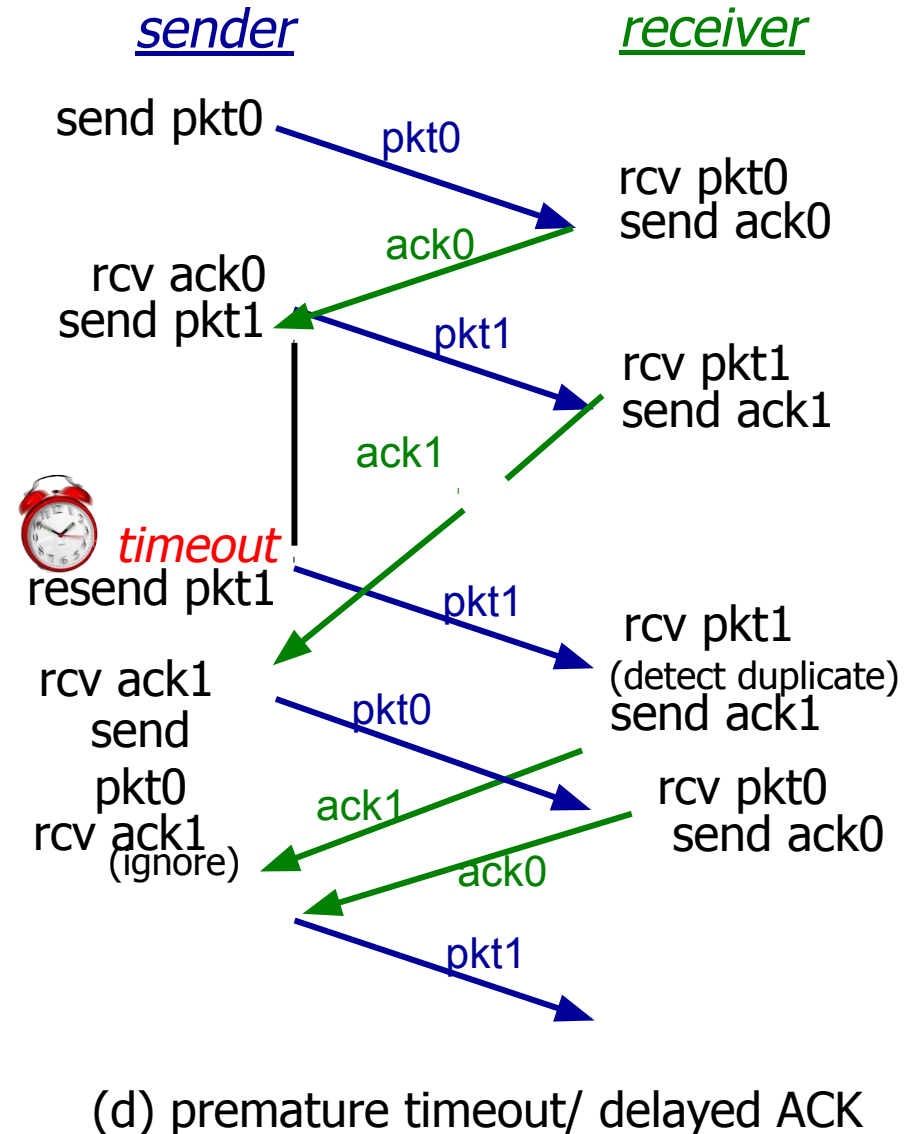
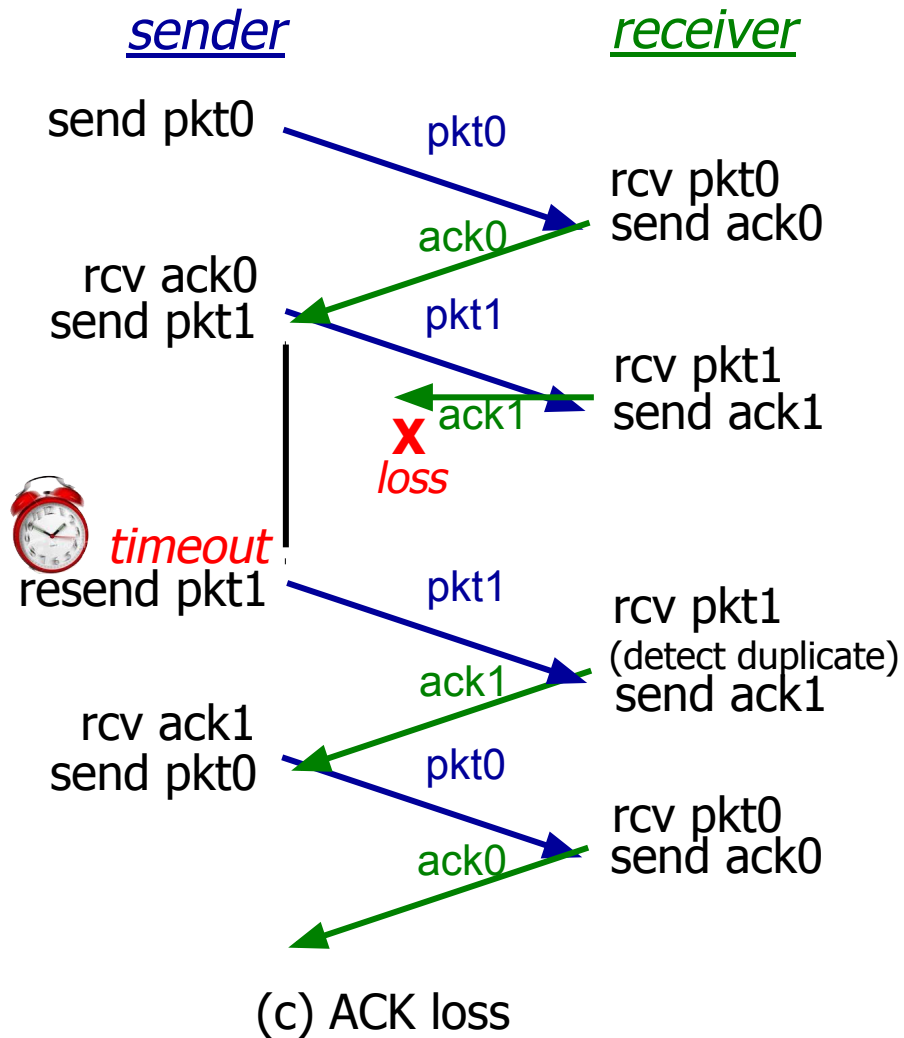




(a) no loss

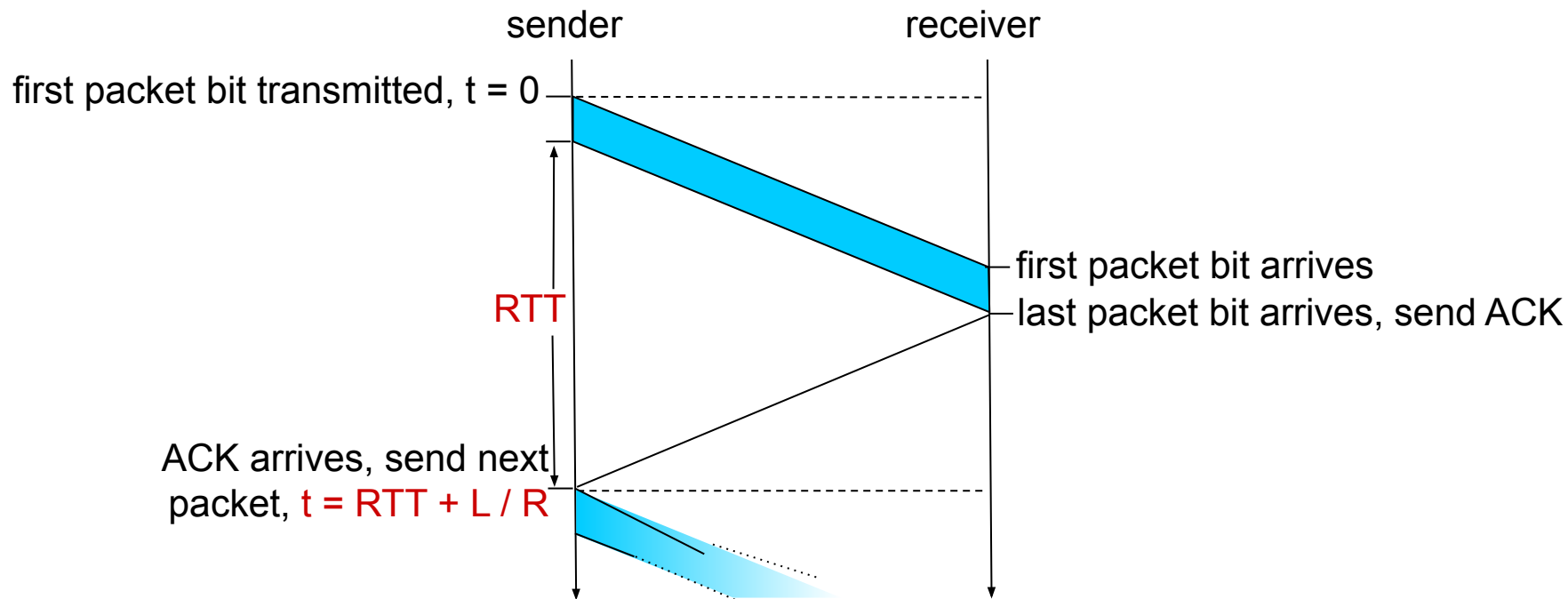


(b) packet loss

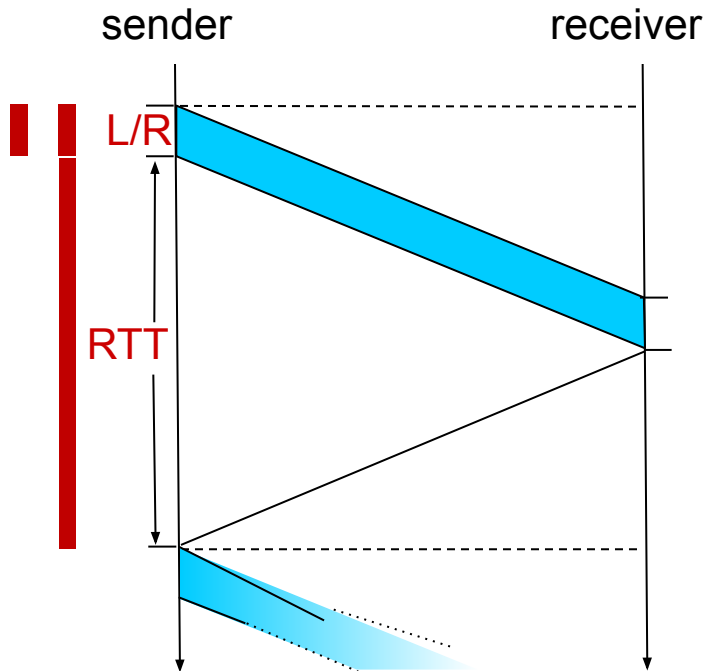


- U_{sender} : *utilization* – fraction of time sender busy sending
- example: 1 Gbps link, 15 ms prop. delay, 8000 bit packet
 - time to transmit packet into channel:

$$D_{trans} = \frac{L}{R} = \frac{8000 \text{ bits}}{10^9 \text{ bits/sec}} = 8 \text{ microsecs}$$



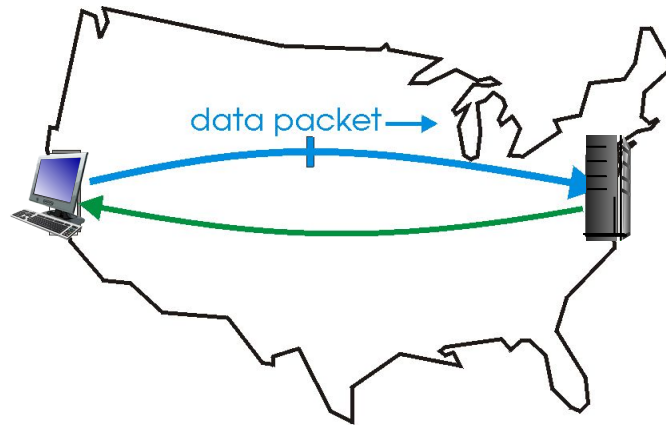
$$\begin{aligned}U_{\text{sender}} &= \frac{L / R}{RTT + L / R} \\&= \frac{.008}{30.008} \\&= 0.00027\end{aligned}$$



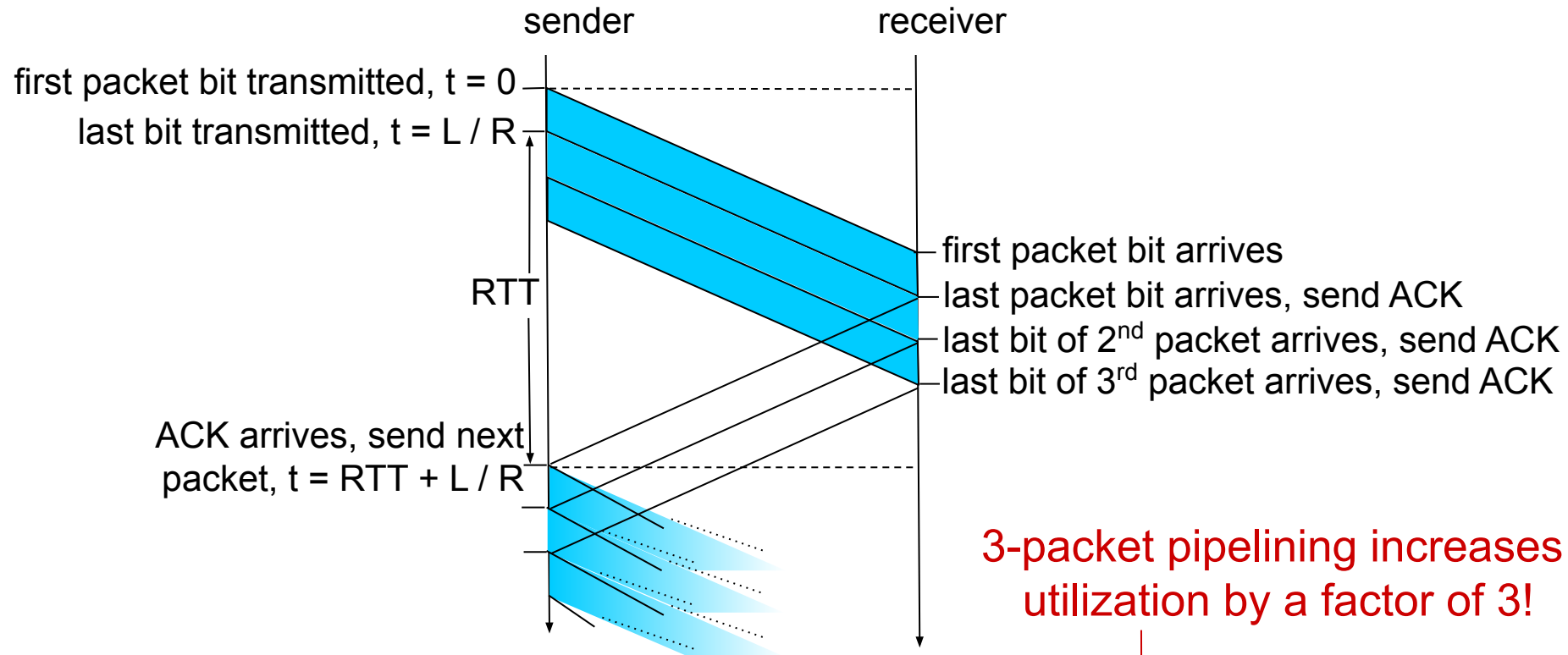
- rdt 3.0 protocol performance stinks!
- Protocol limits performance of underlying infrastructure (channel)

pipelining: sender allows multiple, “in-flight”, yet-to-be-acknowledged packets

- range of sequence numbers must be increased
- buffering at sender and/or receiver



(a) a stop-and-wait protocol in operation



3-packet pipelining increases utilization by a factor of 3!

$$U_{\text{sender}} = \frac{3L / R}{RTT + L / R} = \frac{.0024}{30.008} = 0.00081$$



Thank You
For Your Attention



THANK YOU

Department of Computer Science and Engineering